

# Physlib Monthly Report

1–31 July 2026

**Contributors:** Aleksei Bykov, Alex Zughaid, Andrea Pari, August, Augustinas Malinauskas, doxtor6, Florian Wiesner, Giuseppe Barbalinardo, giuseppe.sorge, Gregory Loges, Jake Uribe, Joseph Tooby-Smith, juanjfndz, Kenneth Jusino, KrystianNowakowski, Lazar Milikic, nateabr, Nicola Bernini, Nicolas Rouquette, Philip Haynes, Robby Sneiderman, Robert Joseph, Shlok Vaibhav Singh, Tracy McSheery, Vasily Ilin

**Reviewers:** Daniel Morrison, doxtor6, Gregory Loges, Joseph Tooby-Smith, nateabr, Zhi Kai Pong

Generated 14 August 2026 from commit 471cf7f.

## Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–31 July 2026 on branch `master` of [leanprover-community/physlib](#), between commits `05930fe` and `471cf7f`. Over that period 25 contributors landed 108 commits, changing 36,698 lines across 356 files and adding 962 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

## Contents

|          |                                                                                 |          |
|----------|---------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                             | <b>2</b> |
| 1.1      | Contributors . . . . .                                                          | 2        |
| 1.2      | Reviewers . . . . .                                                             | 2        |
| 1.3      | Modified subfolders . . . . .                                                   | 3        |
| 1.4      | New declarations by kind . . . . .                                              | 3        |
| <b>2</b> | <b>Details by file</b>                                                          | <b>4</b> |
| 2.1      | Physlib.ClassicalMechanics.DampedHarmonicOscillator.Basic . . . . .             | 4        |
| 2.2      | Physlib.ClassicalMechanics.DampedHarmonicOscillator.Solution . . . . .          | 6        |
| 2.3      | Physlib.ClassicalMechanics.HarmonicOscillator.Geometric.Basic . . . . .         | 7        |
| 2.4      | Physlib.ClassicalMechanics.HarmonicOscillator.Geometric.KineticEnergy . . . . . | 8        |
| 2.5      | Physlib.ClassicalMechanics.HarmonicOscillator.Geometric.Trajectory . . . . .    | 11       |
| 2.6      | Physlib.ClassicalMechanics.Lagrangian.TotalDerivativeEquivalence . . . . .      | 12       |
| 2.7      | Physlib.ClassicalMechanics.RigidBody.AngularMomentum . . . . .                  | 13       |
| 2.8      | Physlib.ClassicalMechanics.RigidBody.AngularVelocity . . . . .                  | 13       |
| 2.9      | Physlib.ClassicalMechanics.RigidBody.Basic . . . . .                            | 17       |
| 2.10     | Physlib.ClassicalMechanics.RigidBody.KineticEnergy . . . . .                    | 18       |
| 2.11     | Physlib.ClassicalMechanics.RigidBody.Motion . . . . .                           | 20       |
| 2.12     | Physlib.CondensedMatter.Thermoelectric.Basic . . . . .                          | 24       |
| 2.13     | Physlib.FluidDynamics.CauchyFlow.Basic . . . . .                                | 26       |
| 2.14     | Physlib.FluidDynamics.CauchyFlow.BodyForce . . . . .                            | 26       |

|      |                                                                                  |    |
|------|----------------------------------------------------------------------------------|----|
| 2.15 | Physlib.FluidDynamics.CauchyFlow.Inviscid . . . . .                              | 26 |
| 2.16 | Physlib.FluidDynamics.CauchyFlow.Momentum . . . . .                              | 27 |
| 2.17 | Physlib.FluidDynamics.CauchyFlow.NavierStokes . . . . .                          | 28 |
| 2.18 | Physlib.FluidDynamics.CauchyFlow.Newtonian . . . . .                             | 28 |
| 2.19 | Physlib.FluidDynamics.Euler.Basic . . . . .                                      | 28 |
| 2.20 | Physlib.FluidDynamics.FluidFlow.Basic . . . . .                                  | 29 |
| 2.21 | Physlib.FluidDynamics.FluidFlow.Continuity . . . . .                             | 29 |
| 2.22 | Physlib.FluidDynamics.FluidFlow.Incompressible . . . . .                         | 30 |
| 2.23 | Physlib.FluidDynamics.FluidFlow.Kinematics . . . . .                             | 31 |
| 2.24 | Physlib.FluidDynamics.FluidFlow.Momentum . . . . .                               | 31 |
| 2.25 | Physlib.FluidDynamics.FluidFlow.Newtonian . . . . .                              | 32 |
| 2.26 | Physlib.FluidDynamics.ThermodynamicCauchyFlow.Basic . . . . .                    | 33 |
| 2.27 | Physlib.FluidDynamics.ThermodynamicCauchyFlow.Bernoulli . . . . .                | 33 |
| 2.28 | Physlib.FluidDynamics.ThermodynamicCauchyFlow.Isentropic . . . . .               | 34 |
| 2.29 | Physlib.Mathematics.CrossProduct . . . . .                                       | 34 |
| 2.30 | Physlib.Mathematics.CrossProductMatrix . . . . .                                 | 34 |
| 2.31 | Physlib.Mathematics.Distribution.Basic . . . . .                                 | 35 |
| 2.32 | Physlib.Mathematics.InnerProductSpace.Gaussian . . . . .                         | 36 |
| 2.33 | Physlib.Mathematics.KroneckerDelta.Basic . . . . .                               | 37 |
| 2.34 | Physlib.Mathematics.KroneckerDelta.Contraction . . . . .                         | 37 |
| 2.35 | Physlib.Mathematics.OrthogonalMatrix . . . . .                                   | 40 |
| 2.36 | Physlib.Mathematics.Resolvent . . . . .                                          | 40 |
| 2.37 | Physlib.Mathematics.SpecialFunctions.Physhermite . . . . .                       | 41 |
| 2.38 | Physlib.Particles.StandardModel.Basic . . . . .                                  | 47 |
| 2.39 | Physlib.Particles.StandardModel.Fermions.DownSinglet . . . . .                   | 48 |
| 2.40 | Physlib.Particles.StandardModel.Fermions.LeptonDoublet . . . . .                 | 50 |
| 2.41 | Physlib.Particles.StandardModel.Fermions.LeptonSinglet . . . . .                 | 53 |
| 2.42 | Physlib.Particles.StandardModel.Fermions.QuarkDoublet . . . . .                  | 55 |
| 2.43 | Physlib.Particles.StandardModel.Fermions.UpSinglet . . . . .                     | 57 |
| 2.44 | Physlib.QuantumMechanics.FreeParticle.Basic . . . . .                            | 60 |
| 2.45 | Physlib.QuantumMechanics.HarmonicOscillator.Basic . . . . .                      | 61 |
| 2.46 | Physlib.QuantumMechanics.HarmonicOscillator.OneDimension.Basic . . . . .         | 64 |
| 2.47 | Physlib.QuantumMechanics.HilbertSpaces.SpaceD.Basic . . . . .                    | 65 |
| 2.48 | Physlib.QuantumMechanics.HilbertSpaces.SpaceD.DirichletSubmodule . . . . .       | 69 |
| 2.49 | Physlib.QuantumMechanics.HilbertSpaces.SpaceD.Fourier . . . . .                  | 70 |
| 2.50 | Physlib.QuantumMechanics.HilbertSpaces.SpaceD.PolyBddSchwartzSubmodule . . . . . | 71 |
| 2.51 | Physlib.QuantumMechanics.HilbertSpaces.SpaceD.SchwartzSubmodule . . . . .        | 71 |
| 2.52 | Physlib.QuantumMechanics.HilbertSpaces.SpaceD.SobolevSubmodule . . . . .         | 72 |
| 2.53 | Physlib.QuantumMechanics.InfiniteSquareWell.Basic . . . . .                      | 73 |
| 2.54 | Physlib.QuantumMechanics.Operators.Multiplication . . . . .                      | 74 |
| 2.55 | Physlib.QuantumMechanics.Operators.OneDimension.Momentum . . . . .               | 77 |
| 2.56 | Physlib.QuantumMechanics.Operators.OneDimension.Parity . . . . .                 | 78 |
| 2.57 | Physlib.QuantumMechanics.Operators.OneDimension.Position . . . . .               | 78 |
| 2.58 | Physlib.QuantumMechanics.Operators.OneDimension.Unbounded . . . . .              | 78 |
| 2.59 | Physlib.QuantumMechanics.Operators.SpectralTheory.SelfAdjoint . . . . .          | 78 |
| 2.60 | Physlib.QuantumMechanics.Operators.SpectralTheory.Symmetric . . . . .            | 79 |
| 2.61 | Physlib.QuantumMechanics.Operators.Unbounded . . . . .                           | 79 |
| 2.62 | Physlib.QuantumMechanics.QuantumSystem.Basic . . . . .                           | 81 |
| 2.63 | Physlib.QuantumMechanics.RectangularBarrier.Basic . . . . .                      | 83 |
| 2.64 | Physlib.Relativity.Fermions.Dirac.Basic . . . . .                                | 85 |
| 2.65 | Physlib.Relativity.Fermions.Weyl.DualLeftHanded . . . . .                        | 86 |

|       |                                                                                    |     |
|-------|------------------------------------------------------------------------------------|-----|
| 2.66  | Physlib.Relativity.Fermions.Weyl.DualRightHanded . . . . .                         | 87  |
| 2.67  | Physlib.Relativity.Fermions.Weyl.Duals . . . . .                                   | 89  |
| 2.68  | Physlib.Relativity.Fermions.Weyl.LeftHanded . . . . .                              | 90  |
| 2.69  | Physlib.Relativity.Fermions.Weyl.RightHanded . . . . .                             | 91  |
| 2.70  | Physlib.Relativity.LorentzGroup.Basic . . . . .                                    | 92  |
| 2.71  | Physlib.Relativity.LorentzGroup.Restricted.FromBoostRotation . . . . .             | 93  |
| 2.72  | Physlib.Relativity.Tensors.Basic . . . . .                                         | 93  |
| 2.73  | Physlib.Relativity.Tensors.ComplexTensor.Metrics.Basic . . . . .                   | 94  |
| 2.74  | Physlib.Relativity.Tensors.ComplexTensor.Units.Basic . . . . .                     | 94  |
| 2.75  | Physlib.Relativity.Tensors.Conjugation.Basic . . . . .                             | 95  |
| 2.76  | Physlib.Relativity.Tensors.Contraction.CrossToEnd . . . . .                        | 95  |
| 2.77  | Physlib.Relativity.Tensors.Contraction.CrossToSlot . . . . .                       | 98  |
| 2.78  | Physlib.Relativity.Tensors.Contraction.Products . . . . .                          | 99  |
| 2.79  | Physlib.Relativity.Tensors.Contraction.Pure . . . . .                              | 100 |
| 2.80  | Physlib.Relativity.Tensors.Contraction.SuccSuccAbove . . . . .                     | 101 |
| 2.81  | Physlib.Relativity.Tensors.Contraction.UnitTensorContraction . . . . .             | 101 |
| 2.82  | Physlib.Relativity.Tensors.Dual . . . . .                                          | 104 |
| 2.83  | Physlib.Relativity.Tensors.Elab . . . . .                                          | 106 |
| 2.84  | Physlib.Relativity.Tensors.Evaluation . . . . .                                    | 109 |
| 2.85  | Physlib.Relativity.Tensors.LeviCivita.Basic . . . . .                              | 110 |
| 2.86  | Physlib.Relativity.Tensors.LeviCivita.Contractions . . . . .                       | 110 |
| 2.87  | Physlib.Relativity.Tensors.RealTensor.Representation.Contraction . . . . .         | 111 |
| 2.88  | Physlib.Relativity.Tensors.Reindexing . . . . .                                    | 112 |
| 2.89  | Physlib.Relativity.Tensors.TensorSpecies.Basic . . . . .                           | 113 |
| 2.90  | Physlib.SpaceAndTime.Time.Derivatives . . . . .                                    | 114 |
| 2.91  | Physlib.SpaceAndTime.Time.MatrixDerivatives . . . . .                              | 114 |
| 2.92  | Physlib.Units.Dimension . . . . .                                                  | 115 |
| 2.93  | Physlib.Units.ISQBridge . . . . .                                                  | 117 |
| 2.94  | Physlib.Units.ISQDimensionBase . . . . .                                           | 119 |
| 2.95  | Physlib.Units.LTMCTDimensionBase . . . . .                                         | 120 |
| 2.96  | Physlib.Units.ParametricDimensionExamples . . . . .                                | 122 |
| 2.97  | Physlib.Units.ParametricUnits . . . . .                                            | 123 |
| 2.98  | Physlib.Units.UnitDependent . . . . .                                              | 124 |
| 2.99  | Physlib.Units.WithDim.Basic . . . . .                                              | 124 |
| 2.100 | PhyslibAlpha.ClassicalFieldTheory.Local.EulerLagrangeEquation . . . . .            | 126 |
| 2.101 | PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.ChargeBalance . . . . .       | 127 |
| 2.102 | PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.EffectivePotential . . . . .  | 127 |
| 2.103 | PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.GaugeSlice . . . . .          | 128 |
| 2.104 | PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.GaugeTorus . . . . .          | 129 |
| 2.105 | PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.Invariants . . . . .          | 130 |
| 2.106 | PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.Module . . . . .              | 138 |
| 2.107 | PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.OrbitRepresentative . . . . . | 139 |
| 2.108 | PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.SwapDoublet . . . . .         | 140 |
| 2.109 | QuantumInfo.ClassicalInfo.Prob . . . . .                                           | 141 |
| 2.110 | QuantumInfo.Entropy.DPI . . . . .                                                  | 141 |
| 2.111 | QuantumInfo.Entropy.Relative . . . . .                                             | 142 |
| 2.112 | scripts.autolabel . . . . .                                                        | 143 |

# 1 Introduction

During Jul 2026, 25 contributors submitted 108 commits that changed 36,698 lines (+23,053 / -13,645) across 356 files spanning 36 top-level areas of the repository. Of those, 93 files were newly added and 8 were removed outright. This report catalogues 962 newly added Lean declarations: 615 lemmas, 214 defs, 58 instances, 20 structures, 18 theorems, 17 abbrevs, 9 informal lemmas, 7 informal definitions, and 4 inductives. We're delighted to recognize the following first-time contributors this month: [Alex Zughaid](#), [August](#), [Augustinas Malinauskas](#), [Giuseppe Barbalinardo](#), [Jake Uribe](#), [Kenneth Jusino](#), [KrystianNowakowski](#), [Nicolas Rouquette](#), [Philip Haynes](#), and [Robert Joseph](#).

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

## 1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

| Contributor                            | Lines Changed | Commits |
|----------------------------------------|---------------|---------|
| <a href="#">Aleksei Bykov</a>          | 179           | 2       |
| <a href="#">Alex Zughaid</a>           | 4             | 1       |
| <a href="#">Andrea Pari</a>            | 1,495         | 1       |
| <a href="#">August</a>                 | 560           | 7       |
| <a href="#">Augustinas Malinauskas</a> | 31            | 1       |
| <a href="#">doxtor6</a>                | 461           | 4       |
| <a href="#">Florian Wiesner</a>        | 1,323         | 1       |
| <a href="#">Giuseppe Barbalinardo</a>  | 214           | 1       |
| <a href="#">giuseppe.sorge</a>         | 1,590         | 12      |
| <a href="#">Gregory Loges</a>          | 3,947         | 17      |
| <a href="#">Jake Uribe</a>             | 23            | 2       |
| <a href="#">Joseph Tooby-Smith</a>     | 6,131         | 18      |
| <a href="#">juanjfndz</a>              | 101           | 1       |
| <a href="#">Kenneth Jusino</a>         | 68            | 2       |
| <a href="#">KrystianNowakowski</a>     | 24            | 1       |
| <a href="#">Lazar Milikic</a>          | 199           | 3       |
| <a href="#">nateabr</a>                | 1,322         | 5       |
| <a href="#">Nicola Bernini</a>         | 93            | 3       |
| <a href="#">Nicolas Rouquette</a>      | 1,669         | 2       |
| <a href="#">Philip Haynes</a>          | 221           | 1       |
| <a href="#">Robby Sneiderman</a>       | 1,684         | 11      |
| <a href="#">Robert Joseph</a>          | 38            | 1       |
| <a href="#">Shlok Vaibhav Singh</a>    | 827           | 4       |
| <a href="#">Tracy McSheery</a>         | 19            | 1       |
| <a href="#">Vasily Ilin</a>            | 15,181        | 6       |

## 1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

| Reviewer | PRs Reviewed |
|----------|--------------|
|----------|--------------|

|                    |    |
|--------------------|----|
| Daniel Morrison    | 14 |
| doxtor6            | 1  |
| Gregory Loges      | 11 |
| Joseph Tooby-Smith | 87 |
| nateabr            | 17 |
| Zhi Kai Pong       | 4  |

### 1.3 Modified subfolders

| Subfolder                         | Files | New | Deleted | Additions | Deletions |
|-----------------------------------|-------|-----|---------|-----------|-----------|
| Physlib/Relativity                | 60    | +11 | -2      | +4,186    | -2,287    |
| Physlib/QuantumMechanics          | 57    | +16 | -5      | +3,178    | -1,368    |
| Physlib/QFT                       | 29    | —   | —       | +1,080    | -2,777    |
| Physlib/Particles                 | 34    | +6  | —       | +1,952    | -1,203    |
| Physlib/SpaceAndTime              | 22    | +6  | —       | +1,148    | -1,826    |
| Physlib/ClassicalMechanics        | 22    | +10 | —       | +2,363    | -595      |
| Physlib/Mathematics               | 19    | +5  | —       | +1,478    | -870      |
| PhyslibAlpha/Particles            | 8     | +8  | —       | +1,736    | -0        |
| scripts                           | 5     | +3  | —       | +1,687    | -6        |
| Physlib/FluidDynamics             | 19    | +15 | -1      | +1,156    | -235      |
| Physlib/Units                     | 12    | +5  | —       | +967      | -326      |
| Physlib/Electromagnetism          | 13    | +1  | —       | +404      | -815      |
| Physlib/StatisticalMechanics      | 4     | +1  | —       | +298      | -726      |
| Physlib/StringTheory              | 9     | —   | —       | +121      | -269      |
| Physlib/CondensedMatter           | 2     | +1  | —       | +249      | -100      |
| QuantumInfo/Entropy               | 2     | —   | —       | +201      | -15       |
| .github/workflows                 | 3     | +2  | —       | +201      | -0        |
| QuantumInfo/ForMathlib            | 13    | —   | —       | +104      | -65       |
| Physlib.lean                      | 1     | —   | —       | +106      | -52       |
| QuantumInfo/Capacity              | 1     | —   | —       | +114      | -34       |
| PhyslibAlpha/ClassicalFieldTheory | 1     | +1  | —       | +100      | -0        |
| Physlib/ClassicalFieldTheory      | 1     | +1  | —       | +91       | -0        |
| docs                              | 1     | +1  | —       | +50       | -0        |
| Physlib/Cosmology                 | 1     | —   | —       | +11       | -23       |
| lake-manifest.json                | 1     | —   | —       | +17       | -17       |
| QuantumInfo/Operators             | 1     | —   | —       | +11       | -10       |
| QuantumInfo/ClassicalInfo         | 4     | —   | —       | +10       | -6        |
| QuantumInfo/States                | 3     | —   | —       | +6        | -6        |
| PhyslibAlpha.lean                 | 1     | —   | —       | +9        | -0        |
| PhyslibAlpha/QuantumMechanics     | 1     | —   | —       | +4        | -4        |
| lakefile.toml                     | 1     | —   | —       | +6        | -2        |
| README.md                         | 1     | —   | —       | +1        | -6        |
| .gitignore                        | 1     | —   | —       | +4        | -0        |
| QuantumInfo/Channels              | 1     | —   | —       | +1        | -1        |
| lean-toolchain                    | 1     | —   | —       | +1        | -1        |
| scripts/MetaPrograms              | 1     | —   | —       | +2        | -0        |

### 1.4 New declarations by kind

| Kind                | Count |
|---------------------|-------|
| lemma               | 615   |
| def                 | 214   |
| instance            | 58    |
| structure           | 20    |
| theorem             | 18    |
| abbrev              | 17    |
| informal_lemma      | 9     |
| informal_definition | 7     |
| inductive           | 4     |

## 2 Details by file

### 2.1 Physlib.ClassicalMechanics.DampedHarmonicOscillator.Basic

[Physlib/ClassicalMechanics/DampedHarmonicOscillator/Basic.lean](#) on GitHub.

#### lemma acceleration\_eq\_of\_equationOfMotion

[\[source\]](#)

*Solving the equation of motion for the acceleration: along a solution the second derivative is  $-(k/m) x - (\gamma/m) \dot{x}$ .*

```
lemma acceleration_eq_of_equationOfMotion (z : Time → EuclideanSpace ℝ (Fin 1))
  (hEOM : S.EquationOfMotion z) (t : Time) :
  ∂t (∂t z) t = (-(S.m-1 * S.k)) • z t + (-(S.m-1 * S.γ)) • ∂t z t
```

#### lemma isUnderdamped\_or\_isCriticallyDamped\_or\_isOverdamped

[\[source\]](#)

*Every damped oscillator is underdamped, critically damped, or overdamped.*

```
lemma isUnderdamped_or_isCriticallyDamped_or_isOverdamped :
  S.IsUnderdamped v S.IsCriticallyDamped v S.IsOverdamped
```

#### def lagrangian

[\[source\]](#)

*The Caldirola–Kanai lagrangian of the damped harmonic oscillator. It is the lagrangian of the underlying undamped harmonic oscillator multiplied by the time-dependent factor  $\exp(\gamma/m * t)$ .*

```
noncomputable def lagrangian (t : Time) (x v : EuclideanSpace ℝ (Fin 1)) : ℝ
```

#### lemma lagrangian\_eq

[\[source\]](#)

```
lemma lagrangian_eq : S.lagrangian = fun (t : Time) (x v : EuclideanSpace ℝ (Fin 1)) =>
  exp (S.γ / S.m * t) * (1 / (2 : ℝ) * S.m * ⟨⟨v, v⟩⟩_ℝ - 1 / (2 : ℝ) * S.k * ⟨⟨x, x⟩⟩_ℝ)
```

#### lemma lagrangian\_eq\_exp\_mul\_kineticEnergy\_sub\_potentialEnergy

[\[source\]](#)

```
lemma lagrangian_eq_exp_mul_kineticEnergy_sub_potentialEnergy (t : Time)
  (xt : Time → EuclideanSpace ℝ (Fin 1)) :
  S.lagrangian t (xt t) (∂t xt t) =
  exp (S.γ / S.m * t) * (S.kineticEnergy xt t - S.potentialEnergy (xt t))
```

**lemma lagrangian\_of\_isUndamped**[\[source\]](#)

*When the damping coefficient is zero, the Caldirola–Kanai lagrangian is the lagrangian of the underlying undamped harmonic oscillator.*

```
lemma lagrangian_of_isUndamped (hS : S.IsUndamped) :
  S.lagrangian = S.toHarmonicOscillator.lagrangian
```

**lemma contDiff\_lagrangian**[\[source\]](#)

```
@[fun_prop]
```

```
lemma contDiff_lagrangian (n : WithTop ℕ) : ContDiff ℝ n 1 S.lagrangian
```

**lemma gradient\_const\_mul**[\[source\]](#)

```
private lemma gradient_const_mul {f : EuclideanSpace ℝ (Fin 1) → ℝ} {x : EuclideanSpace ℝ (Fin 1)}
  (c : ℝ) (hf : DifferentiableAt ℝ f x) :
  gradient (fun y => c * f y) x = c • gradient f x
```

**lemma gradient\_lagrangian\_position\_eq**[\[source\]](#)

```
lemma gradient_lagrangian_position_eq (t : Time) (x v : EuclideanSpace ℝ (Fin 1)) :
  gradient (fun x => S.lagrangian t x v) x = -(exp (S.γ / S.m * t) * S.k) • x
```

**lemma gradient\_lagrangian\_velocity\_eq**[\[source\]](#)

```
lemma gradient_lagrangian_velocity_eq (t : Time) (x v : EuclideanSpace ℝ (Fin 1)) :
  gradient (S.lagrangian t x) v = (exp (S.γ / S.m * t) * S.m) • v
```

**def gradLagrangian**[\[source\]](#)

*The variational gradient of the action of the damped harmonic oscillator.*

```
noncomputable def gradLagrangian (x_t : Time → EuclideanSpace ℝ (Fin 1)) :
  Time → EuclideanSpace ℝ (Fin 1)
```

**lemma gradLagrangian\_eq\_eulerLagrange0p**[\[source\]](#)

```
lemma gradLagrangian_eq_eulerLagrange0p (x_t : Time → EuclideanSpace ℝ (Fin 1))
  (hq : ContDiff ℝ ∞ x_t) :
  S.gradLagrangian x_t = eulerLagrange0p S.lagrangian x_t
```

**lemma deriv\_exp\_smul**[\[source\]](#)

```
private lemma deriv_exp_smul (a : ℝ) (y : Time → EuclideanSpace ℝ (Fin 1))
  (hy : Differentiable ℝ y) (t : Time) :
  ∂_t (fun t' : Time => exp (a * t'.val) • y t') t =
  exp (a * t.val) • (∂_t y t + a • y t)
```

**lemma gradLagrangian\_eq\_force**[\[source\]](#)

*The variational gradient of the Caldirola–Kanai action is the exponential factor times the difference of the force and mass times acceleration appearing in Newton’s second law.*

```
lemma gradLagrangian_eq_force (x_t : Time → EuclideanSpace ℝ (Fin 1)) (hx : ContDiff ℝ ∞ x_t) :
  S.gradLagrangian x_t = fun t : Time =>
    exp (S.γ / S.m * t) • (force S x_t t - S.m • ∂_t (∂_t x_t) t)
```

**lemma equationOfMotion\_iff\_gradLagrangian\_zero**[\[source\]](#)

```
lemma equationOfMotion_iff_gradLagrangian_zero (x_t : Time → EuclideanSpace ℝ (Fin 1))
  (hx : ContDiff ℝ ∞ x_t) :
  S.EquationOfMotion x_t ↔ S.gradLagrangian x_t = 0
```

**2.2 Physlib.ClassicalMechanics.DampedHarmonicOscillator.Solution**

[Physlib/ClassicalMechanics/DampedHarmonicOscillator/Solution.lean](#) on GitHub.

**lemma trajectory\_contDiff**[\[source\]](#)

*The selected trajectory is smooth.*

```
lemma trajectory_contDiff (IC : InitialConditions) :
  ContDiff ℝ ∞ (S.trajectory IC)
```

**lemma trajectory\_apply\_zero**[\[source\]](#)

*The selected trajectory has initial position  $IC.x_0$ .*

```
lemma trajectory_apply_zero (IC : InitialConditions) : S.trajectory IC 0 = IC.x_0
```

**lemma trajectory\_velocity\_at\_zero**[\[source\]](#)

*The selected trajectory has initial velocity  $IC.v_0$ .*

```
lemma trajectory_velocity_at_zero (IC : InitialConditions) :
  ∂_t (S.trajectory IC) 0 = IC.v_0
```

**def phaseVectorField**[\[source\]](#)

*The phase-space vector field of the damped oscillator, sending  $(x, \dot{x})$  to  $(\dot{x}, -(k/m)x - (\gamma/m)\dot{x})$ , as a continuous linear map on  $E \times E$ .*

```
private noncomputable def phaseVectorField :
  EuclideanSpace ℝ (Fin 1) × EuclideanSpace ℝ (Fin 1) → L[ℝ]
  EuclideanSpace ℝ (Fin 1) × EuclideanSpace ℝ (Fin 1)
```

**lemma phaseVectorField\_apply**[\[source\]](#)

```
private lemma phaseVectorField_apply (a b : EuclideanSpace ℝ (Fin 1)) :
  S.phaseVectorField (a, b) = (b, (-(S.m⁻¹ * S.k)) • a + (-(S.m⁻¹ * S.γ)) • b)
```



**lemma toRealCLE\_symm\_one**[\[source\]](#)

```
private lemma toRealCLE_symm_one : Time.toRealCLE.symm (1 : ℝ) = (1 : Time)
```

**lemma hasDerivAt\_comp\_toRealCLE\_symm**[\[source\]](#)

*Bridge from the time derivative to HasDerivAt for a curve reparametrised through the canonical  $\mathbb{R} \approx L[\mathbb{R}]$  Time equivalence.*

```
private lemma hasDerivAt_comp_toRealCLE_symm (w : Time → EuclideanSpace ℝ (Fin 1)) (τ : ℝ)
  (hw : DifferentiableAt ℝ w (Time.toRealCLE.symm τ)) :
  HasDerivAt (fun τ : ℝ => w (Time.toRealCLE.symm τ))
    (∂τ w (Time.toRealCLE.symm τ)) τ
```

**lemma phaseCurve\_hasDerivAt**[\[source\]](#)

*The phase curve  $\tau \mapsto (z \ t, \dot{x} \ t)$  (with  $t = \text{toRealCLE.symm } \tau$ ) of a smooth solution  $z$  solves the first-order phase-space ODE with vector field `phaseVectorField`.*

```
private lemma phaseCurve_hasDerivAt (z : Time → EuclideanSpace ℝ (Fin 1))
  (hz : ContDiff ℝ ∞ z) (hEOM : S.EquationOfMotion z) (τ : ℝ) :
  HasDerivAt (fun τ : ℝ => (z (Time.toRealCLE.symm τ), ∂τ z (Time.toRealCLE.symm τ)))
    (S.phaseVectorField (z (Time.toRealCLE.symm τ), ∂τ z (Time.toRealCLE.symm τ))) τ
```

**lemma equationOfMotion\_unique**[\[source\]](#)

*Any two smooth solutions of the damped equation of motion with the same initial position and velocity are equal.*

```
lemma equationOfMotion_unique (x y : Time → EuclideanSpace ℝ (Fin 1))
  (hx : ContDiff ℝ ∞ x) (hy : ContDiff ℝ ∞ y)
  (hEOMx : S.EquationOfMotion x) (hEOMy : S.EquationOfMotion y)
  (h0 : x 0 = y 0) (hv0 : ∂τ x 0 = ∂τ y 0) :
  x = y
```

**lemma trajectories\_unique**[\[source\]](#)

*The selected trajectory is the unique smooth solution of the damped equation of motion with the given initial conditions.*

```
lemma trajectories_unique (IC : InitialConditions) (x : Time → EuclideanSpace ℝ (Fin 1))
  (hx : ContDiff ℝ ∞ x) (hEOM : S.EquationOfMotion x)
  (hx0 : x 0 = IC.x0) (hv0 : ∂τ x 0 = IC.v0) :
  x = S.trajectory IC
```

**2.3 Physlib.ClassicalMechanics.HarmonicOscillator.Geometric.Basic**

[Physlib/ClassicalMechanics/HarmonicOscillator/Geometric/Basic.lean](#) on GitHub.

**def valDiffeomorph**[\[source\]](#)

*The global coordinate chart as an analytic diffeomorphism between ConfigurationSpace and its EuclideanSpace  $\mathbb{R}$  (Fin 1) model, upgrading valHomeomorphism to a smooth identification.*

```
def valDiffeomorph :
  ConfigurationSpace  $\approx_m^w$  (  $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)}), \mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$  )
  EuclideanSpace  $\mathbb{R}$  (Fin 1) where
```

**instance instNormedAddCommGroupTangent**

[\[source\]](#)

*The model-space normed additive group structure on tangent spaces.*

```
instance instNormedAddCommGroupTangent (q : ConfigurationSpace) :
  NormedAddCommGroup (TangentSpace  $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$  q)
```

**instance instNormedSpaceTangent**

[\[source\]](#)

*The model-space real normed vector space structure on tangent spaces.*

```
instance instNormedSpaceTangent (q : ConfigurationSpace) :
  NormedSpace  $\mathbb{R}$  (TangentSpace  $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$  q)
```

**def tangentCoord**

[\[source\]](#)

*The chart-induced continuous linear equivalence from the tangent space at  $q$  to the Euclidean coordinate model  $\text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)}$ .*

```
def tangentCoord (q : ConfigurationSpace) :
  TangentSpace  $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$  q  $\approx_{L[\mathbb{R}]}$  EuclideanSpace  $\mathbb{R}$  (Fin 1) where
```

**instance instFiniteDimensionalTangent**

[\[source\]](#)

*Each tangent space is finite-dimensional because it is linearly equivalent to  $\text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)}$  through  $\text{tangentCoord}$ .*

```
instance instFiniteDimensionalTangent (q : ConfigurationSpace) :
  FiniteDimensional  $\mathbb{R}$  (TangentSpace  $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$  q)
```

**def toSpace**

[\[source\]](#)

*The position in one-dimensional space associated to the configuration.*

```
def toSpace (q : ConfigurationSpace) : Space 1
```

**lemma toSpace\_apply**

[\[source\]](#)

*The physical-space coordinate associated to a configuration is its chosen global coordinate.*

```
lemma toSpace_apply (q : ConfigurationSpace) (i : Fin 1) : q.toSpace i = q.val i
```

## 2.4 Physlib.ClassicalMechanics.HarmonicOscillator.Geometric.KineticEnergy

[Physlib/ClassicalMechanics/HarmonicOscillator/Geometric/KineticEnergy.lean](#) on GitHub.

**def massMetricVal**[\[source\]](#)

*The value of the oscillator mass metric at  $q$ . It is the mass-scaled pullback of the Euclidean inner product along `tangentCoord  $q$` , so `massMetricVal  $S$   $q$   $v$   $w$`  =  $S.m * \langle\langle \text{tangentCoord } q \text{ } v, \text{tangentCoord } q \text{ } w \rangle\rangle_{\mathbb{R}}$ .*

```
noncomputable def massMetricVal (S : HarmonicOscillator) (q : ConfigurationSpace) :
  TangentSpace  $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$   $q \rightarrow L[\mathbb{R}]$ 
  TangentSpace  $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$   $q \rightarrow L[\mathbb{R}] \mathbb{R}$ 
```

**lemma massMetricVal\_apply**[\[source\]](#)

*Applying the mass metric value to two tangent vectors gives the mass-scaled Euclidean inner product of their coordinate representatives.*

```
lemma massMetricVal_apply
  (S : HarmonicOscillator) (q : ConfigurationSpace)
  (v w : TangentSpace  $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$  q) :
  massMetricVal S q v w = S.m *  $\langle\langle \text{tangentCoord } q \text{ } v, \text{tangentCoord } q \text{ } w \rangle\rangle_{\mathbb{R}}$ 
```

**lemma tangentCoord\_ne\_zero**[\[source\]](#)

*A nonzero tangent vector has nonzero coordinate representative under `tangentCoord`.*

```
lemma tangentCoord_ne_zero {q : ConfigurationSpace}
  {v : TangentSpace  $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$  q}
  (hv : v ≠ 0) : tangentCoord q v ≠ 0
```

**lemma massMetricVal\_pos**[\[source\]](#)

*The oscillator mass metric is positive on nonzero tangent vectors.*

```
lemma massMetricVal_pos
  (S : HarmonicOscillator) (q : ConfigurationSpace)
  (v : TangentSpace  $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$  q) (hv : v ≠ 0) :
  0 < massMetricVal S q v v
```

**lemma massMetricVal\_isVonNBounded**[\[source\]](#)

*The mass metric unit ball is bounded in the model norm.*

```
lemma massMetricVal_isVonNBounded
  (S : HarmonicOscillator) (q : ConfigurationSpace) :
  IsVonNBounded  $\mathbb{R}$ 
  {v : TangentSpace  $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$  q | massMetricVal S q v v < 1}
```

**lemma massMetricVal\_contMDiff**[\[source\]](#)

*The oscillator mass metric is constant in the global tangent-bundle chart.*

```
lemma massMetricVal_contMDiff (S : HarmonicOscillator) :
  ContMDiff  $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$ 
  ( $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$ .prod
    $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)}) \rightarrow L[\mathbb{R}]$ 
    $\text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)} \rightarrow L[\mathbb{R}] \mathbb{R})$   $\omega$ 
  (fun q : ConfigurationSpace =>
```

```

TotalSpace.mk'
(EuclideanSpace ℝ (Fin 1) →L[ℝ] EuclideanSpace ℝ (Fin 1) →L[ℝ] ℝ)
(E := fun q : ConfigurationSpace =>
  TangentSpace ℐ(ℝ, EuclideanSpace ℝ (Fin 1)) q →L[ℝ]
  TangentSpace ℐ(ℝ, EuclideanSpace ℝ (Fin 1)) q →L[ℝ] ℝ)
q (massMetricVal S q))

```

**def massRiemannianMetric**[\[source\]](#)

*The mass-scaled Euclidean inner product as a Riemannian metric on the oscillator configuration space.*

```

noncomputable def massRiemannianMetric (S : HarmonicOscillator) :
  ContMDiffRiemannianMetric ℐ(ℝ, EuclideanSpace ℝ (Fin 1)) ω
  (EuclideanSpace ℝ (Fin 1))
  (TangentSpace ℐ(ℝ, EuclideanSpace ℝ (Fin 1)) : ConfigurationSpace → Type _) where

```

**def geometricKineticEnergy**[\[source\]](#)

*The defining formula for the oscillator geometric kinetic energy.*

```

noncomputable def geometricKineticEnergy
  (S : HarmonicOscillator) (q : ConfigurationSpace)
  (v : TangentSpace ℐ(ℝ, EuclideanSpace ℝ (Fin 1)) q) : ℝ

```

**lemma geometricKineticEnergy\_eq**[\[source\]](#)

*The geometric kinetic energy is one half the mass Riemannian metric applied to v twice.*

```

lemma geometricKineticEnergy_eq
  (S : HarmonicOscillator) (q : ConfigurationSpace)
  (v : TangentSpace ℐ(ℝ, EuclideanSpace ℝ (Fin 1)) q) :
  geometricKineticEnergy S q v =
    (1 / 2 : ℝ) * S.massRiemannianMetric.inner q v v

```

**lemma massRiemannianMetric\_pos**[\[source\]](#)

*The oscillator mass Riemannian metric is positive definite.*

```

lemma massRiemannianMetric_pos
  (S : HarmonicOscillator) (q : ConfigurationSpace)
  (v : TangentSpace ℐ(ℝ, EuclideanSpace ℝ (Fin 1)) q)
  (hv : v ≠ 0) :
  0 < S.massRiemannianMetric.inner q v v

```

**lemma massRiemannianMetric\_inner\_apply**[\[source\]](#)

*In the global coordinate, the oscillator mass metric is the mass-scaled Euclidean inner product of coordinate representatives.*

```

lemma massRiemannianMetric_inner_apply
  (S : HarmonicOscillator) (q : ConfigurationSpace)
  (v w : TangentSpace ℐ(ℝ, EuclideanSpace ℝ (Fin 1)) q) :
  S.massRiemannianMetric.inner q v w =

```

$$S.m * \langle\langle \text{tangentCoord } q \ v, \text{ tangentCoord } q \ w \rangle\rangle_{\mathbb{R}}$$

### lemma geometricKineticEnergy\_massMetric\_eq

[\[source\]](#)

*The metric-induced kinetic energy for the mass metric has the standard harmonic-oscillator coordinate form.*

```
lemma geometricKineticEnergy_massMetric_eq
  (S : HarmonicOscillator) (q : ConfigurationSpace)
  (v : TangentSpace  $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$  q) :
  geometricKineticEnergy S q v =
    (1 / 2 :  $\mathbb{R}$ ) * S.m *  $\langle\langle \text{tangentCoord } q \ v, \text{ tangentCoord } q \ v \rangle\rangle_{\mathbb{R}}$ 
```

## 2.5 Physlib.ClassicalMechanics.HarmonicOscillator.Geometric.Trajectory

[Physlib/ClassicalMechanics/HarmonicOscillator/Geometric/Trajectory.lean](#) on GitHub.

### abbrev Trajectory

[\[source\]](#)

*A trajectory of the oscillator: a curve in the configuration manifold  $Q$ .*

```
abbrev Trajectory
```

### def coord

[\[source\]](#)

*The coordinate curve of a trajectory: its reading in the global Euclidean coordinate.*

```
def coord (γ : Trajectory) : Time → EuclideanSpace  $\mathbb{R}$  (Fin 1)
```

### def toSpace

[\[source\]](#)

*The physical position of the oscillator along a trajectory, over time.*

```
def toSpace (γ : Trajectory) : Time → Space 1
```

### lemma coord\_apply

[\[source\]](#)

```
lemma coord_apply (γ : Trajectory) (t : Time) : coord γ t = (γ t).val
```

### lemma contMDiff\_iff\_contDiff\_coord

[\[source\]](#)

*A trajectory is smooth as a curve in  $Q$  iff its coordinate curve is ordinarily smooth.*

```
lemma contMDiff_iff_contDiff_coord {n : WithTop  $\mathbb{N}_{\infty}$ } (γ : Trajectory) :
  ContMDiff  $\mathcal{I}(\mathbb{R}, \text{Time})$   $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$  n γ ↔
  ContDiff  $\mathbb{R}$  n (coord γ)
```

### def velocity

[\[source\]](#)

*The geometric velocity of a trajectory at time  $t$ : a tangent vector at  $\gamma \ t$ .*

```
noncomputable def velocity (γ : Trajectory) (t : Time) :
  TangentSpace  $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$  (γ t)
```

**lemma mdifferentiableAt\_iff\_differentiableAt\_coord**[\[source\]](#)

*A trajectory is MDifferentiableAt a time iff its coordinate curve is ordinarily differentiable there, since the two differ by the coordinate diffeomorphism.*

```
lemma mdifferentiableAt_iff_differentiableAt_coord (γ : Trajectory) (t : Time) :
  MDifferentiableAt  $\mathcal{I}(\mathbb{R}, \text{Time})$   $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$  γ t ↔
  DifferentiableAt  $\mathbb{R}$  (coord γ) t
```

**lemma mfderiv\_eq\_mfderiv\_coord**[\[source\]](#)

*A trajectory and its coordinate curve have the same manifold derivative, since they differ only by the coordinate diffeomorphism. This is the one lemma that unfolds the chart; every later result uses only the stable mfderiv/fderiv API.*

```
private lemma mfderiv_eq_mfderiv_coord (γ : Trajectory) (t : Time) :
  mfderiv  $\mathcal{I}(\mathbb{R}, \text{Time})$   $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$  γ t =
  mfderiv  $\mathcal{I}(\mathbb{R}, \text{Time})$   $\mathcal{I}(\mathbb{R}, \text{EuclideanSpace } \mathbb{R} \text{ (Fin 1)})$  (coord γ) t
```

**lemma velocity\_eq\_deriv\_coord**[\[source\]](#)

*In the global coordinate, geometric velocity is represented by the time derivative of the coordinate curve.*

```
lemma velocity_eq_deriv_coord (γ : Trajectory) (t : Time) :
  velocity γ t =  $\partial_t$  (coord γ) t
```

**2.6 Physlib.ClassicalMechanics.Lagrangian.TotalDerivativeEquivalence**

[Physlib/ClassicalMechanics/Lagrangian/TotalDerivativeEquivalence.lean](#) on GitHub.

**lemma isTotalTimeDerivative\_neg**[\[source\]](#)

*Elementary fact: if  $\delta L$  is a time derivative, then so is  $-\delta L$ .*

```
lemma isTotalTimeDerivative_neg {δL : Time → X → X → ℝ} (h : IsTotalTimeDerivative δL) :
  IsTotalTimeDerivative (- δL)
```

**lemma totalTimeDerivative\_contDiff**[\[source\]](#)

*If  $\delta L$  is a total time derivative (of a smooth function), then it is smooth*

```
lemma totalTimeDerivative_contDiff {δL : Time → X → X → ℝ} (h : IsTotalTimeDerivative δL):
  ContDiff  $\mathbb{R} \infty$  δL
```

**lemma totalTimeDerivative\_hasZeroVarGradient**[\[source\]](#)

*Total time derivative has a variational derivative, which is zero*

```
lemma totalTimeDerivative_hasZeroVarGradient [CompleteSpace X] {δL : Time → X → X → ℝ}
  (h : IsTotalTimeDerivative δL) (q : Time → X) (hq : ContDiff  $\mathbb{R} \infty$  q):
  HasVarGradientAt (fun q' t => δL t (q' t) ( $\partial_t$  q' t)) (fun _ => 0) q
```

**lemma totalTimeDerivative\_hasVarGradientAt\_equivalence**[\[source\]](#)

*If two lagrangians,  $L$  and  $L'$ , differ by a total time derivative, and  $L$  has a variational derivative  $\text{grad}$ , then so does  $L'$ .*

```
lemma totalTimeDerivative_hasVarGradientAt_equivalence [CompleteSpace X] (L δL : Time → X → X → ℝ)
  (hδL : IsTotalTimeDerivative δL)
  (q : Time → X) (hq : ContDiff ℝ ∞ q) (grad : Time → X)
  (hgrad : HasVarGradientAt (fun q' t => L t (q' t) (fderiv ℝ q' t 1)) grad q) :
  HasVarGradientAt (fun q' t => (L + δL) t (q' t) (fderiv ℝ q' t 1)) grad q
```

**lemma totalTimeDerivative\_varGradient\_equivalenvce**[\[source\]](#)

```
lemma totalTimeDerivative_varGradient_equivalenvce [CompleteSpace X] (L L' : Time → X → X → ℝ)
  (htot : IsTotalTimeDerivative (L' - L))
  (q : Time → X) (hq : ContDiff ℝ ∞ q) :
  (δ (q' := q), ∫ t, L' t (q' t) (fderiv ℝ q' t 1)) =
  (δ (q' := q), ∫ t, L t (q' t) (fderiv ℝ q' t 1))
```

**lemma totalTimeDerivative\_eulerLagrange\_equivalenvce**[\[source\]](#)

*Corollary: If  $L$  and  $L'$  differ by a total time derivative, then the corresponding Euler-Lagrange operators coincide*

```
lemma totalTimeDerivative_eulerLagrange_equivalenvce [CompleteSpace X] (L L' : Time → X → X → ℝ)
  (htot : IsTotalTimeDerivative (L' - L)) (hContDiff : (ContDiff ℝ ∞ 1L) v (ContDiff ℝ ∞ 1L'))
  (q : Time → X) (hq : ContDiff ℝ ∞ q) : eulerLagrangeOp L q = eulerLagrangeOp L' q
```

**2.7 Physlib.ClassicalMechanics.RigidBody.AngularMomentum**

[Physlib/ClassicalMechanics/RigidBody/AngularMomentum.lean](#) on GitHub.

**def angularMomentum**[\[source\]](#)

*The angular momentum  $L = \int r \times (\omega \times r) \, dm$  of a rigid body rotating with angular velocity  $\omega$  about its reference point (each body point at  $r$  moving with velocity  $\omega \times r$ ). Its  $i$ -th component is  $p$  applied to the scalar field  $[r \times (\omega \times r)]_i$ .*

```
noncomputable def angularMomentum (R : RigidBody 3) (ω : Fin 3 → ℝ) : Fin 3 → ℝ
```

**theorem angularMomentum\_eq\_inertiaTensor\_mulVec**[\[source\]](#)

*The angular momentum of a rigid body equals its inertia tensor applied to the angular velocity:  $L = I \omega$ .*

```
theorem angularMomentum_eq_inertiaTensor_mulVec (R : RigidBody 3) (ω : Fin 3 → ℝ) :
  R.angularMomentum ω = R.inertiaTensor *v ω
```

**2.8 Physlib.ClassicalMechanics.RigidBody.AngularVelocity**

[Physlib/ClassicalMechanics/RigidBody/AngularVelocity.lean](#) on GitHub.

**def angularVelocityTensor**[\[source\]](#)

The angular velocity tensor  $\Omega(t) = \dot{R}(t) R(t)^T$  of a rigid body in motion, where  $R(t) = \text{orientation } t$ . It is the antisymmetric tensor  $\Omega$  in the Landau–Lifshitz decomposition  $v = V + \Omega \times r$  of the velocity of a point of the body.

```
noncomputable def angularVelocityTensor (M : RigidBodyMotion d) (t : Time) :
  Matrix (Fin d) (Fin d) ℝ
```

**lemma angularVelocityTensor\_eq**[\[source\]](#)

```
lemma angularVelocityTensor_eq (M : RigidBodyMotion d) (t : Time) :
  M.angularVelocityTensor t = ∂ₜ (fun s => (M.orientation s).1) t * ((M.orientation t).1)T
```

**lemma angularVelocityTensor\_transpose**[\[source\]](#)

The angular velocity tensor is skew-symmetric,  $\Omega^T = -\Omega$ : it lies in the Lie algebra  $\mathfrak{so}(d)$ . This is the litmus check that  $\Omega = \dot{R} R^T$  is a genuine angular-velocity tensor, and follows by differentiating the orthogonality identity  $R R^T = 1$ .

```
lemma angularVelocityTensor_transpose (M : RigidBodyMotion d) (t : Time)
  (hR : DifferentiableAt ℝ (fun s => (M.orientation s).1) t) :
  (M.angularVelocityTensor t)T = - M.angularVelocityTensor t
```

**lemma angularVelocityTensor\_of\_orientation\_const**[\[source\]](#)

A rigid body whose orientation is constant in time has zero angular velocity.

```
lemma angularVelocityTensor_of_orientation_const (M : RigidBodyMotion d)
  (R : Matrix.specialOrthogonalGroup (Fin d) ℝ) (h : M.orientation = fun _ => R) :
  M.angularVelocityTensor = 0
```

**def angularVelocity**[\[source\]](#)

The angular velocity *vector*  $\omega(t)$  of a rigid body moving in three-dimensional space: the vector dual to the angular velocity tensor  $\Omega(t)$  under the hat map,  $\omega = \Omega^\vee$ . It is the angular velocity  $\omega$  in the Landau–Lifshitz decomposition  $v = V + \omega \times r$  of the velocity of a point of the body, where  $\omega \times r$  is the cross product with  $\omega$ .

```
noncomputable def angularVelocity (M : RigidBodyMotion 3) (t : Time) : Fin 3 → ℝ
```

**lemma angularVelocity\_eq**[\[source\]](#)

```
lemma angularVelocity_eq (M : RigidBodyMotion 3) (t : Time) :
  M.angularVelocity t = crossProductVee (M.angularVelocityTensor t)
```

**lemma crossProductMatrix\_angularVelocity**[\[source\]](#)

The hat map recovers the angular velocity tensor from the angular velocity vector,  $[\omega]_\times = \Omega$ . This is the defining relationship between the vector and tensor forms of the angular velocity in three dimensions; it holds because  $\Omega$  is skew-symmetric.

```
lemma crossProductMatrix_angularVelocity (M : RigidBodyMotion 3) (t : Time)
  (hR : DifferentiableAt ℝ (fun s => (M.orientation s).1) t) :
```



```
crossProductMatrix (M.angularVelocity t) = M.angularVelocityTensor t
```

### lemma angularVelocity\_of\_orientation\_const

[\[source\]](#)

*A rigid body whose orientation is constant in time has zero angular velocity vector.*

```
lemma angularVelocity_of_orientation_const (M : RigidBodyMotion 3)
  (R : Matrix.specialOrthogonalGroup (Fin 3) ℝ) (h : M.orientation = fun _ => R) :
  M.angularVelocity = 0
```

### lemma angularVelocityTensor\_mul\_orientation

[\[source\]](#)

*The time derivative of the orientation is  $\dot{R} = \Omega R$ , recovering the orientation path from its angular velocity tensor  $\Omega = \dot{R} R^T$  via the orthogonality  $R^T R = 1$ .*

```
lemma angularVelocityTensor_mul_orientation (M : RigidBodyMotion d) (t : Time) :
  M.angularVelocityTensor t * (M.orientation t).1 = ∂ₜ (fun s => (M.orientation s).1) t
```

### lemma deriv\_orientation\_mulVec\_eq\_angularVelocity\_cross

[\[source\]](#)

*The rotational part of the velocity of the body point  $y$  is the cross product of the angular velocity with the point's position relative to the centre of mass:  $\dot{R}(t) (y - c) = \omega(t) \times (\text{displacement } t \ y - \text{comTrajectory } t)$ .*

```
lemma deriv_orientation_mulVec_eq_angularVelocity_cross (M : RigidBodyMotion 3) (y : Space 3)
  (t : Time) (hR : DifferentiableAt ℝ (fun s => (M.orientation s).1) t) :
  ∂ₜ (fun s => (M.orientation s).1) t *ₒ (fun j => y j - M.centerOfMass j)
  = M.angularVelocity t ×₃ fun j => M.displacement t y j - M.comTrajectory t j
```

### theorem velocity\_eq\_angularVelocity

[\[source\]](#)

*The Landau–Lifshitz velocity decomposition  $v = V + \omega \times r$  for a rigid body in three dimensions: the velocity of a body point is the centre-of-mass velocity plus the cross product of the angular velocity with the point's position relative to the centre of mass.*

```
theorem velocity_eq_angularVelocity (M : RigidBodyMotion 3) (y : Space 3) (t : Time) (i : Fin 3)
  (hR : Differentiable ℝ (fun s => (M.orientation s).1)) (hX : Differentiable ℝ M.comTrajectory) :
  M.velocity y t i = M.centerOfMassVelocity t i
  + (M.angularVelocity t ×₃ fun j => M.displacement t y j - M.comTrajectory t j) i
```

### def bodyAngularVelocityTensor

[\[source\]](#)

*The body-frame (co-rotating) angular velocity tensor  $\Omega_{\text{body}}(t) = R(t)^T \dot{R}(t)$  of a rigid body in motion, where  $R(t) = \text{orientation } t$ .*

```
noncomputable def bodyAngularVelocityTensor (M : RigidBodyMotion d) (t : Time) :
  Matrix (Fin d) (Fin d) ℝ
```

### lemma bodyAngularVelocityTensor\_eq

[\[source\]](#)

```
lemma bodyAngularVelocityTensor_eq (M : RigidBodyMotion d) (t : Time) :
```

```
M.bodyAngularVelocityTensor t = ((M.orientation t).1)T * ∂t (fun s => (M.orientation s).1) t
```

### lemma orientation\_mul\_bodyAngularVelocityTensor

[\[source\]](#)

The time derivative of the orientation is  $\dot{R} = R \Omega_{\text{body}}$ , recovering the orientation path from its body-frame angular velocity tensor via the orthogonality  $R R^T = 1$ .

```
lemma orientation_mul_bodyAngularVelocityTensor (M : RigidBodyMotion d) (t : Time) :
  (M.orientation t).1 * M.bodyAngularVelocityTensor t = ∂t (fun s => (M.orientation s).1) t
```

### lemma angularVelocityTensor\_eq\_orientation\_conj

[\[source\]](#)

The spatial tensor is the body-frame tensor rotated into the inertial frame,  $\Omega = R \Omega_{\text{body}} R^T$ .

```
lemma angularVelocityTensor_eq_orientation_conj (M : RigidBodyMotion d) (t : Time) :
  M.angularVelocityTensor t
  = (M.orientation t).1 * M.bodyAngularVelocityTensor t * ((M.orientation t).1)T
```

### lemma bodyAngularVelocityTensor\_eq\_orientation\_conj

[\[source\]](#)

The body-frame tensor is the spatial tensor conjugated into the body frame,  $\Omega_{\text{body}} = R^T \Omega R$ .

```
lemma bodyAngularVelocityTensor_eq_orientation_conj (M : RigidBodyMotion d) (t : Time) :
  M.bodyAngularVelocityTensor t
  = ((M.orientation t).1)T * M.angularVelocityTensor t * (M.orientation t).1
```

### lemma bodyAngularVelocityTensor\_transpose

[\[source\]](#)

The body-frame angular velocity tensor is skew-symmetric,  $\Omega_{\text{body}}^T = -\Omega_{\text{body}}$ : it lies in the Lie algebra  $\mathfrak{so}(d)$ , inherited from the spatial tensor by conjugation.

```
lemma bodyAngularVelocityTensor_transpose (M : RigidBodyMotion d) (t : Time)
  (hR : DifferentiableAt ℝ (fun s => (M.orientation s).1) t) :
  (M.bodyAngularVelocityTensor t)T = - M.bodyAngularVelocityTensor t
```

### def bodyAngularVelocity

[\[source\]](#)

The body-frame angular velocity *\*vector\**  $\omega_{\text{body}}(t)$  of a rigid body moving in three-dimensional space: the vector dual to the body-frame angular velocity tensor  $\Omega_{\text{body}}(t)$  under the hat map,  $\omega_{\text{body}} = \Omega_{\text{body}}^v$ .

```
noncomputable def bodyAngularVelocity (M : RigidBodyMotion 3) (t : Time) : Fin 3 → ℝ
```

### lemma bodyAngularVelocity\_eq

[\[source\]](#)

```
lemma bodyAngularVelocity_eq (M : RigidBodyMotion 3) (t : Time) :
  M.bodyAngularVelocity t = crossProductVee (M.bodyAngularVelocityTensor t)
```

**lemma crossProductMatrix\_bodyAngularVelocity**[\[source\]](#)

*The hat map recovers the body-frame angular velocity tensor from the body-frame angular velocity vector,  $[\omega_{\text{body}}]_{\times} = \Omega_{\text{body}}$ ; it holds because  $\Omega_{\text{body}}$  is skew-symmetric.*

```
lemma crossProductMatrix_bodyAngularVelocity (M : RigidBodyMotion 3) (t : Time)
  (hR : DifferentiableAt ℝ (fun s => (M.orientation s).1) t) :
  crossProductMatrix (M.bodyAngularVelocity t) = M.bodyAngularVelocityTensor t
```

**lemma bodyAngularVelocityTensor\_of\_orientation\_const**[\[source\]](#)

*A rigid body whose orientation is constant in time has zero body-frame angular velocity tensor.*

```
lemma bodyAngularVelocityTensor_of_orientation_const (M : RigidBodyMotion d)
  (R : Matrix.specialOrthogonalGroup (Fin d) ℝ) (h : M.orientation = fun _ => R) :
  M.bodyAngularVelocityTensor = 0
```

**lemma bodyAngularVelocity\_of\_orientation\_const**[\[source\]](#)

*A rigid body whose orientation is constant in time has zero body-frame angular velocity vector.*

```
lemma bodyAngularVelocity_of_orientation_const (M : RigidBodyMotion 3)
  (R : Matrix.specialOrthogonalGroup (Fin 3) ℝ) (h : M.orientation = fun _ => R) :
  M.bodyAngularVelocity = 0
```

**lemma deriv\_orientation\_mulVec\_eq\_orientation\_bodyAngularVelocity\_cross**[\[source\]](#)

*The rotational velocity written through the body-frame angular velocity,  $\dot{R}(t) v = R(t) (\omega_{\text{body}}(t) \times v)$ : the derivative of the orientation acts as the body-frame angular velocity  $\omega_{\text{body}}$  crossed with  $v$ , rotated back into the inertial frame by  $R$ .*

```
lemma deriv_orientation_mulVec_eq_orientation_bodyAngularVelocity_cross (M : RigidBodyMotion 3)
  (v : Fin 3 → ℝ) (t : Time)
  (hR : DifferentiableAt ℝ (fun s => (M.orientation s).1) t) :
  ∂ₜ (fun s => (M.orientation s).1) t *ₐ v
  = (M.orientation t).1 *ₐ (M.bodyAngularVelocity t ×ₐ v)
```

**2.9 Physlib.ClassicalMechanics.RigidBody.Basic**

[Physlib/ClassicalMechanics/RigidBody/Basic.lean](#) on GitHub.

**lemma rho\_one**[\[source\]](#)

*The mass distribution applied to the constant function 1 is the total mass.*

```
@[simp]
lemma rho_one {d : ℕ} (R : RigidBody d) :
  R.ρ (1 : C^T (ℒ(ℝ, Space d), Space d; ℒ(ℝ, ℝ), ℝ)) = R.mass
```

**def cmap**[\[source\]](#)

Bundle a smooth real-valued function on *Space d* as an element of the space of test functions. Keeping this as a named constructor ensures the resulting type head stays *ContMDiffMap*, so the module/ring operations and *comp* resolve correctly.

```
def cmap {d : ℕ} (f : Space d → ℝ) (hf : ContDiff ℝ τ f) :
  C^τ (ℐ(ℝ, Space d), Space d; ℐ(ℝ, ℝ), ℝ)
```

**lemma cmap\_apply**

[\[source\]](#)

@[simp]

```
lemma cmap_apply {d : ℕ} (f : Space d → ℝ) (hf : ContDiff ℝ τ f) (y : Space d) :
  cmap f hf y = f y
```

**lemma rho\_coord\_sub\_centerOfMass**

[\[source\]](#)

The first moment of the mass distribution about its own centre of mass vanishes: for nonzero mass,  $\rho$  of the centred  $j$ -th coordinate function is zero.

```
lemma rho_coord_sub_centerOfMass {d : ℕ} (R : RigidBody d) (h : R.mass ≠ 0) (j : Fin d) :
  R.ρ (cmap (fun y => y j - R.centerOfMass j) (by fun_prop)) = 0
```

## 2.10 Physlib.ClassicalMechanics.RigidBody.KineticEnergy

[Physlib/ClassicalMechanics/RigidBody/KineticEnergy.lean](#) on GitHub.

**def rotationalKineticEnergy**

[\[source\]](#)

The rotational kinetic energy of a rigid body rotating with angular velocity  $\omega$  about its reference point: half the contraction of  $\omega$  with the inertia tensor,  $T = \frac{1}{2} \omega \cdot (I \omega)$ .

```
noncomputable def rotationalKineticEnergy (R : RigidBody 3) (ω : Fin 3 → ℝ) : ℝ
```

**lemma rotationalKineticEnergy\_eq\_angularMomentum**

[\[source\]](#)

The rotational kinetic energy is half the contraction of the angular velocity with the angular momentum:  $T = \frac{1}{2} \omega \cdot L$ .

```
lemma rotationalKineticEnergy_eq_angularMomentum (R : RigidBody 3) (ω : Fin 3 → ℝ) :
  R.rotationalKineticEnergy ω = (1 / (2 : ℝ)) * (ω ·_v R.angularMomentum ω)
```

**theorem rotationalKineticEnergy\_eq\_integral**

[\[source\]](#)

The rotational kinetic energy equals the mass integral of the local rotational speed squared:  $T = \frac{1}{2} \int |\omega \times r|^2 dm$ .

```
theorem rotationalKineticEnergy_eq_integral (R : RigidBody 3) (ω : Fin 3 → ℝ) :
  R.rotationalKineticEnergy ω
  = (1 / (2 : ℝ)) * R.ρ (fun x => (ω ×₃ (x : Fin 3 → ℝ)) ·_v (ω ×₃ (x : Fin 3 → ℝ)),
    ContDiff.contMDiff <| (contDiff_cross_dotProduct_cross ω).comp
    (contDiff_pi.mpr fun i => Space.eval_contDiff i))
```

**def kineticEnergy**[\[source\]](#)

The total kinetic energy of a rigid body in motion at time  $t$ : half the mass integral of the squared speed of the body points,  $T = \frac{1}{2} \int \langle \langle v, v \rangle \rangle dm$ , with the point velocity taken in the closed form `velocityClosedForm`.

```
noncomputable def kineticEnergy {d : ℕ} (M : RigidBodyMotion d) (t : Time) : ℝ
```

**lemma kineticEnergy\_eq\_integral\_velocity**[\[source\]](#)

For a differentiable motion the integrand of `kineticEnergy` is the squared speed of the honest point velocity:  $T = \frac{1}{2} \int \langle \langle v, v \rangle \rangle dm$  with  $v = \partial_t (\text{displacement} \cdot y)$ .

```
lemma kineticEnergy_eq_integral_velocity {d : ℕ} (M : RigidBodyMotion d) (t : Time)
(hR : Differentiable ℝ (fun s => (M.orientation s).1))
(hX : Differentiable ℝ M.comTrajectory) :
M.kineticEnergy t = (1 / (2 : ℝ)) * M.p (cmap
  (fun y => ((M.velocity y t, M.velocity y t))_ℝ))
  (by
    simp only [← M.velocityClosedForm_eq_velocity t hR hX]
    exact M.contDiff_velocityClosedForm_inner t)
```

**lemma kineticEnergy\_integrand\_split**[\[source\]](#)

The squared-speed integrand of `kineticEnergy` splits into the squared rotational speed  $|\dot{R}(y - c)|^2$ , a term linear in the body-frame coordinate  $y - c$ , and the constant squared centre-of-mass speed  $\langle \langle V, V \rangle \rangle$ .

```
lemma kineticEnergy_integrand_split {d : ℕ} (M : RigidBodyMotion d) (t : Time) :
cmap (fun y => ((M.velocityClosedForm t y, M.velocityClosedForm t y))_ℝ))
  (M.contDiff_velocityClosedForm_inner t)
= cmap (fun y =>
  (∂_t (fun s => (M.orientation s).1) t *v fun j => y j - M.centerOfMass j) ·v
  (∂_t (fun s => (M.orientation s).1) t *v fun j => y j - M.centerOfMass j))
  (by simp only [dotProduct, Matrix.mulVec]; fun_prop)
+ ∑ j, (2 * ((M.centerOfMassVelocity t : Fin d → ℝ) v*
  ∂_t (fun s => (M.orientation s).1) t) j)) •
  cmap (fun y => y j - M.centerOfMass j) (by fun_prop)
+ ((M.centerOfMassVelocity t, M.centerOfMassVelocity t)_ℝ) •
  (1 : C^T (ℐ(ℝ, Space d), Space d; ℐ(ℝ, ℝ), ℝ))
```

**theorem kineticEnergy\_eq\_translational\_add\_rotational**[\[source\]](#)

*\*\*König's theorem\*\**, general form: the total kinetic energy of a rigid body in motion splits into the kinetic energy of the centre of mass plus the rotational energy about the centre of mass,  $T = \frac{1}{2} M \langle \langle V, V \rangle \rangle + \frac{1}{2} \int |\dot{R}(y - c)|^2 dm$ .

```
theorem kineticEnergy_eq_translational_add_rotational {d : ℕ} (M : RigidBodyMotion d) (t : Time)
(h : M.mass ≠ 0) :
M.kineticEnergy t
= (1 / (2 : ℝ)) * M.mass * ((M.centerOfMassVelocity t, M.centerOfMassVelocity t)_ℝ)
+ (1 / (2 : ℝ)) * M.p (cmap (fun y =>
  (∂_t (fun s => (M.orientation s).1) t *v fun j => y j - M.centerOfMass j) ·v
  (∂_t (fun s => (M.orientation s).1) t *v fun j => y j - M.centerOfMass j))
  (by simp only [dotProduct, Matrix.mulVec]; fun_prop))
```

**theorem kineticEnergy\_eq\_translational\_add\_angularVelocity**[\[source\]](#)

*\*\*König's theorem\*\* in three dimensions: the total kinetic energy of a rigid body in motion splits as  $T = \frac{1}{2} M \langle V, V \rangle + \frac{1}{2} \int |\omega \times r|^2 dm$ , with  $\omega$  the angular velocity vector and  $r = \text{displacement} - \text{comTrajectory}$  the position of the body point relative to the centre of mass.*

```

theorem kineticEnergy_eq_translational_add_angularVelocity (M : RigidBodyMotion 3) (t : Time)
  (h : M.mass ≠ 0) (hR : DifferentiableAt ℝ (fun s => (M.orientation s).1) t) :
  M.kineticEnergy t
    = (1 / (2 : ℝ)) * M.mass * (⟨M.centerOfMassVelocity t, M.centerOfMassVelocity t⟩_ℝ)
      + (1 / (2 : ℝ)) * M.p (cmap (fun y =>
        (M.angularVelocity t ×₃ fun j => M.displacement t y j - M.comTrajectory t j) ·_v
        (M.angularVelocity t ×₃ fun j => M.displacement t y j - M.comTrajectory t j)))
    (by
      exact (contDiff_cross_dotProduct_cross (M.angularVelocity t)).comp
        (contDiff_pi.mpr fun j =>
          ((Space.eval_contDiff j).comp (M.displacement_contDiff t)).sub
            contDiff_const)))

```

**theorem kineticEnergy\_eq\_translational\_add\_bodyAngularVelocity**[\[source\]](#)

*\*\*König's theorem\*\* in the body frame. The total kinetic energy  $M.kineticEnergy t$ , formed from the lab-frame point velocities, splits at the centre of mass ( $\text{centerOfMass} = 0$ ) as  $T = \frac{1}{2} M \langle V, V \rangle + \text{rotationalKineticEnergy } \omega_{\text{body}}$ . The rotational energy is a frame-independent scalar, so it is evaluated here from the *body-frame* angular velocity  $\omega_{\text{body}}$ : the spatial form  $|\dot{R} (y - c)|^2$  and the body form  $|\omega_{\text{body}} \times (y - c)|^2$  agree because  $\dot{R} (y - c) = R (\omega_{\text{body}} \times (y - c))$  and  $R$ , being orthogonal, preserves the norm.*

```

theorem kineticEnergy_eq_translational_add_bodyAngularVelocity (M : RigidBodyMotion 3) (t :
  Time)
  (h : M.mass ≠ 0) (hR : DifferentiableAt ℝ (fun s => (M.orientation s).1) t)
  (hc : M.centerOfMass = 0) :
  M.kineticEnergy t
    = (1 / (2 : ℝ)) * M.mass * (⟨M.centerOfMassVelocity t, M.centerOfMassVelocity t⟩_ℝ)
      + M.toRigidBody.rotationalKineticEnergy (M.bodyAngularVelocity t)

```

**2.11 Physlib.ClassicalMechanics.RigidBody.Motion**

[Physlib/ClassicalMechanics/RigidBody/Motion.lean](#) on GitHub.

**structure RigidBodyMotion**[\[source\]](#)

*A motion of a rigid body in  $d$ -dimensional space: the body together with the inertial-frame trajectory of its centre of mass and its time-dependent orientation (a rotation about the centre of mass).*

```

structure RigidBodyMotion (d : ℕ) extends RigidBody d where
  comTrajectory : Time → Space d
  orientation : Time → Matrix.specialOrthogonalGroup (Fin d) ℝ

```

**lemma orientation\_mul\_transpose**[\[source\]](#)

*The orientation matrix is special orthogonal, so  $R R^T = 1$ .*

```
lemma orientation_mul_transpose {d : ℕ} (M : RigidBodyMotion d) (t : Time) :
  (M.orientation t).1 * ((M.orientation t).1)T = 1
```

### def centerOfMassVelocity

[\[source\]](#)

The velocity of the centre of mass of a rigid body in motion, defined as the time-derivative of its centre-of-mass trajectory. This is the velocity  $V$  in the Landau–Lifshitz decomposition  $v = V + \Omega \times r$  of the velocity of a point of the body.

```
noncomputable def centerOfMassVelocity {d : ℕ} (M : RigidBodyMotion d) : Time → Space d
```

### lemma centerOfMassVelocity\_eq

[\[source\]](#)

```
lemma centerOfMassVelocity_eq {d : ℕ} (M : RigidBodyMotion d) :
  M.centerOfMassVelocity = ∂t M.comTrajectory
```

### lemma centerOfMassVelocity\_of\_comTrajectory\_const

[\[source\]](#)

A rigid body whose centre of mass is stationary has zero centre-of-mass velocity.

```
lemma centerOfMassVelocity_of_comTrajectory_const {d : ℕ} (M : RigidBodyMotion d) (c : Space d)
  (h : M.comTrajectory = fun _ => c) : M.centerOfMassVelocity = 0
```

### def linearMomentum

[\[source\]](#)

The linear momentum of a rigid body in motion: the total mass times the velocity of the centre of mass.

```
noncomputable def linearMomentum {d : ℕ} (M : RigidBodyMotion d) : Time → Space d
```

### lemma linearMomentum\_eq

[\[source\]](#)

```
lemma linearMomentum_eq {d : ℕ} (M : RigidBodyMotion d) :
  M.linearMomentum = fun t => M.mass • M.centerOfMassVelocity t
```

### def displacement

[\[source\]](#)

The rigid displacement carrying the body frame into the inertial frame at time  $t$ : the rotation  $\text{orientation } t$  about the centre of mass, followed by the translation placing the centre of mass at  $\text{comTrajectory } t$ .

```
noncomputable def displacement {d : ℕ} (M : RigidBodyMotion d) (t : Time) : Space d → Space d
```

### lemma displacement\_apply

[\[source\]](#)

The  $k$ -th coordinate of the rigid displacement applied to  $y$ .

```
lemma displacement_apply {d : ℕ} (M : RigidBodyMotion d) (t : Time) (y : Space d) (k : Fin d) :
  M.displacement t y k =
    (∑ j, (M.orientation t).val k j * (y j - M.centerOfMass j)) + M.comTrajectory t k
```

**lemma displacement\_contDiff**[\[source\]](#)

```
lemma displacement_contDiff {d : ℕ} (M : RigidBodyMotion d) (t : Time) :
  ContDiff ℝ T (M.displacement t)
```

**lemma orientation\_mulVec\_sub\_centerOfMass**[\[source\]](#)

*Rotating the body-frame position of  $y$  relative to the centre of mass gives its inertial-frame position relative to the moving centre of mass:  $R(t) (y - c) = \text{displacement } t y - \text{comTrajectory } t$ .*

```
lemma orientation_mulVec_sub_centerOfMass {d : ℕ} (M : RigidBodyMotion d) (t : Time)
  (y : Space d) :
  (M.orientation t).l *v (fun j => y j - M.centerOfMass j)
  = fun j => M.displacement t y j - M.comTrajectory t j
```

**def massDistribution**[\[source\]](#)

*The mass distribution of the rigid body in motion at time  $t$ : the pushforward of the body-fixed mass distribution along the rigid displacement, acting on a test function  $f$  by  $f \mapsto \rho (f \circ \text{displacement } t)$ .*

```
noncomputable def massDistribution {d : ℕ} (M : RigidBodyMotion d) (t : Time) : RigidBody d
  where
```

**lemma massDistribution\_mass**[\[source\]](#)

*The motion preserves the total mass: the mass distribution at any time  $t$  has the same total mass as the body.*

```
@[simp]
lemma massDistribution_mass {d : ℕ} (M : RigidBodyMotion d) (t : Time) :
  (M.massDistribution t).mass = M.mass
```

**lemma contMDiffMap\_sum\_apply**[\[source\]](#)

*Evaluation commutes with finite sums of smooth functions.*

```
private lemma contMDiffMap_sum_apply {d : ℕ} {ι : Type*} (s : Finset ι)
  (f : ι → C^T (ℒ(ℝ, Space d), Space d; ℒ(ℝ, ℝ), ℝ)) (y : Space d) :
  (∑ j ∈ s, f j) y = ∑ j ∈ s, f j y
```

**lemma massDistribution\_centerOfMass**[\[source\]](#)

*The centre of mass of the moving mass distribution tracks the prescribed trajectory: for a body of nonzero mass, the centre of mass of  $\text{massDistribution } M t$  is exactly  $\text{comTrajectory } t$ . This is the decisive check that  $\text{comTrajectory}$  and  $\text{orientation}$  are wired correctly in  $\text{RigidBodyMotion}$ .*

```
lemma massDistribution_centerOfMass {d : ℕ} (M : RigidBodyMotion d) (t : Time) (h : M.mass ≠ 0) :
  (M.massDistribution t).centerOfMass = M.comTrajectory t
```



**def velocity**[\[source\]](#)

*The velocity of the material point  $y$  of a rigid body in motion: the inertial-frame time derivative of the trajectory  $s \mapsto$  displacement  $s$   $y$  of that point.*

```
noncomputable def velocity {d : ℕ} (M : RigidBodyMotion d) (y : Space d) : Time → Space d
```

**lemma velocity\_eq**[\[source\]](#)

```
lemma velocity_eq {d : ℕ} (M : RigidBodyMotion d) (y : Space d) (t : Time) :  
  M.velocity y t = ∂t (fun s => M.displacement s y) t
```

**lemma velocity\_apply**[\[source\]](#)

*The  $i$ -th component of the velocity of a body point is the time derivative of the  $i$ -th coordinate of its inertial-frame trajectory.*

```
lemma velocity_apply {d : ℕ} (M : RigidBodyMotion d) (y : Space d) (t : Time) (i : Fin d)  
  (hd : Differentiable ℝ (fun s => M.displacement s y)) :  
  M.velocity y t i = ∂t (fun s => M.displacement s y i) t
```

**lemma velocity\_centerOfMass**[\[source\]](#)

*The material point at the centre of mass moves with the centre-of-mass velocity, for any motion:  $v(\text{centreOfMass}) = V$ . This is the velocity counterpart of `massDistribution_centerOfMass`.*

```
lemma velocity_centerOfMass {d : ℕ} (M : RigidBodyMotion d) :  
  M.velocity M.centerOfMass = M.centerOfMassVelocity
```

**lemma velocity\_of\_orientation\_const**[\[source\]](#)

*A rigid body in pure translation (constant orientation) has every point moving with the centre-of-mass velocity:  $v = V$ .*

```
lemma velocity_of_orientation_const {d : ℕ} (M : RigidBodyMotion d) (y : Space d)  
  (R : Matrix.specialOrthogonalGroup (Fin d) ℝ) (h : M.orientation = fun _ => R) :  
  M.velocity y = M.centerOfMassVelocity
```

**lemma velocity\_eq\_deriv\_orientation**[\[source\]](#)

*The velocity of a body point decomposes as  $v = \dot{R} (y - c) + V$ : the rate of change of the orientation acting on the body-frame position, plus the centre-of-mass velocity.*

```
lemma velocity_eq_deriv_orientation {d : ℕ} (M : RigidBodyMotion d) (y : Space d) (t : Time)  
  (i : Fin d) (hR : Differentiable ℝ (fun s => (M.orientation s).1))  
  (hX : Differentiable ℝ M.comTrajectory) :  
  M.velocity y t i  
  = (∂t (fun s => (M.orientation s).1) t *v fun j => y j - M.centerOfMass j) i  
  + M.centerOfMassVelocity t i
```

**def velocityClosedForm**[\[source\]](#)

*The closed form  $\dot{R}(t) (y - c) + V(t)$  of the velocity of the body point  $y$  at time  $t$ . Unlike `velocity` — whose junk values on non-differentiable motions need not vary*

continuously with  $y$  — it is polynomial in  $y$  for any motion, so its squared speed can be bundled as the smooth integrand of the total kinetic energy; for differentiable motions the two agree, see `velocityClosedForm_eq_velocity`.

```
noncomputable def velocityClosedForm {d : ℕ} (M : RigidBodyMotion d) (t : Time) (y : Space d) :
  Space d
```

**lemma velocityClosedForm\_apply**

[\[source\]](#)

*The  $i$ -th coordinate of the closed-form velocity.*

```
lemma velocityClosedForm_apply {d : ℕ} (M : RigidBodyMotion d) (t : Time) (y : Space d)
  (i : Fin d) :
  M.velocityClosedForm t y i
    = (∂t (fun s => (M.orientation s).1) t *v fun j => y j - M.centerOfMass j) i
      + M.centerOfMassVelocity t i
```

**lemma velocityClosedForm\_val**

[\[source\]](#)

*The closed-form velocity as a plain vector-valued function of the coordinates of  $y$ .*

```
lemma velocityClosedForm_val {d : ℕ} (M : RigidBodyMotion d) (t : Time) (y : Space d) :
  (M.velocityClosedForm t y : Fin d → ℝ)
    = (∂t (fun s => (M.orientation s).1) t *v fun j => y j - M.centerOfMass j)
      + (M.centerOfMassVelocity t : Fin d → ℝ)
```

**lemma velocityClosedForm\_eq\_velocity**

[\[source\]](#)

*For a differentiable motion the closed-form velocity is the honest point velocity.*

```
lemma velocityClosedForm_eq_velocity {d : ℕ} (M : RigidBodyMotion d) (t : Time)
  (hR : Differentiable ℝ (fun s => (M.orientation s).1))
  (hX : Differentiable ℝ M.comTrajectory) (y : Space d) :
  M.velocityClosedForm t y = M.velocity y t
```

**lemma contDiff\_velocityClosedForm\_inner**

[\[source\]](#)

*The squared speed of a body point, in closed form, is a smooth function of the point.*

```
lemma contDiff_velocityClosedForm_inner {d : ℕ} (M : RigidBodyMotion d) (t : Time) :
  ContDiff ℝ ⊤ fun y : Space d => (⟨⟨M.velocityClosedForm t y, M.velocityClosedForm t y⟩⟩_ℝ)
```

## 2.12 Physlib.CondensedMatter.Thermoelectric.Basic

[Physlib/CondensedMatter/Thermoelectric/Basic.lean](#) on GitHub.

**structure ThermoelectricMaterial**

[\[source\]](#)

*A thermoelectric material in the linear-response regime, characterized by its four transport coefficients.*

```
structure ThermoelectricMaterial where
  S : ℝ
  σ : ℝ
  κl : ℝ
  κe : ℝ
```

```

σ_pos : 0 < σ
κl_pos : 0 < κl
ke_nonneg : 0 ≤ ke

```

**def powerFactor**[\[source\]](#)

*The thermoelectric power factor  $PF = \sigma S^2$  of a material.*

```
def powerFactor (M : ThermoelectricMaterial) : ℝ
```

**lemma powerFactor\_pos**[\[source\]](#)

*The power factor is positive when the Seebeck coefficient is nonzero.*

```
lemma powerFactor_pos {M : ThermoelectricMaterial} (hS : M.S ≠ 0) :
  0 < M.powerFactor
```

**def totalThermalConductivity**[\[source\]](#)

*The total thermal conductivity  $\kappa_l + \kappa_e$  of a material, the sum of the lattice (phonon) and electronic contributions.*

```
def totalThermalConductivity (M : ThermoelectricMaterial) : ℝ
```

**lemma totalThermalConductivity\_pos**[\[source\]](#)

*The total thermal conductivity is positive.*

```
lemma totalThermalConductivity_pos (M : ThermoelectricMaterial) :
  0 < M.totalThermalConductivity
```

**def figureOfMerit**[\[source\]](#)

*The dimensionless thermoelectric figure of merit  $zT = PF \cdot T / (\kappa_l + \kappa_e)$  of a material at absolute temperature  $T$ .*

```
def figureOfMerit (M : ThermoelectricMaterial) (T : ℝ) : ℝ
```

**lemma figureOfMerit\_eq**[\[source\]](#)

*The figure of merit in its standard flat form  $zT = \sigma S^2 T / (\kappa_l + \kappa_e)$ .*

```
lemma figureOfMerit_eq (M : ThermoelectricMaterial) (T : ℝ) :
  M.figureOfMerit T = M.σ * M.S ^ 2 * T / (M.κl + M.κe)
```

**lemma figureOfMerit\_pos**[\[source\]](#)

*The figure of merit is positive at positive temperature when the Seebeck coefficient is nonzero.*

```
lemma figureOfMerit_pos {M : ThermoelectricMaterial} {T : ℝ}
  (hS : M.S ≠ 0) (hT : 0 < T) :
  0 < M.figureOfMerit T
```

**lemma figureOfMerit\_le\_of\_le**[\[source\]](#)

*Lowering the lattice thermal conductivity raises the figure of merit: if  $M.\kappa l \leq \kappa l'$  then the material with lattice conductivity  $\kappa l'$  (all other coefficients equal) has the smaller  $zT$ . This is the phonon-glass electron-crystal design principle: scatter phonons without degrading electronic transport. Note the hypotheses: no condition on the Seebeck coefficient is needed, since monotonicity only requires the numerator  $\sigma S^2 T$  to be nonnegative.*

```
lemma figureOfMerit_le_of_le (M : ThermoelectricMaterial) {κl' T : ℝ}
  (hpos : 0 < κl') (h : M.κl ≤ κl') (hT : 0 ≤ T) :
  figureOfMerit { M with κl := κl', κl_pos := hpos } T ≤ M.figureOfMerit T
```

**2.13 Physlib.FluidDynamics.CauchyFlow.Basic**

[Physlib/FluidDynamics/CauchyFlow/Basic.lean](#) on GitHub.

**structure CauchyFlow**[\[source\]](#)

*A fluid flow equipped with Cauchy stress and specific body-force fields.*

*These are the additional independent fields needed to state a local momentum balance. The Cauchy stress represents contact forces, and the specific body force represents volumetric force per unit mass. Pressure and viscosity enter through stress laws rather than as redundant fields of CauchyFlow itself.*

```
structure CauchyFlow (d : ℕ) extends FluidFlow d where
  stress : StressTensor d
  specificBodyForce : VectorField d
```

**2.14 Physlib.FluidDynamics.CauchyFlow.BodyForce**

[Physlib/FluidDynamics/CauchyFlow/BodyForce.lean](#) on GitHub.

**def HasBodyForcePotential**[\[source\]](#)

*A flow has body-force potential  $\Phi$  when its specific body force is minus the gradient of  $\Phi$ .*

```
def HasBodyForcePotential (d : ℕ) (flow : CauchyFlow d) (potential : Space d → ℝ) : Prop
```

**def HasConservativeBodyForce**[\[source\]](#)

*A flow has conservative body force when its specific body force has some potential.*

```
def HasConservativeBodyForce (d : ℕ) (flow : CauchyFlow d) : Prop
```

**2.15 Physlib.FluidDynamics.CauchyFlow.Inviscid**

[Physlib/FluidDynamics/CauchyFlow/Inviscid.lean](#) on GitHub.

**def IsInviscid**[\[source\]](#)

*A Cauchy flow is inviscid with pressure  $p$  when its stress is the isotropic pressure stress  $-p I$ .*

*In this formulation, inviscid means that the Cauchy stress has no viscous or shear contribution. It does not rule out body forces; those are carried separately by `CauchyFlow.specificBodyForce`.*

```
def IsInviscid (d : ℕ) (flow : CauchyFlow d) (pressure : ScalarField d) : Prop
```

**lemma matrixDiv\_inviscid\_pressure\_stress**

[\[source\]](#)

*The matrix divergence of the inviscid pressure stress  $-p I$  is  $-\text{grad } p$ .*

```
lemma matrixDiv_inviscid_pressure_stress (d : ℕ) (pressureAtTime : Space d → ℝ) :
  matrixDiv d (fun x => (-(pressureAtTime x)) • (1 : Matrix (Fin d) (Fin d) ℝ)) =
    -∇ pressureAtTime
```

**theorem matrixDiv\_stress\_eq\_neg\_grad\_pressure\_of\_is\_inviscid**

[\[source\]](#)

*In an inviscid Cauchy flow, the stress-divergence force is the usual negative pressure-gradient term.*

```
theorem matrixDiv_stress_eq_neg_grad_pressure_of_is_inviscid
  (d : ℕ) (flow : CauchyFlow d) (pressure : ScalarField d)
  (hInviscid : IsInviscid d flow pressure) :
  ∀ t, matrixDiv d (flow.stress t) = -∇ (pressure t)
```

## 2.16 Physlib.FluidDynamics.CauchyFlow.Momentum

[Physlib/FluidDynamics/CauchyFlow/Momentum.lean](#) on GitHub.

**def CauchyMomentumEquation**

[\[source\]](#)

*Conservation of momentum in conservative matrix-divergence form.*

*The equation is*

$$\text{partial\_t } (\rho u) + \text{matrixDiv } (\rho u \otimes u) = \text{matrixDiv } \sigma + \rho f.$$

*Here stress is intentionally not yet specialized to a constitutive law.*

```
def CauchyMomentumEquation (d : ℕ) (flow : CauchyFlow d) : Prop
```

**def ConvectiveCauchyMomentumEquation**

[\[source\]](#)

*Conservation of momentum in convective form.*

*The equation is*

$$\rho (\text{partial\_t } u + (u \cdot \nabla)u) = \text{matrixDiv } \sigma + \rho f.$$

*Here stress is intentionally not yet specialized to a constitutive law.*

```
def ConvectiveCauchyMomentumEquation (d : ℕ) (flow : CauchyFlow d) : Prop
```

**lemma cauchyMomentumEquation\_iff\_convectiveCauchyMomentumEquation**

[\[source\]](#)

*The conservative and convective Cauchy momentum equations are equivalent when the classical continuity equation holds.*

*The differentiability assumptions are exactly the product-rule assumptions used to rewrite `partial_t (rho u)` and `matrixDiv (rho u ⊗ u)`.*

```
lemma cauchyMomentumEquation_iff_convectiveCauchyMomentumEquation
  (d : ℕ) (flow : CauchyFlow d)
  (hContinuity : FluidFlow.ClassicalContinuityEquation d flow.toFluidFlow)
  (hRhoTime : ∀ t x, DifferentiableAt ℝ (flow.rho · x) t)
  (hVelocityTime : ∀ t x, DifferentiableAt ℝ (flow.velocity · x) t)
  (hMomentumDensity : ∀ t, Differentiable ℝ (FluidFlow.momentumDensity d flow.toFluidFlow t))
  (hVelocitySpace : ∀ t, Differentiable ℝ (flow.velocity t)) :
  CauchyMomentumEquation d flow ↔ ConvectiveCauchyMomentumEquation d flow
```

## 2.17 Physlib.FluidDynamics.CauchyFlow.NavierStokes

[Physlib/FluidDynamics/CauchyFlow/NavierStokes.lean](#) on GitHub.

**theorem navier\_stokes\_iff\_convective\_navier\_stokes** [\[source\]](#)

*The conservative and convective Navier-Stokes forms are equivalent when the fields are differentiable enough for the product rules.*

```
theorem navier_stokes_iff_convective_navier_stokes
  (d : ℕ) (flow : CauchyFlow d)
  (pressure shearViscosity secondViscosity : ScalarField d)
  (hRhoTime : ∀ t x, DifferentiableAt ℝ (flow.rho · x) t)
  (hVelocityTime : ∀ t x, DifferentiableAt ℝ (flow.velocity · x) t)
  (hMomentumDensity : ∀ t,
    Differentiable ℝ (FluidFlow.momentumDensity d flow.toFluidFlow t))
  (hVelocitySpace : ∀ t, Differentiable ℝ (flow.velocity t)) :
  NavierStokes d flow pressure shearViscosity secondViscosity ↔
  ConvectiveNavierStokes d flow pressure shearViscosity secondViscosity
```

## 2.18 Physlib.FluidDynamics.CauchyFlow.Newtonian

[Physlib/FluidDynamics/CauchyFlow/Newtonian.lean](#) on GitHub.

**def IsNewtonian** [\[source\]](#)

*A Cauchy flow is Newtonian when its stress tensor has the Newtonian constitutive form.*

```
def IsNewtonian
  (d : ℕ) (flow : CauchyFlow d)
  (pressure shearViscosity secondViscosity : ScalarField d) : Prop
```

## 2.19 Physlib.FluidDynamics.Euler.Basic

[Physlib/FluidDynamics/Euler/Basic.lean](#) on GitHub.

**def Euler** [\[source\]](#)

*The conservative Euler equations: continuity, Cauchy momentum, and inviscid stress.*

```
def Euler (d : ℕ) (flow : CauchyFlow d) (pressure : ScalarField d) : Prop
```

**def ConvectiveEuler**[\[source\]](#)

*The convective Euler equations: continuity, convective Cauchy momentum, and inviscid stress.*

```
def ConvectiveEuler (d : ℕ) (flow : CauchyFlow d) (pressure : ScalarField d) : Prop
```

**theorem euler\_iff\_convectiveEuler**[\[source\]](#)

*The conservative and convective Euler forms are equivalent when the fields are differentiable enough for the product rules.*

```
theorem euler_iff_convectiveEuler
  (d : ℕ) (flow : CauchyFlow d) (pressure : ScalarField d)
  (hRhoTime : ∀ t x, DifferentiableAt ℝ (flow.rho · x) t)
  (hVelocityTime : ∀ t x, DifferentiableAt ℝ (flow.velocity · x) t)
  (hMomentumDensity : ∀ t,
    Differentiable ℝ (FluidFlow.momentumDensity d flow.toFluidFlow t))
  (hVelocitySpace : ∀ t, Differentiable ℝ (flow.velocity t)) :
  Euler d flow pressure ↔ ConvectiveEuler d flow pressure
```

**2.20 Physlib.FluidDynamics.FluidFlow.Basic**

[Physlib/FluidDynamics/FluidFlow/Basic.lean](#) on GitHub.

**structure FluidFlow**[\[source\]](#)

*The density and velocity fields of a fluid on  $d$ -dimensional space.*

*These fields are the independent data needed for kinematics and mass transport. Quantities such as mass flux, momentum density, and material acceleration are derived from them. Dynamic and thermodynamic fields are introduced only by extension structures whose statements need them.*

```
structure FluidFlow (d : ℕ) where
  rho : MassDensity d
  velocity : VelocityField d
```

**2.21 Physlib.FluidDynamics.FluidFlow.Continuity**

[Physlib/FluidDynamics/FluidFlow/Continuity.lean](#) on GitHub.

**def ClassicalContinuityEquation**[\[source\]](#)

*Classical conservation of mass in conservative form,  $\text{partial\_t } \rho + \text{div } (\rho u) = 0$ .*

*The equation is asserted at points where the time derivative of  $\rho$  and the spatial divergence of  $\rho u$  are classical derivatives.*

```
def ClassicalContinuityEquation (d : ℕ) (fluid : FluidFlow d) : Prop
```

**def continuityResidual**[\[source\]](#)

*The scalar continuity-equation residual  $\text{partial\_t } \rho + \text{div } (\rho u)$ .*

```
noncomputable def continuityResidual (d : ℕ) (fluid : FluidFlow d) : Time → Space d → ℝ
```

**def SmoothContinuityEquation**

[\[source\]](#)

*A stronger continuity equation for globally differentiable fields.*

*This version records the first-order regularity needed by the classical continuity equation: the density is differentiable in time, the mass flux  $\rho u$  is differentiable in space, and the continuity residual vanishes everywhere.*

```
def SmoothContinuityEquation (d : ℕ) (fluid : FluidFlow d) : Prop
```

**lemma classicalContinuityEquation\_of\_smoothContinuityEquation**

[\[source\]](#)

*A smooth continuity equation satisfies the guarded classical continuity equation.*

```
lemma classicalContinuityEquation_of_smoothContinuityEquation
  (d : ℕ) (fluid : FluidFlow d) :
  SmoothContinuityEquation d fluid → ClassicalContinuityEquation d fluid
```

## 2.22 Physlib.FluidDynamics.FluidFlow.Incompressible

[Physlib/FluidDynamics/FluidFlow/Incompressible.lean](#) on GitHub.

**def incompressibilityResidual**

[\[source\]](#)

*The incompressibility residual, given by the divergence of the velocity field.*

```
noncomputable def incompressibilityResidual (d : ℕ) (fluid : FluidFlow d) :
  Time → Space d → ℝ
```

**def ClassicalIncompressible**

[\[source\]](#)

*A classical incompressible flow has divergence-free velocity at points where the velocity field is differentiable.*

```
def ClassicalIncompressible (d : ℕ) (fluid : FluidFlow d) : Prop
```

**def SmoothIncompressible**

[\[source\]](#)

*A smooth incompressible flow has globally differentiable velocity and vanishing incompressibility residual everywhere.*

```
def SmoothIncompressible (d : ℕ) (fluid : FluidFlow d) : Prop
```

**lemma classicalIncompressible\_of\_smoothIncompressible**

[\[source\]](#)

*A smooth incompressible flow is classically incompressible.*

```
lemma classicalIncompressible_of_smoothIncompressible
  (d : ℕ) (fluid : FluidFlow d) :
  SmoothIncompressible d fluid → ClassicalIncompressible d fluid
```



## 2.23 Physlib.FluidDynamics.FluidFlow.Kinematics

[Physlib/FluidDynamics/FluidFlow/Kinematics.lean](#) on GitHub.

### def materialDerivative

[\[source\]](#)

*The material derivative  $D_t f = \text{partial}_t f + u \cdot \text{grad } f$  of a scalar field.*

```
noncomputable def materialDerivative (d : ℕ) (fluid : FluidFlow d)
  (field : ScalarField d) : ScalarField d
```

### def specificKineticEnergy

[\[source\]](#)

*The specific kinetic energy  $|u|^2 / 2$  of a fluid flow.*

```
noncomputable def specificKineticEnergy (d : ℕ) (fluid : FluidFlow d) : ScalarField d
```

## 2.24 Physlib.FluidDynamics.FluidFlow.Momentum

[Physlib/FluidDynamics/FluidFlow/Momentum.lean](#) on GitHub.

### def momentumDensity

[\[source\]](#)

*The momentum density  $\rho u$ .*

```
def momentumDensity (d : ℕ) (fluid : FluidFlow d) : MomentumDensityField d
```

### def momentumFlux

[\[source\]](#)

*The convective momentum flux  $\rho u \otimes u$ .*

```
def momentumFlux (d : ℕ) (fluid : FluidFlow d) : Time → Space d → Matrix (Fin d) (Fin d) ℝ
```

### def convectiveTerm

[\[source\]](#)

*The nonlinear transport term  $(u \cdot \nabla)u$ .*

```
noncomputable def convectiveTerm (d : ℕ) (fluid : FluidFlow d) : VectorField d
```

### def materialAcceleration

[\[source\]](#)

*The material acceleration  $\partial_t u + (u \cdot \nabla)u$ .*

```
noncomputable def materialAcceleration (d : ℕ) (fluid : FluidFlow d) : VectorField d
```

### lemma timeDeriv\_smul\_velocity

[\[source\]](#)

*Product rule for the time derivative of a scalar field times a velocity field.*

```
lemma timeDeriv_smul_velocity (d : ℕ) (rhoAtPosition : Time → ℝ)
  (velocityAtPosition : Time → EuclideanSpace ℝ (Fin d)) (t : Time)
  (hRho : DifferentiableAt ℝ rhoAtPosition t)
  (hVelocity : DifferentiableAt ℝ velocityAtPosition t) :
  ∂_t (fun t' => rhoAtPosition t' • velocityAtPosition t') t =
    rhoAtPosition t • ∂_t velocityAtPosition t + ∂_t rhoAtPosition t • velocityAtPosition t
```

**lemma timeDeriv\_momentumDensity**[\[source\]](#)

*Product rule for the time derivative of the momentum density  $\rho u$ .*

```
lemma timeDeriv_momentumDensity (d : ℕ) (fluid : FluidFlow d)
  (t : Time) (x : Space d)
  (hRho : DifferentiableAt ℝ (fluid.rho · x) t)
  (hVelocity : DifferentiableAt ℝ (fluid.velocity · x) t) :
  ∂ₜ (momentumDensity d fluid · x) t =
    fluid.rho t x • ∂ₜ (fluid.velocity · x) t + ∂ₜ (fluid.rho · x) t • fluid.velocity t x
```

**lemma spaceDeriv\_momentumFlux\_component**[\[source\]](#)

*Product rule for one spatial derivative of one component of  $\rho u \otimes u$ .*

```
lemma spaceDeriv_momentumFlux_component (d : ℕ) (fluid : FluidFlow d)
  (t : Time) (x : Space d) (i j : Fin d)
  (hMomentumDensity : Differentiable ℝ (momentumDensity d fluid t))
  (hVelocity : Differentiable ℝ (fluid.velocity t)) :
  ∂[j] (fun x' => momentumFlux d fluid t x' i j) x =
    fluid.velocity t x i • ∂[j] (fun x' => momentumDensity d fluid t x' j) x +
    ∂[j] (fun x' => fluid.velocity t x' i) x • momentumDensity d fluid t x j
```

**lemma matrixDiv\_momentumFlux**[\[source\]](#)

*The matrix divergence of  $\rho u \otimes u$  split into continuity and convective parts.*

```
lemma matrixDiv_momentumFlux (d : ℕ) (fluid : FluidFlow d)
  (t : Time) (x : Space d)
  (hMomentumDensity : Differentiable ℝ (momentumDensity d fluid t))
  (hVelocity : Differentiable ℝ (fluid.velocity t)) :
  matrixDiv d (momentumFlux d fluid t) x =
    (∇ · momentumDensity d fluid t) x • fluid.velocity t x +
    fluid.rho t x • convectiveTerm d fluid t x
```

**lemma momentumTransport\_eq\_materialAcceleration\_add\_continuityResidual**[\[source\]](#)

*The conservative momentum transport terms equal mass density times material acceleration plus the continuity residual times velocity.*

```
lemma momentumTransport_eq_materialAcceleration_add_continuityResidual
  (d : ℕ) (fluid : FluidFlow d)
  (t : Time) (x : Space d)
  (hRhoTime : DifferentiableAt ℝ (fluid.rho · x) t)
  (hVelocityTime : DifferentiableAt ℝ (fluid.velocity · x) t)
  (hMomentumDensity : Differentiable ℝ (momentumDensity d fluid t))
  (hVelocitySpace : Differentiable ℝ (fluid.velocity t)) :
  ∂ₜ (momentumDensity d fluid · x) t + matrixDiv d (momentumFlux d fluid t) x =
    fluid.rho t x • materialAcceleration d fluid t x +
    continuityResidual d fluid t x • fluid.velocity t x
```

**2.25 Physlib.FluidDynamics.FluidFlow.Newtonian**

[Physlib/FluidDynamics/FluidFlow/Newtonian.lean](#) on GitHub.

**def velocityGradient**[\[source\]](#)

*The spatial velocity-gradient matrix, with entries  $\text{partial}_j u_i$ .*

```
noncomputable def velocityGradient (d : ℕ) (flow : FluidFlow d) :
  Time → Space d → Matrix (Fin d) (Fin d) ℝ
```

**def newtonianStressTensor**[\[source\]](#)

*The Newtonian stress tensor  $-p I + \mu (\text{grad } u + \text{grad } u^T) + \lambda (\text{div } u) I$ .*

*The scalar fields are pressure, shear viscosity, and second viscosity.*

```
noncomputable def newtonianStressTensor (d : ℕ) (flow : FluidFlow d)
  (pressure shearViscosity secondViscosity : ScalarField d) : StressTensor d
```

**2.26 Physlib.FluidDynamics.ThermodynamicCauchyFlow.Basic**

[Physlib/FluidDynamics/ThermodynamicCauchyFlow/Basic.lean](#) on GitHub.

**structure ThermodynamicCauchyFlow**[\[source\]](#)

*A Cauchy flow equipped with thermodynamic entropy and enthalpy fields.*

*These fields are independent of the kinematic and momentum-balance data unless additional thermodynamic constitutive laws are imposed. They are sufficient for the current isentropic and Bernoulli APIs; other thermodynamic fields belong in later extensions that require them.*

```
structure ThermodynamicCauchyFlow (d : ℕ) extends CauchyFlow d where
  entropy : ScalarField d
  enthalpy : ScalarField d
```

**2.27 Physlib.FluidDynamics.ThermodynamicCauchyFlow.Bernoulli**

[Physlib/FluidDynamics/ThermodynamicCauchyFlow/Bernoulli.lean](#) on GitHub.

**def bernoulliFunction**[\[source\]](#)

*The Bernoulli function  $|u|^2 / 2 + h + \Phi$ .*

```
noncomputable def bernoulliFunction
  (d : ℕ) (flow : ThermodynamicCauchyFlow d) (potential : Space d → ℝ) : ScalarField d
```

**def LocalBernoulliLaw**[\[source\]](#)

*A local Bernoulli law: the Bernoulli function has zero spatial gradient.*

```
def LocalBernoulliLaw
  (d : ℕ) (flow : ThermodynamicCauchyFlow d) (potential : Space d → ℝ) : Prop
```

**def BernoulliLaw**[\[source\]](#)

*A global Bernoulli law: the Bernoulli function is spatially constant at each time.*

```
def BernoulliLaw
```

```
(d : ℕ) (flow : ThermodynamicCauchyFlow d) (potential : Space d → ℝ) : Prop
```

## 2.28 Physlib.FluidDynamics.ThermodynamicCauchyFlow.Isentropic

[Physlib/FluidDynamics/ThermodynamicCauchyFlow/Isentropic.lean](#) on GitHub.

```
def IsIsentropic
```

[\[source\]](#)

*A thermodynamic flow is isentropic when the entropy is materially conserved along the underlying fluid velocity field.*

```
def IsIsentropic (d : ℕ) (flow : ThermodynamicCauchyFlow d) : Prop
```

## 2.29 Physlib.Mathematics.CrossProduct

[Physlib/Mathematics/CrossProduct.lean](#) on GitHub.

```
lemma cross_cross_self_apply
```

[\[source\]](#)

*The component form of the triple cross product  $v \times_3 (w \times_3 v)$ : by the bac-cab identity its  $i$ -th entry is  $|v|^2 w_i - (v \cdot w) v_i$ , written with the explicit component sums  $\sum_k, (v_k)^2$  and  $\sum_j, v_j * w_j$ .*

```
lemma cross_cross_self_apply (v w : Fin 3 → ℝ) (i : Fin 3) :
(v ×₃ (w ×₃ v)) i = (∑ k, (v k) ^ 2) * w i - (∑ j, v j * w j) * v i
```

```
lemma dotProduct_cross_cross_self
```

[\[source\]](#)

*Contracting  $w$  with the triple cross product  $v \times_3 (w \times_3 v)$  gives  $(w \times_3 v) \cdot_v (w \times_3 v)$  (over  $\mathbb{R}$ , the squared length  $|w \times v|^2$ ), by two cyclic permutations of the scalar triple product.*

```
lemma dotProduct_cross_cross_self {R : Type*} [CommRing R] (v w : Fin 3 → R) :
w ·_v (v ×₃ (w ×₃ v)) = (w ×₃ v) ·_v (w ×₃ v)
```

```
lemma contDiff_cross_dotProduct_cross
```

[\[source\]](#)

*The squared length  $(w \times_3 v) \cdot_v (w \times_3 v)$  of the cross product with a fixed vector  $w$  is a smooth function of  $v$ .*

```
lemma contDiff_cross_dotProduct_cross (w : Fin 3 → ℝ) :
ContDiff ℝ ⊤ fun v : Fin 3 → ℝ => (w ×₃ v) ·_v (w ×₃ v)
```

## 2.30 Physlib.Mathematics.CrossProductMatrix

[Physlib/Mathematics/CrossProductMatrix.lean](#) on GitHub.

```
lemma crossProductMatrix_crossProductVee
```

[\[source\]](#)

*On skew-symmetric matrices the hat map is also a right inverse of the vee map: if  $A^T = -A$  then  $[A^v]_\times = A$ . Together with `crossProductVee_crossProductMatrix` this identifies  $\mathbb{R}^3$  with the skew-symmetric  $3 \times 3$  matrices `so(3)`.*

```
lemma crossProductMatrix_crossProductVee {A : Matrix (Fin 3) (Fin 3) ℝ} (hA : AT = -A) :
  crossProductMatrix (crossProductVee A) = A
```

### 2.31 Physlib.Mathematics.Distribution.Basic

[Physlib/Mathematics/Distribution/Basic.lean](#) on GitHub.

**def** toTemperedDistribution

[\[source\]](#)

*The Mathlib tempered distribution associated to a Physlib distribution.*

```
def toTemperedDistribution (u : E →d[ℂ] F) : S'(E, F)
```

**lemma** toTemperedDistribution\_apply

[\[source\]](#)

@[simp]

```
lemma toTemperedDistribution_apply (u : E →d[ℂ] F) (η : S(E, ℂ)) :
  u.toTemperedDistribution η = u η
```

**lemma** toTemperedDistribution\_fourierTransform

[\[source\]](#)

*Conversion to Mathlib tempered distributions commutes with the Fourier transform.*

@[simp]

```
lemma toTemperedDistribution_fourierTransform (u : E →d[ℂ] F) :
  (u.fourierTransform E F).toTemperedDistribution = ℱ u.toTemperedDistribution
```

**def** offFiniteMeasure

[\[source\]](#)

*The scalar distribution associated to a finite measure, acting on a Schwartz map by integration.*

```
def offFiniteMeasure (μ : Measure E) [IsFiniteMeasure μ] : E →d[ℝ] ℝ
```

**lemma** offFiniteMeasure\_apply

[\[source\]](#)

@[simp]

```
lemma offFiniteMeasure_apply (μ : Measure E) [IsFiniteMeasure μ] (η : S(E, ℝ)) :
  offFiniteMeasure ℝ μ η = ∫ x, η x ∂μ
```

**lemma** toTemperedDistribution\_offFiniteMeasure

[\[source\]](#)

*The finite-measure distribution agrees with Mathlib's tempered distribution associated to the same measure.*

@[simp]

```
lemma toTemperedDistribution_offFiniteMeasure (μ : Measure E) [IsFiniteMeasure μ] :
  (offFiniteMeasure ℝ μ).toTemperedDistribution = μ.toTemperedDistribution
```

**lemma lipschitzWith\_integral\_of\_le**[\[source\]](#)

```
private lemma lipschitzWith_integral_of_le {μ ρ : Measure E} (hμρ : μ ≤ ρ) :
  LipschitzWith 1 (fun f : Lp C 1 ρ => ∫ x, f x ∂μ)
```

**lemma integral\_boundedContinuous\_eq\_of\_forall\_schwartz\_integral\_eq**[\[source\]](#)

```
private lemma integral_boundedContinuous_eq_of_forall_schwartz_integral_eq
  [FiniteDimensional ℝ E] {μ ν : Measure E} [IsFiniteMeasure μ] [IsFiniteMeasure ν]
  (h : ∀ η : S(E, ℂ), ∫ x, η x ∂μ = ∫ x, η x ∂ν)
  (f : BoundedContinuousFunction E ℂ) :
  ∫ x, f x ∂μ = ∫ x, f x ∂ν
```

**lemma measure\_eq\_of\_forall\_schwartz\_integral\_eq**[\[source\]](#)

```
private lemma measure_eq_of_forall_schwartz_integral_eq
  {E : Type} [NormedAddCommGroup E] [NormedSpace ℝ E] [MeasurableSpace E]
  [BorelSpace E] [SecondCountableTopology E] [FiniteDimensional ℝ E]
  {μ ν : Measure E} [IsFiniteMeasure μ] [IsFiniteMeasure ν]
  (h : ∀ η : S(E, ℂ), ∫ x, η x ∂μ = ∫ x, η x ∂ν) :
  μ = ν
```

**lemma ofFiniteMeasure\_eq\_iff**[\[source\]](#)

*The complex scalar distribution associated to a finite measure determines the measure.*

```
lemma ofFiniteMeasure_eq_iff
  {E : Type} [NormedAddCommGroup E] [NormedSpace ℝ E] [MeasurableSpace E]
  [BorelSpace E] [SecondCountableTopology E] [FiniteDimensional ℝ E]
  {μ ν : Measure E} [IsFiniteMeasure μ] [IsFiniteMeasure ν] :
  ofFiniteMeasure C μ = ofFiniteMeasure C ν ↔ μ = ν
```

**2.32 Physlib.Mathematics.InnerProductSpace.Gaussian**

[Physlib/Mathematics/InnerProductSpace/Gaussian.lean](#) on GitHub.

**lemma norm\_iteratedFDeriv\_innerSL\_le\_pow**[\[source\]](#)

*Crude bounds on the norms of iterated Fréchet derivatives of innerSL ℝ in the form required by norm\_iteratedFDeriv\_comp\_le.*

```
private lemma norm_iteratedFDeriv_innerSL_le_pow {n : ℕ} (hn : 1 ≤ n) :
  ||iteratedFDeriv ℝ n (fun x : E ↦ innerSL ℝ x x) x|| ≤ (2 + ||x|| ^ 2) ^ n
```

**lemma pow\_mul\_exp\_bddAbove**[\[source\]](#)

```
private lemma pow_mul_exp_bddAbove {s : ℝ} (hs : 0 ≤ s) (n : ℕ) :
  ∃ C > 0, ∀ x ≥ 0, x ^ s * (2 + x) ^ n * rexp (-2-1 * x) ≤ C
```

**def realStdGaussian**[\[source\]](#)

*The (unnormalized) real-valued standard Gaussian  $\exp (-2^{-1} * ||x|| ^ 2)$  as a Schwartz map.*

*This definition is used to construct the general  $RCLike$ -valued Gaussian and the preferred version to use is `gaussian  $\mathbb{R} B x_0$`  (or `stdGaussian  $E \mathbb{R}$`  for mean zero, variance one).*

```
def realStdGaussian :  $\mathcal{S}(E, \mathbb{R})$  where
```

**def gaussian**

[\[source\]](#)

*The (unnormalized)  $\mathbb{k}$ -valued Gaussian  $\exp (-2^{-1} * \|B^{-1} (x - x_0)\|^2)$  as a Schwartz map.*

```
def gaussian :  $\mathcal{S}(E, \mathbb{k})$ 
```

**lemma gaussian\_apply**

[\[source\]](#)

```
@[simp]
```

```
lemma gaussian_apply : gaussian  $\mathbb{k} B x_0 x = \text{ofReal} (\text{rexp} (-2^{-1} * \|B.\text{symm} (x - x_0)\|^2))$ 
```

**abbrev gaussian<sub>0</sub>**

[\[source\]](#)

*The (unnormalized)  $\mathbb{k}$ -valued Gaussian  $\exp (-2^{-1} * \|B^{-1} x\|^2)$  as a Schwartz map.*

```
abbrev gaussian0 :  $\mathcal{S}(E, \mathbb{k})$ 
```

**abbrev stdGaussian**

[\[source\]](#)

*The (unnormalized)  $\mathbb{k}$ -valued standard Gaussian  $\exp (-2^{-1} * \|x\|^2)$  as a Schwartz map.*

```
abbrev stdGaussian :  $\mathcal{S}(E, \mathbb{k})$ 
```

## 2.33 Physlib.Mathematics.KroneckerDelta.Basic

[Physlib/Mathematics/KroneckerDelta/Basic.lean](#) on GitHub.

**lemma kroneckerDelta\_finSumFinEquiv**

[\[source\]](#)

*The Kronecker delta is invariant under the component-index equivalence `finSumFinEquiv`.*

```
lemma kroneckerDelta_finSumFinEquiv (a b : Fin 1  $\oplus$  Fin 3) :  
  kroneckerDelta (finSumFinEquiv a) (finSumFinEquiv b) = kroneckerDelta a b
```

## 2.34 Physlib.Mathematics.KroneckerDelta.Contraction

[Physlib/Mathematics/KroneckerDelta/Contraction.lean](#) on GitHub.

**lemma det\_add\_rankOne\_aux**

[\[source\]](#)

*Expanding the determinant of a rank-one row update over a finite set of rows. For  $i \in s$  the row  $A i$  is replaced by  $A i + w i \cdot b$ ; the other rows are untouched.*

```
private lemma det_add_rankOne_aux {ι : Type*} [DecidableEq ι] [Fintype ι] {R : Type*}
  [CommRing R] (A : Matrix ι ι R) (w b : ι → R) (s : Finset ι) :
  (A + Matrix.of fun i j => (if i ∈ s then w i else 0) * b j).det
  = A.det + ∑ i ∈ s, w i * (A.updateRow i b).det
```

### lemma det\_add\_rankOne

[\[source\]](#)

*\*\*Rank-one determinant update\*\* (the ring-general matrix determinant lemma for an outer product, valid even when  $A$  is singular). Adding the rank-one matrix  $w \otimes b$  to  $A$  changes the determinant by  $\sum i, w i * \det (A.updateRow i b)$ .*

*Mathlib only provides this when  $\det A$  is a unit (`Matrix.det_add_replicateCol_mul_replicateRow`); the singular case is needed here because Kronecker-delta matrices are typically singular.*

```
private lemma det_add_rankOne {ι : Type*} [DecidableEq ι] [Fintype ι] {R : Type*}
  [CommRing R] (A : Matrix ι ι R) (w b : ι → R) :
  (A + Matrix.of fun i j => w i * b j).det = A.det + ∑ i, w i * (A.updateRow i b).det
```

### lemma generalizedKroneckerDelta\_mul

[\[source\]](#)

*The product of two Levi-Civita-type symbols is a generalized Kronecker delta:  $\delta^{\{\mu\}}_{\{\cdot\}} \cdot \delta^{\{\nu\}}_{\{\cdot\}} = \delta^{\{\mu\}}_{\{\nu\}}$ , where each single factor is a Kronecker matrix against the identity. This is the Lean form of  $\epsilon^{\{\mu_1 \dots \mu_n\}} \epsilon_{\{\nu_1 \dots \nu_n\}} = \delta^{\{\mu_1 \dots \mu_n\}}_{\{\nu_1 \dots \nu_n\}}$ .*

```
lemma generalizedKroneckerDelta_mul (μ ν : α → α) :
  generalizedKroneckerDelta μ id * generalizedKroneckerDelta ν id
  = generalizedKroneckerDelta μ ν
```

### lemma generalizedKroneckerDelta\_sum\_snoc

[\[source\]](#)

*\*\*Generalized Kronecker delta contraction.\*\* Summing a `generalizedKroneckerDelta` over one shared index appended at the end lowers the rank by one and pulls out a factor of  $\text{card } \alpha - n$ . This is the reusable combinatorial fact behind all epsilon-epsilon identities.*

```
lemma generalizedKroneckerDelta_sum_snoc {n : ℕ} (μ ν : Fin n → α) :
  ∑ a : α, generalizedKroneckerDelta (Fin.snoc μ a) (Fin.snoc ν a)
  = ((Fintype.card α : ℤ) - n) * generalizedKroneckerDelta μ ν
```

### lemma sum\_over\_snoc

[\[source\]](#)

*Split a sum over  $(k+1)$ -tuples into the last entry and the initial  $k$ -tuple.*

```
private lemma sum_over_snoc {X : Type*} [Fintype X] {M : Type*} [AddCommMonoid M] {k : ℕ}
  (F : (Fin (k + 1) → X) → M) :
  ∑ h : Fin (k + 1) → X, F h = ∑ h' : Fin k → X, ∑ c : X, F (Fin.snoc h' c)
```

### lemma sum\_generalizedKroneckerDelta\_self

[\[source\]](#)

*\*\*Full contraction.\*\* Iterating the snoc contraction over all four indices:  $\sum_f \delta^{\{f\}}_{\{f\}} = 4!$ . Here  $f$  ranges over all maps  $\text{Fin } 4 \rightarrow \text{Fin } 4$ .*



```

lemma sum_generalizedKroneckerDelta_self (k : ℕ) :
  ∑ h : Fin k → Fin 4, generalizedKroneckerDelta h h
  = ∏ j ∈ Finset.range k, ((4 : ℤ) - j)

```

**lemma sum\_generalizedKroneckerDelta\_cons**

[\[source\]](#)

*\*\*Single contraction.\*\* Contracting the last  $k$  of  $k+1$  index pairs leaves one free pair  $\sigma, \tau$ , with the factorial factor  $(4-1)(4-2)\dots$ .*

```

lemma sum_generalizedKroneckerDelta_cons (σ τ : Fin 4) (k : ℕ) :
  ∑ h : Fin k → Fin 4,
    generalizedKroneckerDelta (Fin.cons σ h) (Fin.cons τ h)
  = (∏ j ∈ Finset.range k, ((3 : ℤ) - j)) * ((kroneckerDelta σ τ : ℕ) : ℤ)

```

**lemma sum\_generalizedKroneckerDelta\_cons₂**

[\[source\]](#)

*\*\*Double contraction.\*\* Contracting the last  $k$  of  $k+2$  index pairs leaves two free pairs, with value a  $2 \times 2$  generalized Kronecker delta times the factorial factor.*

```

lemma sum_generalizedKroneckerDelta_cons₂ (ρ σ τ ω : Fin 4) (k : ℕ) :
  ∑ h : Fin k → Fin 4,
    generalizedKroneckerDelta (Fin.cons ρ (Fin.cons σ h)) (Fin.cons τ (Fin.cons ω h))
  = (∏ j ∈ Finset.range k, ((2 : ℤ) - j))
    * generalizedKroneckerDelta ![ρ, σ] ![τ, ω]

```

**lemma sum\_generalizedKroneckerDelta\_mul\_self**

[\[source\]](#)

*Symbol-level full contraction over  $\text{Fin } 4 \rightarrow \text{Fin } 4$ .*

```

lemma sum_generalizedKroneckerDelta_mul_self :
  ∑ g : Fin 4 → Fin 4,
    generalizedKroneckerDelta g id * generalizedKroneckerDelta g id = (24 : ℤ)

```

**lemma sum\_generalizedKroneckerDelta\_mul\_cons**

[\[source\]](#)

*Symbol-level triple contraction, one free pair  $\sigma, \tau$ .*

```

lemma sum_generalizedKroneckerDelta_mul_cons (σ τ : Fin 4) :
  ∑ h : Fin 3 → Fin 4,
    generalizedKroneckerDelta (Fin.cons σ h) id
    * generalizedKroneckerDelta (Fin.cons τ h) id
  = 6 * ((kroneckerDelta σ τ : ℕ) : ℤ)

```

**lemma sum\_generalizedKroneckerDelta\_mul\_cons₂**

[\[source\]](#)

*Symbol-level double contraction, two free pairs.*

```

lemma sum_generalizedKroneckerDelta_mul_cons₂ (ρ σ τ ω : Fin 4) :
  ∑ h : Fin 2 → Fin 4,
    generalizedKroneckerDelta (Fin.cons ρ (Fin.cons σ h)) id
    * generalizedKroneckerDelta (Fin.cons τ (Fin.cons ω h)) id
  = 2 * (((kroneckerDelta ρ τ : ℕ) : ℤ) * ((kroneckerDelta σ ω : ℕ) : ℤ)
    - ((kroneckerDelta ρ ω : ℕ) : ℤ) * ((kroneckerDelta σ τ : ℕ) : ℤ))

```

## 2.35 Physlib.Mathematics.OrthogonalMatrix

[Physlib/Mathematics/OrthogonalMatrix.lean](#) on GitHub.

### lemma dotProduct\_mulVec\_orthogonal

[\[source\]](#)

*An orthogonal matrix preserves the dot product: if  $A^T A = 1$  then  $(A \cdot v) \cdot_v (A \cdot w) = v \cdot_v w$ . Taking  $w = v$  shows orthogonal matrices preserve squared lengths,  $(A \cdot v) \cdot_v (A \cdot v) = v \cdot_v v$ .*

```
lemma dotProduct_mulVec_orthogonal {R : Type*} [CommRing R] {n : Type*} [Fintype n]
  [DecidableEq n] {A : Matrix n n R} (hA : AT * A = 1) (v w : n → R) :
  (A *v v) ·v (A *v w) = v ·v w
```

## 2.36 Physlib.Mathematics.Resolvent

[Physlib/Mathematics/Resolvent.lean](#) on GitHub.

### lemma mem\_resolventSet\_of\_im\_ne\_zero

[\[source\]](#)

*Every real number lies in the  $\mathbb{R}$ -resolvent set of a non-real complex number:  $t - z$  is invertible for all  $t : \mathbb{R}$ .*

```
lemma mem_resolventSet_of_im_ne_zero (hz : z.im ≠ 0) (t : ℝ) : t ∈ resolventSet ℝ z
```

### lemma resolventSet\_eq\_univ

[\[source\]](#)

*The  $\mathbb{R}$ -resolvent set of a non-real complex number is all of  $\mathbb{R}$ .*

```
lemma resolventSet_eq_univ (hz : z.im ≠ 0) : resolventSet ℝ z = Set.univ
```

### lemma norm\_resolvent\_le

[\[source\]](#)

*The resolvent is globally bounded by  $|z.im|^{-1}$ : the imaginary part of the denominator  $t - z$  is exactly  $-z.im$ .*

```
lemma norm_resolvent_le (hz : z.im ≠ 0) (t : ℝ) : ||resolvent z t|| ≤ |z.im|-1
```

### lemma contDiff\_resolvent

[\[source\]](#)

*The resolvent of a non-real complex number is smooth along  $\mathbb{R}$ .*

@[fun\_prop]

```
lemma contDiff_resolvent (hz : z.im ≠ 0) : ContDiff ℝ ∞ (resolvent (R := ℝ) z)
```

### lemma iteratedDeriv\_resolvent

[\[source\]](#)

*Closed form for the iterated derivatives of the resolvent: the  $n$ -th derivative is  $(-1)^n \cdot n! \cdot (\text{resolvent } z)^{n+1}$ .*

```
lemma iteratedDeriv_resolvent (hz : z.im ≠ 0) (n : ℕ) :
  iteratedDeriv n (resolvent (R := ℝ) z)
  = fun t ↦ (-1) ^ n * (n ! : ℂ) * resolvent z t ^ (n + 1)
```

**lemma norm\_iteratedDeriv\_resolvent\_le**[\[source\]](#)

*Every iterated derivative of the resolvent is globally bounded, explicitly by  $n! \cdot (|z.\text{im}| ^ (n + 1))^{-1}$ .*

```
lemma norm_iteratedDeriv_resolvent_le (hz : z.im ≠ 0) (n : ℕ) (t : ℝ) :
  ||iteratedDeriv n (resolvent z) t|| ≤ n ! * (|z.im| ^ (n + 1))-1
```

**lemma hasTemperateGrowth\_resolvent**[\[source\]](#)

*The resolvent of a non-real complex number has temperate growth along  $\mathbb{R}$ .*

```
@[fun_prop]
lemma hasTemperateGrowth_resolvent (hz : z.im ≠ 0) :
  Function.HasTemperateGrowth (resolvent (R := ℝ) z)
```

**2.37 Physlib.Mathematics.SpecialFunctions.PhysHermite**

[Physlib/Mathematics/SpecialFunctions/PhysHermite.lean](#) on GitHub.

**def physHermite**[\[source\]](#)

*The physicist's Hermite polynomials are defined as polynomials over  $\mathbb{Z}$  in  $X$  recursively with  $\text{physHermite } 0 = 1$  and*

*$\text{physHermite } (n + 1) = 2 \cdot X * \text{physHermite } n - \text{derivative } (\text{physHermite } n)$ .*

*This polynomial will often be cast as a function  $\mathbb{R} \rightarrow \mathbb{R}$  by evaluating the polynomial at  $X$ .*

```
noncomputable def physHermite : ℕ → Polynomial ℤ
| 0 => 1
| n + 1 => 2 * X * physHermite n - derivative (physHermite n)
```

**lemma physHermite\_succ**[\[source\]](#)

*The defining recursion  $\text{physHermite } (n + 1) = (2x - d/dx) (\text{physHermite } n)$ .*

```
lemma physHermite_succ (n : ℕ) :
  physHermite (n + 1) = 2 * X * physHermite n - derivative (physHermite n)
```

**lemma physHermite\_eq\_iterate**[\[source\]](#)

*The Rodrigues formula  $\text{physHermite } n = (2x - d/dx)^n 1$ .*

```
lemma physHermite_eq_iterate (n : ℕ) :
  physHermite n = (fun p => 2 * X * p - derivative p)[n] 1
```

**lemma physHermite\_zero**[\[source\]](#)

```
@[simp]
lemma physHermite_zero : physHermite 0 = C 1
```

**lemma physHermite\_one**[\[source\]](#)

```
@[simp]
lemma physHermite_one : physHermite 1 = 2 * X
```

**lemma derivative\_physHermite\_succ**[\[source\]](#)

```
lemma derivative_physHermite_succ : (n : ℕ) →
  derivative (physHermite (n + 1)) = 2 * (n + 1) • physHermite n
| 0 => by simp [physHermite_one]
| n + 1 => by
  rw [physHermite_succ]
  simp only [derivative_physHermite_succ n, nsmul_eq_mul, Nat.cast_ofNat, Nat.cast_add,
    Nat.cast_one, derivative_sub, derivative_mul, derivative_ofNat, zero_mul, derivative_X,
    mul_one, zero_add, derivative_add, derivative_natCast, derivative_one, add_zero]
  simp only [physHermite_succ, nsmul_eq_mul]
  ring
```

**lemma derivative\_physHermite**[\[source\]](#)

```
lemma derivative_physHermite : (n : ℕ) →
  derivative (physHermite n) = 2 * n • physHermite (n - 1)
| 0 => by simp
| n + 1 => by simp [derivative_physHermite_succ]
```

**lemma physHermite\_succ'**[\[source\]](#)

```
lemma physHermite_succ' (n : ℕ) :
  physHermite (n + 1) = 2 • X * physHermite n - 2 * n • physHermite (n - 1)
```

**lemma coeff\_physHermite\_succ\_zero**[\[source\]](#)

```
lemma coeff_physHermite_succ_zero (n : ℕ) :
  coeff (physHermite (n + 1)) 0 = - coeff (physHermite n) 1
```

**lemma coeff\_physHermite\_succ\_succ**[\[source\]](#)

```
lemma coeff_physHermite_succ_succ (n k : ℕ) : coeff (physHermite (n + 1)) (k + 1) =
  2 * coeff (physHermite n) k - (k + 2) * coeff (physHermite n) (k + 2)
```

**lemma coeff\_physHermite\_of\_lt**[\[source\]](#)

```
lemma coeff_physHermite_of_lt {n k : ℕ} (hnk : n < k) : coeff (physHermite n) k = 0
```

**lemma coeff\_physHermite\_self**[\[source\]](#)

```
@[simp]
lemma coeff_physHermite_self (n : ℕ) : coeff (physHermite n) n = 2 ^ n
```

**lemma degree\_physHermite**[\[source\]](#)

```
@[simp]
lemma degree_physHermite (n : ℕ) : degree (physHermite n) = n
```

**lemma natDegree\_physHermite**

[\[source\]](#)

```
@[simp]
lemma natDegree_physHermite (n : ℕ) : (physHermite n).natDegree = n
```

**lemma physHermite\_leadingCoeff**

[\[source\]](#)

```
@[simp]
lemma physHermite_leadingCoeff (n : ℕ) : (physHermite n).leadingCoeff = 2 ^ n
```

**lemma physHermite\_ne\_zero**

[\[source\]](#)

```
@[simp]
lemma physHermite_ne_zero (n : ℕ) : physHermite n ≠ 0
```

**lemma iterate\_derivative\_physHermite\_of\_gt**

[\[source\]](#)

```
lemma iterate_derivative_physHermite_of_gt {n m : ℕ} (h : n < m) :
  derivative^[m] (physHermite n) = 0
```

**lemma iterate\_derivative\_physHermite\_self**

[\[source\]](#)

```
@[simp]
lemma iterate_derivative_physHermite_self (n : ℕ) :
  derivative^[n] (physHermite n) = C (n ! * 2 ^ n : ℤ)
```

**abbrev realEval**

[\[source\]](#)

*Cast an integer polynomial to a function  $\mathbb{R} \rightarrow \mathbb{R}$  by evaluation of the indeterminate.*

```
@[coe]
noncomputable abbrev realEval (p : Polynomial ℤ) : ℝ → ℝ
```

**instance CoeFun (Polynomial ℤ) (fun \_ ↦ ℝ → ℝ)**

[\[source\]](#)

```
noncomputable instance : CoeFun (Polynomial ℤ) (fun _ ↦ ℝ → ℝ)
```

**lemma physHermite\_zero\_coe**

[\[source\]](#)

```
lemma physHermite_zero_coe : (physHermite 0 : ℝ → ℝ) = fun _ ↦ 1
```

**lemma physHermite\_succ\_coe**

[\[source\]](#)

*The defining recursion for physHermite as functions  $\mathbb{R} \rightarrow \mathbb{R}$  (c.f. physHermite\_succ).*

```
lemma physHermite_succ_coe (n : ℕ) :
  (physHermite (n + 1) : ℝ → ℝ) =
    2 • (fun x => x) * (physHermite n : ℝ → ℝ) - deriv (physHermite n)
```

**lemma physHermite\_succ\_apply**[\[source\]](#)

```
lemma physHermite_succ_apply (n : ℕ) (x : ℝ) :
  physHermite (n + 1) x = 2 * x * physHermite n x - deriv (physHermite n) x
```

**lemma physHermite\_succ\_coe'**[\[source\]](#)

*The two-term recursion for physHermite as functions  $\mathbb{R} \rightarrow \mathbb{R}$  (c.f. physHermite\_succ').*

```
lemma physHermite_succ_coe' (n : ℕ) :
  (physHermite (n + 1) : ℝ → ℝ) =
    2 • (fun x => x) * (physHermite n : ℝ → ℝ) - (2 * n) • (physHermite (n - 1) : ℝ → ℝ)
```

**lemma physHermite\_succ\_apply'**[\[source\]](#)

```
lemma physHermite_succ_apply' (n : ℕ) (x : ℝ) :
  physHermite (n + 1) x = 2 * x * physHermite n x - 2 * n * physHermite (n - 1) x
```

**lemma deriv\_physHermite**[\[source\]](#)

```
lemma deriv_physHermite (n : ℕ) : deriv (physHermite n) = 2 * n * physHermite (n - 1)
```

**lemma iterate\_deriv\_physHermite\_eq\_iterate\_derivative**[\[source\]](#)

```
lemma iterate_deriv_physHermite_eq_iterate_derivative (n m : ℕ) :
  deriv^[m] (physHermite n) = derivative^[m] (physHermite n)
```

**lemma fderiv\_physHermite**[\[source\]](#)

```
lemma fderiv_physHermite (n : ℕ) (x : ℝ) :
  fderiv ℝ (physHermite n) x = (1 : ℝ → L[ℝ] ℝ).smulRight (deriv (physHermite n) x)
```

**lemma physHermite\_neg**[\[source\]](#)

*The physHermite polynomials are alternately even and odd.*

```
@[simp]
lemma physHermite_neg (n : ℕ) (x : ℝ) : physHermite n (-x) = (-1) ^ n * physHermite n x
```

**lemma physHermite\_even**[\[source\]](#)

```
lemma physHermite_even {n : ℕ} (hn : Even n) : Function.Even (physHermite n)
```

**lemma physHermite\_odd**[\[source\]](#)

```
lemma physHermite_odd {n : ℕ} (hn : Odd n) : Function.Odd (physHermite n)
```

**lemma physHermite\_differentiable**[\[source\]](#)

```
@[fun_prop]
lemma physHermite_differentiable (n : ℕ) : Differentiable ℝ (physHermite n)
```

**lemma deriv\_physHermite\_differentiable**[\[source\]](#)

@[fun\_prop]

**lemma** deriv\_physHermite\_differentiable (n m :  $\mathbb{N}$ ) : Differentiable  $\mathbb{R}$  (deriv<sup>[m]</sup> (physHermite n))**lemma physHermite\_continuous**[\[source\]](#)

@[fun\_prop]

**lemma** physHermite\_continuous (n :  $\mathbb{N}$ ) : Continuous (physHermite n)**lemma physHermite\_contDiff**[\[source\]](#)

@[fun\_prop]

**lemma** physHermite\_contDiff (n :  $\mathbb{N}$ ) (m : WithTop  $\mathbb{N}_\infty$ ) : ContDiff  $\mathbb{R}$  m (physHermite n)**lemma physHermite\_hasTemperateGrowth**[\[source\]](#)

@[fun\_prop]

**lemma** physHermite\_hasTemperateGrowth (n :  $\mathbb{N}$ ) : HasTemperateGrowth (physHermite n)**lemma deriv\_gaussian\_eq\_physHermite\_mul\_gaussian**[\[source\]](#)**lemma** deriv\_gaussian\_eq\_physHermite\_mul\_gaussian (n :  $\mathbb{N}$ ) (x :  $\mathbb{R}$ ) :  
deriv<sup>[n]</sup> (**fun** y => Real.exp (- y ^ 2)) x =  
(-1 :  $\mathbb{R}$ ) ^ n \* physHermite n x \* Real.exp (- x ^ 2)**lemma physHermite\_eq\_deriv\_gaussian**[\[source\]](#)**lemma** physHermite\_eq\_deriv\_gaussian (n :  $\mathbb{N}$ ) (x :  $\mathbb{R}$ ) :  
physHermite n x = (-1 :  $\mathbb{R}$ ) ^ n \* deriv<sup>[n]</sup>  
(**fun** y => Real.exp (- y ^ 2)) x / Real.exp (- x ^ 2)**lemma physHermite\_eq\_deriv\_gaussian'**[\[source\]](#)**lemma** physHermite\_eq\_deriv\_gaussian' (n :  $\mathbb{N}$ ) (x :  $\mathbb{R}$ ) :  
physHermite n x = (-1 :  $\mathbb{R}$ ) ^ n \* deriv<sup>[n]</sup> (**fun** y => Real.exp (- y ^ 2)) x \*  
Real.exp (x ^ 2)**lemma gaussian\_integrable\_polynomial\_cons**[\[source\]](#)

@[fun\_prop]

**lemma** gaussian\_integrable\_polynomial\_cons {b c :  $\mathbb{R}$ } (hb : 0 < b) (P : Polynomial  $\mathbb{Z}$ ) :  
MeasureTheory.Integrable **fun** x :  $\mathbb{R}$  => (P.aeval (c \* x)) \* Real.exp (-b \* x ^ 2)**lemma gaussian\_integrable\_polynomial**[\[source\]](#)

@[fun\_prop]

**lemma** gaussian\_integrable\_polynomial {b :  $\mathbb{R}$ } (hb : 0 < b) (P : Polynomial  $\mathbb{Z}$ ) :  
MeasureTheory.Integrable **fun** x :  $\mathbb{R}$  => (P.aeval x) \* Real.exp (-b \* x ^ 2)

**lemma physHermite\_gaussian\_integrable**[\[source\]](#)

```
@[fun_prop]
lemma physHermite_gaussian_integrable (n p m : ℕ) :
  MeasureTheory.Integrable (deriv^[m] (physHermite p) * deriv^[n] fun x => Real.exp (-x ^ 2))
  MeasureTheory.volume
```

**lemma integral\_physHermite\_mul\_physHermite\_eq\_integral\_deriv\_exp**[\[source\]](#)

```
lemma integral_physHermite_mul_physHermite_eq_integral_deriv_exp (n m : ℕ) :
  ∫ x : ℝ, (physHermite n x * physHermite m x) * Real.exp (-x ^ 2) =
  (-1 : ℝ) ^ m * ∫ x : ℝ, (physHermite n x * (deriv^[m] fun x => Real.exp (-x ^ 2)) x)
```

**lemma integral\_physHermite\_mul\_physHermite\_eq\_integral\_deriv\_inductive**[\[source\]](#)

```
lemma integral_physHermite_mul_physHermite_eq_integral_deriv_inductive (n m : ℕ) :
  (p : ℕ) → (hpm : p ≤ m) →
  ∫ x : ℝ, (physHermite n x * physHermite m x) * Real.exp (- x ^ 2) =
  (-1 : ℝ) ^ (m - p) * ∫ x : ℝ, (deriv^[p] (physHermite n) x *
  (deriv^[m - p] fun x => Real.exp (-x ^ 2)) x)
| 0, h => integral_physHermite_mul_physHermite_eq_integral_deriv_exp n m
| p + 1, h => by
  rw [integral_physHermite_mul_physHermite_eq_integral_deriv_inductive n m p (by omega),
    show m - p = m - (p + 1) + 1 by omega, Function.iterate_succ_apply', pow_succ, mul_assoc,
    Function.iterate_succ_apply']
  congr
  have h1 : ∫ (x : ℝ), deriv^[p] (physHermite n) x *
    deriv (deriv^[m - (p + 1)] fun x => Real.exp (-x ^ 2)) x =
    - ∫ (x : ℝ), deriv (deriv^[p] (physHermite n)) x *
    deriv^[m - (p + 1)] (fun x => Real.exp (-x ^ 2)) x
```

**lemma integral\_physHermite\_mul\_physHermite\_eq\_integral\_deriv**[\[source\]](#)

```
lemma integral_physHermite_mul_physHermite_eq_integral_deriv (n m : ℕ) :
  ∫ x : ℝ, (physHermite n x * physHermite m x) * Real.exp (- x ^ 2) =
  ∫ x : ℝ, (deriv^[m] (physHermite n) x * (Real.exp (-x ^ 2)))
```

**lemma physHermite\_orthogonal\_lt**[\[source\]](#)

```
lemma physHermite_orthogonal_lt {n m : ℕ} (hnm : n < m) :
  ∫ x : ℝ, (physHermite n x * physHermite m x) * Real.exp (- x ^ 2) = 0
```

**theorem physHermite\_orthogonal**[\[source\]](#)

```
theorem physHermite_orthogonal {n m : ℕ} (hnm : n ≠ m) :
  ∫ x : ℝ, (physHermite n x * physHermite m x) * Real.exp (- x ^ 2) = 0
```

**lemma physHermite\_orthogonal\_cons**[\[source\]](#)

```
lemma physHermite_orthogonal_cons {n m : ℕ} (hnm : n ≠ m) (c : ℝ) :
  ∫ x : ℝ, (physHermite n (c * x) * physHermite m (c * x)) *
  Real.exp (- c ^ 2 * x ^ 2) = 0
```



**theorem physHermite\_norm**[\[source\]](#)

```

theorem physHermite_norm (n : ℕ) :
  ∫ x : ℝ, (physHermite n x * physHermite n x) * Real.exp (- x ^ 2) =
  ↑n ! * 2 ^ n * √Real.pi

```

**lemma physHermite\_norm\_cons**[\[source\]](#)

```

lemma physHermite_norm_cons (n : ℕ) (c : ℝ) :
  ∫ x : ℝ, (physHermite n (c * x) * physHermite n (c * x)) * Real.exp (- c ^ 2 * x ^ 2) =
  |c-1| * (↑n ! * 2 ^ n * √Real.pi)

```

**lemma polynomial\_mem\_physHermite\_span\_induction**[\[source\]](#)

```

lemma polynomial_mem_physHermite_span_induction (P : Polynomial ℤ) : (n : ℕ) →
  (hn : P.natDegree = n) →
  (P : ℝ → ℝ) ∈ Submodule.span ℝ (Set.range (fun n => (physHermite n : ℝ → ℝ)))
| 0, h => by
  obtain (x, rfl)

```

**lemma polynomial\_mem\_physHermite\_span**[\[source\]](#)

```

lemma polynomial_mem_physHermite_span (P : Polynomial ℤ) :
  (P : ℝ → ℝ) ∈ Submodule.span ℝ (Set.range (fun n => (physHermite n : ℝ → ℝ)))

```

**lemma cos\_mem\_physHermite\_span\_topologicalClosure**[\[source\]](#)

```

lemma cos_mem_physHermite_span_topologicalClosure (c : ℝ) :
  (fun (x : ℝ) => Real.cos (c * x)) ∈
  (Submodule.span ℝ (Set.range (fun n => (physHermite n : ℝ → ℝ)))).topologicalClosure

```

**2.38 Physlib.Particles.StandardModel.Basic**

[Physlib/Particles/StandardModel/Basic.lean](#) on GitHub.

**lemma gaugeGroupℤ₆SU30fRoot\_eq\_mul\_id**[\[source\]](#)

```

lemma gaugeGroupℤ₆SU30fRoot_eq_mul_id (α : rootsOfUnity 6 ℂ) :
  (gaugeGroupℤ₆SU30fRoot α).1 = ((α : ℂ×) : ℂ) ^ 2 • 1

```

**lemma gaugeGroupℤ₆SU30fRoot\_toEuclideanLin\_apply**[\[source\]](#)

```

lemma gaugeGroupℤ₆SU30fRoot_toEuclideanLin_apply (α : rootsOfUnity 6 ℂ)
  (v : EuclideanSpace ℂ (Fin 3)) :
  (gaugeGroupℤ₆SU30fRoot α).1.toEuclideanLin v = ((α : ℂ×) : ℂ) ^ 2 • v

```

**lemma gaugeGroupℤ₆SU20fRoot\_eq\_mul\_id**[\[source\]](#)

```

lemma gaugeGroupℤ₆SU20fRoot_eq_mul_id (α : rootsOfUnity 6 ℂ) :
  (gaugeGroupℤ₆SU20fRoot α).1 = star ((α : ℂ×) : ℂ) ^ 3 • 1

```

**lemma gaugeGroupZ6SU20fRoot\_toEuclideanLin\_apply**[\[source\]](#)

```
lemma gaugeGroupZ6SU20fRoot_toEuclideanLin_apply (α : rootsOfUnity 6 ℂ)
  (v : EuclideanSpace ℂ (Fin 2)) :
  (gaugeGroupZ6SU20fRoot α).1.toEuclideanLin v = star ((α : ℂx) : ℂ) ^ 3 • v
```

**lemma subgroup\_le\_subgroup\_Z6**[\[source\]](#)

```
lemma subgroup_le_subgroup_Z6 (q : GaugeGroupQuot) : subgroup q ≤ gaugeGroupZ6SubGroup
```

**2.39 Physlib.Particles.StandardModel.Fermions.DownSinglet**

[Physlib/Particles/StandardModel/Fermions/DownSinglet.lean](#) on GitHub.

**structure DownSinglet**[\[source\]](#)

*The target vector space of one Standard Model down-type singlet quark. It carries the  $(3, 1)_{-2}$  representation of the gauge group.*

```
@[ext]
structure DownSinglet where
  val : Fermion.RightHandedWeyl ⊗[ℂ] EuclideanSpace ℂ (Fin 3)
```

**def valEquiv**[\[source\]](#)

*Identifies a down-type singlet with its underlying tensor-product value.*

```
def valEquiv : DownSinglet ≈ Fermion.RightHandedWeyl ⊗[ℂ] EuclideanSpace ℂ (Fin 3) where
```

**instance AddCommGroup DownSinglet**[\[source\]](#)

```
instance : AddCommGroup DownSinglet
```

**instance Module ℂ DownSinglet**[\[source\]](#)

```
instance : Module ℂ DownSinglet
```

**def valLinEquiv**[\[source\]](#)

*The linear identification with the underlying tensor product.*

```
def valLinEquiv : DownSinglet ≈ℓ[ℂ]
  Fermion.RightHandedWeyl ⊗[ℂ] EuclideanSpace ℂ (Fin 3) where
```

**lemma valLinEquiv\_apply**[\[source\]](#)

```
@[simp]
lemma valLinEquiv_apply (d : DownSinglet) : valLinEquiv d = d.val
```

**lemma valLinEquiv\_symm\_apply**[\[source\]](#)

```

lemma valLinEquiv_symm_apply
  (m : Fermion.RightHandedWeyl ⊗[ℂ] EuclideanSpace ℂ (Fin 3)) :
  valLinEquiv.symm m = ⟨m⟩

```

**lemma val\_add**[\[source\]](#)

```

@[simp]
lemma val_add (d1 d2 : DownSinglet) : (d1 + d2).val = d1.val + d2.val

```

**lemma val\_smul**[\[source\]](#)

```

@[simp]
lemma val_smul (r : ℂ) (d : DownSinglet) : (r • d).val = r • d.val

```

**def repLorentzGroup**[\[source\]](#)

*The right-handed Lorentz representation on down-type singlet quarks.*

```

noncomputable def repLorentzGroup : Representation ℂ (SL(2,ℂ)) DownSinglet where

```

**def repGaugeGroupI**[\[source\]](#)

*The  $(3, 1)_{-2}$  action of the unquotiented Standard Model gauge group.*

```

noncomputable def repGaugeGroupI : Representation ℂ GaugeGroupI DownSinglet where

```

**lemma repGaugeGroupI\_tmul**[\[source\]](#)

*The gauge action on a pure spinor-colour tensor.*

```

lemma repGaugeGroupI_tmul (g : GaugeGroupI) (ψ : Fermion.RightHandedWeyl)
  (v : EuclideanSpace ℂ (Fin 3)) :
  repGaugeGroupI g (ψ ⊗ℂ v) =
  ((star g.toU1.1 ^ 2) • ψ ⊗ℂ g.toSU3.1.toEuclideanLin v)

```

**lemma repGaugeGroupI\_tmul\_basis\_eq\_sum**[\[source\]](#)

*Expands the gauge action in the spinor-colour basis.*

```

lemma repGaugeGroupI_tmul_basis_eq_sum (g : GaugeGroupI) (k : Fin 2) (i : Fin 3) :
  repGaugeGroupI g
    (RightHandedWeyl.basis k ⊗ℂ EuclideanSpace.basisFun (Fin 3) ℂ i) =
  ∑ i' : Fin 3, (star g.toU1.1 ^ 2 * g.toSU3.1 i' i) •
    ((RightHandedWeyl.basis k ⊗ℂ
      EuclideanSpace.basisFun (Fin 3) ℂ i') : DownSinglet)

```

**lemma repGaugeGroupI\_eq\_iff\_mul\_eq**[\[source\]](#)

*Two gauge elements induce the same action exactly when their hypercharge-colour coefficients agree.*

```

lemma repGaugeGroupI_eq_iff_mul_eq {g1 g2 : GaugeGroupI} :
  repGaugeGroupI g1 = repGaugeGroupI g2 ↔ ∀ i i',
    star g1.toU1.1 ^ 2 * g1.toSU3.1 i' i =
      star g2.toU1.1 ^ 2 * g2.toSU3.1 i' i

```

**lemma mem\_repGaugeGroupI\_ker\_iff\_eq**

[\[source\]](#)

*Characterizes the full-group elements acting trivially on the down-type singlet.*

```

lemma mem_repGaugeGroupI_ker_iff_eq {g : GaugeGroupI} :
  g ∈ repGaugeGroupI.ker ↔ ∃ a : ℂ,
    g.toSU3.1 = a • 1 ∧ a * star g.toU1.1 ^ 2 = 1

```

**lemma gaugeGroup\_subgroup\_ℤ<sub>6</sub>\_le\_ker\_repGaugeGroupI**

[\[source\]](#)

*The central ℤ<sub>6</sub> subgroup acts trivially on (3, 1)<sub>-2</sub>.*

```

lemma gaugeGroup_subgroup_ℤ6_le_ker_repGaugeGroupI :
  GaugeGroupQuot.subgroup .ℤ6 ≤ repGaugeGroupI.ker

```

**lemma gaugeGroup\_subgroup\_le\_ker\_repGaugeGroupI**

[\[source\]](#)

*Every supported quotient subgroup acts trivially on the down-type singlet.*

```

lemma gaugeGroup_subgroup_le_ker_repGaugeGroupI (Q : GaugeGroupQuot) :
  Q.subgroup ≤ repGaugeGroupI.ker

```

**def repGaugeGroup**

[\[source\]](#)

*The (3, 1)<sub>-2</sub> representation for every supported global form of the Standard Model gauge group.*

```

noncomputable def repGaugeGroup : (Q : GaugeGroupQuot) →
  Representation ℂ (GaugeGroup Q) DownSinglet
| .I => repGaugeGroupI
| .ℤ6 => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .ℤ6)
| .ℤ2 => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .ℤ2)
| .ℤ3 => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .ℤ3)

```

## 2.40 Physlib.Particles.StandardModel.Fermions.LeptonDoublet

[Physlib/Particles/StandardModel/Fermions/LeptonDoublet.lean](#) on GitHub.

**structure LeptonDoublet**

[\[source\]](#)

*The target vector space of one Standard Model lepton doublet. It carries the (1, 2)<sub>-3</sub> representation of the gauge group.*

@[ext]

**structure** LeptonDoublet **where**

```

  val : Fermion.LeftHandedWeyl ⊗[ℂ] EuclideanSpace ℂ (Fin 2)

```

**def valEquiv** [\[source\]](#)

*Identifies a lepton doublet with its underlying tensor-product value.*

```
def valEquiv : LeptonDoublet ≈ Fermion.LeftHandedWeyl ⊗[ℂ] EuclideanSpace ℂ (Fin 2) where
```

**instance AddCommGroup LeptonDoublet** [\[source\]](#)

```
instance : AddCommGroup LeptonDoublet
```

**instance Module ℂ LeptonDoublet** [\[source\]](#)

```
instance : Module ℂ LeptonDoublet
```

**def valLinEquiv** [\[source\]](#)

*The linear identification with the underlying tensor product.*

```
def valLinEquiv : LeptonDoublet ≈l[ℂ]
  Fermion.LeftHandedWeyl ⊗[ℂ] EuclideanSpace ℂ (Fin 2) where
```

**lemma valLinEquiv\_apply** [\[source\]](#)

```
@[simp]
lemma valLinEquiv_apply (l : LeptonDoublet) : valLinEquiv l = l.val
```

**lemma valLinEquiv\_symm\_apply** [\[source\]](#)

```
lemma valLinEquiv_symm_apply
  (m : Fermion.LeftHandedWeyl ⊗[ℂ] EuclideanSpace ℂ (Fin 2)) :
  valLinEquiv.symm m = ⟨m⟩
```

**lemma val\_add** [\[source\]](#)

```
@[simp]
lemma val_add (l₁ l₂ : LeptonDoublet) : (l₁ + l₂).val = l₁.val + l₂.val
```

**lemma val\_smul** [\[source\]](#)

```
@[simp]
lemma val_smul (r : ℂ) (l : LeptonDoublet) : (r • l).val = r • l.val
```

**def repLorentzGroup** [\[source\]](#)

*The left-handed Lorentz representation on lepton doublets.*

```
noncomputable def repLorentzGroup : Representation ℂ (SL(2,ℂ)) LeptonDoublet where
```

**def repGaugeGroupI** [\[source\]](#)

*The  $(1, 2)_{-3}$  action of the unquotiented Standard Model gauge group.*

```
noncomputable def repGaugeGroupI : Representation ℂ GaugeGroupI LeptonDoublet where
```

**lemma repGaugeGroupI\_tmul**[\[source\]](#)

*The gauge action on a pure spinor–weak tensor.*

```
lemma repGaugeGroupI_tmul (g : GaugeGroupI) (v : Fermion.LeftHandedWeyl)
  (w : EuclideanSpace ℂ (Fin 2)) :
  repGaugeGroupI g (v ⊗ₜ w) =
    ((star g.toU1.1 ^ 3) • v ⊗ₜ (g.toSU2.1.toEuclideanLin w))
```

**lemma repGaugeGroupI\_tmul\_basis\_eq\_sum**[\[source\]](#)

*Expands the gauge action in the spinor–weak basis.*

```
lemma repGaugeGroupI_tmul_basis_eq_sum (g : GaugeGroupI) (k j : Fin 2) :
  repGaugeGroupI g (LeftHandedWeyl.basis k ⊗ₜ [C]
    EuclideanSpace.basisFun (Fin 2) ℂ j) =
    ∑ j' : Fin 2, (star g.toU1.1 ^ 3 * g.toSU2.1 j' j)
    • ((LeftHandedWeyl.basis k ⊗ₜ [C]
      EuclideanSpace.basisFun (Fin 2) ℂ j') : LeptonDoublet)
```

**lemma repGaugeGroupI\_eq\_iff\_mul\_eq**[\[source\]](#)

*Two gauge elements induce the same action exactly when their weak-basis coefficients agree.*

```
lemma repGaugeGroupI_eq_iff_mul_eq {g₁ g₂ : GaugeGroupI} :
  repGaugeGroupI g₁ = repGaugeGroupI g₂ ↔ ∀ j j',
  star g₁.toU1.1 ^ 3 * g₁.toSU2.1 j' j =
  star g₂.toU1.1 ^ 3 * g₂.toSU2.1 j' j
```

**lemma mem\_repGaugeGroupI\_ker\_iff\_eq**[\[source\]](#)

*Characterizes the full-group elements acting trivially on the lepton doublet.*

```
lemma mem_repGaugeGroupI_ker_iff_eq {g : GaugeGroupI} :
  g ∈ repGaugeGroupI.ker ↔ ∃ a : ℂ, g.toSU2.1 = a • 1 ∧
  a * star g.toU1.1 ^ 3 = 1
```

**lemma gaugeGroup\_subgroup\_ℤ₆\_le\_ker\_repGaugeGroupI**[\[source\]](#)

*The central  $\mathbb{Z}_6$  subgroup acts trivially on  $(1, 2)_{-3}$ .*

```
lemma gaugeGroup_subgroup_ℤ₆_le_ker_repGaugeGroupI :
  GaugeGroupQuot.subgroup .ℤ₆ ≤ repGaugeGroupI.ker
```

**lemma gaugeGroup\_subgroup\_le\_ker\_repGaugeGroupI**[\[source\]](#)

*Every supported quotient subgroup acts trivially on the lepton doublet.*

```
lemma gaugeGroup_subgroup_le_ker_repGaugeGroupI (Q : GaugeGroupQuot) :
  Q.subgroup ≤ repGaugeGroupI.ker
```

**def repGaugeGroup**[\[source\]](#)

*The  $(1, 2)_{-3}$  representation for every supported global form of the Standard Model gauge group.*

```
noncomputable def repGaugeGroup : (Q : GaugeGroupQuot) →
  Representation ℂ (GaugeGroup Q) LeptonDoublet
| .I => repGaugeGroupI
| .Z6 => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .Z6)
| .Z2 => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .Z2)
| .Z3 => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .Z3)
```

## 2.41 Physlib.Particles.StandardModel.Fermions.LeptonSinglet

[Physlib/Particles/StandardModel/Fermions/LeptonSinglet.lean](#) on GitHub.

### structure LeptonSinglet

[\[source\]](#)

*The target vector space of one Standard Model charged-lepton singlet. It carries the  $(1, 1)_{-6}$  representation of the gauge group.*

```
@[ext]
structure LeptonSinglet where
  val : Fermion.RightHandedWeyl
```

### def valEquiv

[\[source\]](#)

*Identifies a charged-lepton singlet with its underlying Weyl spinor.*

```
def valEquiv : LeptonSinglet ≈ Fermion.RightHandedWeyl where
```

### instance AddCommGroup LeptonSinglet

[\[source\]](#)

```
instance : AddCommGroup LeptonSinglet
```

### instance Module ℂ LeptonSinglet

[\[source\]](#)

```
instance : Module ℂ LeptonSinglet
```

### def valLinEquiv

[\[source\]](#)

*The linear identification with the underlying Weyl-spinor space.*

```
def valLinEquiv : LeptonSinglet ≈ℂ Fermion.RightHandedWeyl where
```

### lemma valLinEquiv\_apply

[\[source\]](#)

```
@[simp]
lemma valLinEquiv_apply (l : LeptonSinglet) : valLinEquiv l = l.val
```

### lemma valLinEquiv\_symm\_apply

[\[source\]](#)

```
lemma valLinEquiv_symm_apply (m : Fermion.RightHandedWeyl) :
  valLinEquiv.symm m = ⟨m⟩
```

**lemma val\_add**[\[source\]](#)

```
@[simp]
lemma val_add (l1 l2 : LeptonSinglet) : (l1 + l2).val = l1.val + l2.val
```

**lemma val\_smul**[\[source\]](#)

```
@[simp]
lemma val_smul (r : ℂ) (l : LeptonSinglet) : (r • l).val = r • l.val
```

**def repLorentzGroup**[\[source\]](#)

*The right-handed Lorentz representation on charged-lepton singlets.*

```
noncomputable def repLorentzGroup : Representation ℂ (SL(2,ℂ)) LeptonSinglet where
```

**def repGaugeGroupI**[\[source\]](#)

*The  $(1, 1)_{-6}$  action of the unquotiented Standard Model gauge group.*

```
noncomputable def repGaugeGroupI : Representation ℂ GaugeGroupI LeptonSinglet where
```

**lemma repGaugeGroupI\_apply**[\[source\]](#)

*The gauge group acts on a charged-lepton singlet by the hypercharge scalar alone.*

```
lemma repGaugeGroupI_apply (g : GaugeGroupI) (ψ : Fermion.RightHandedWeyl) :
  repGaugeGroupI g (ψ) = (star g.toU1.1 ^ 6) • ψ
```

**lemma repGaugeGroupI\_basis**[\[source\]](#)

*The gauge action is diagonal in the standard Weyl basis.*

```
lemma repGaugeGroupI_basis (g : GaugeGroupI) (k : Fin 2) :
  repGaugeGroupI g (RightHandedWeyl.basis k) =
    (star g.toU1.1 ^ 6) • (RightHandedWeyl.basis k) : LeptonSinglet
```

**lemma repGaugeGroupI\_eq\_iff**[\[source\]](#)

*Two gauge elements induce the same action exactly when their hypercharge scalars agree.*

```
lemma repGaugeGroupI_eq_iff {g1 g2 : GaugeGroupI} :
  repGaugeGroupI g1 = repGaugeGroupI g2 ↔
  star g1.toU1.1 ^ 6 = star g2.toU1.1 ^ 6
```

**lemma mem\_repGaugeGroupI\_ker\_iff\_eq**[\[source\]](#)

*Characterizes the full-group elements acting trivially on the charged-lepton singlet.*

```
lemma mem_repGaugeGroupI_ker_iff_eq {g : GaugeGroupI} :
  g ∈ repGaugeGroupI.ker ↔ star g.toU1.1 ^ 6 = 1
```



**lemma gaugeGroup\_subgroup\_ℤ₆\_le\_ker\_repGaugeGroupI**[\[source\]](#)

*The central  $\mathbb{Z}_6$  subgroup acts trivially on  $(1, 1)_{-6}$ .*

```
lemma gaugeGroup_subgroup_ℤ₆_le_ker_repGaugeGroupI :
  GaugeGroupQuot.subgroup .ℤ₆ ≤ repGaugeGroupI.ker
```

**lemma gaugeGroup\_subgroup\_le\_ker\_repGaugeGroupI**[\[source\]](#)

*Every supported quotient subgroup acts trivially on the charged-lepton singlet.*

```
lemma gaugeGroup_subgroup_le_ker_repGaugeGroupI (Q : GaugeGroupQuot) :
  Q.subgroup ≤ repGaugeGroupI.ker
```

**def repGaugeGroup**[\[source\]](#)

*The  $(1, 1)_{-6}$  representation for every supported global form of the Standard Model gauge group.*

```
noncomputable def repGaugeGroup : (Q : GaugeGroupQuot) →
  Representation ℂ (GaugeGroup Q) LeptonSinglet
| .I => repGaugeGroupI
| .ℤ₆ => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .ℤ₆)
| .ℤ₂ => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .ℤ₂)
| .ℤ₃ => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .ℤ₃)
```

**2.42 Physlib.Particles.StandardModel.Fermions.QuarkDoublet**

[Physlib/Particles/StandardModel/Fermions/QuarkDoublet.lean](#) on GitHub.

**structure QuarkDoublet**[\[source\]](#)

*The vector space of a quark field in the Standard Model. These live in the  $(3, 2)_{-1}$  representation of the gauge group.*

```
@[ext]
structure QuarkDoublet where
  val : Fermion.LeftHandedWeyl ⊗[ℂ] EuclideanSpace ℂ (Fin 3) ⊗[ℂ] EuclideanSpace ℂ (Fin 2)
```

**def valEquiv**[\[source\]](#)

*The linear equivalence between QuarkDoublet and its underlying tensor product space.*

```
def valEquiv : QuarkDoublet ≈
  Fermion.LeftHandedWeyl ⊗[ℂ] EuclideanSpace ℂ (Fin 3) ⊗[ℂ] EuclideanSpace ℂ (Fin 2) where
```

**instance AddCommGroup QuarkDoublet**[\[source\]](#)

```
instance : AddCommGroup QuarkDoublet
```

**instance Module ℂ QuarkDoublet**[\[source\]](#)

```
instance : Module ℂ QuarkDoublet
```

**def valLinEquiv**[\[source\]](#)

*The linear equivalence between QuarkDoublet and its underlying tensor product space.*

```
def valLinEquiv : QuarkDoublet ≈ℓ [ℂ]
  Fermion.LeftHandedWeyl ⊗ [ℂ] EuclideanSpace ℂ (Fin 3) ⊗ [ℂ] EuclideanSpace ℂ (Fin 2) where
```

**lemma valLinEquiv\_apply**[\[source\]](#)

```
@[simp]
lemma valLinEquiv_apply (q : QuarkDoublet) : valLinEquiv q = q.val
```

**lemma valLinEquiv\_symm\_apply**[\[source\]](#)

```
lemma valLinEquiv_symm_apply
  (m : Fermion.LeftHandedWeyl ⊗ [ℂ] EuclideanSpace ℂ (Fin 3) ⊗ [ℂ] EuclideanSpace ℂ (Fin 2)) :
  valLinEquiv.symm m = ⟨m⟩
```

**lemma val\_add**[\[source\]](#)

```
@[simp]
lemma val_add (q1 q2 : QuarkDoublet) : (q1 + q2).val = q1.val + q2.val
```

**lemma val\_smul**[\[source\]](#)

```
@[simp]
lemma val_smul (r : ℂ) (q : QuarkDoublet) : (r • q).val = r • q.val
```

**def repLorentzGroup**[\[source\]](#)

*The representation of the Lorentz group on the space of quark fields.*

```
noncomputable def repLorentzGroup : Representation ℂ (SL(2,ℂ)) QuarkDoublet where
```

**def repGaugeGroupI**[\[source\]](#)

*The action of the full Standard Model gauge group on quark fields.*

```
noncomputable def repGaugeGroupI : Representation ℂ GaugeGroupI QuarkDoublet where
```

**lemma repGaugeGroupI\_tmul**[\[source\]](#)

```
lemma repGaugeGroupI_tmul (g : GaugeGroupI) (ψ : Fermion.LeftHandedWeyl)
  (v : EuclideanSpace ℂ (Fin 3)) (w : EuclideanSpace ℂ (Fin 2)) :
  repGaugeGroupI g (ψ ⊗ℓ v ⊗ℓ w) = ⟨g.toU1 • ψ ⊗ℓ (g.toSU3.1.toEuclideanLin v) ⊗ℓ
    (g.toSU2.1.toEuclideanLin w)⟩
```

**lemma repGaugeGroupI\_tmul\_basis\_eq\_sum**[\[source\]](#)

*The action of the full gauge group on a tensor product of basis elements, expanded as a sum over the columns of the SU(3) and SU(2) matrices.*

```

lemma repGaugeGroupI_tmul_basis_eq_sum (g : GaugeGroupI) (k : Fin 2) (i : Fin 3) (j : Fin 2) :
  repGaugeGroupI g (LeftHandedWeyl.basis k ⊗ₜ[ℂ] EuclideanSpace.basisFun (Fin 3) ℂ i
    ⊗ₜ[ℂ] EuclideanSpace.basisFun (Fin 2) ℂ j) =
    ∑ i' : Fin 3, ∑ j' : Fin 2, (g.toU1.1 * g.toSU3.1 i' i * g.toSU2.1 j' j)
    • ((LeftHandedWeyl.basis k ⊗ₜ[ℂ] EuclideanSpace.basisFun (Fin 3) ℂ i'
      ⊗ₜ[ℂ] EuclideanSpace.basisFun (Fin 2) ℂ j') : QuarkDoublet)

```

**lemma repGaugeGroupI\_eq\_iff\_mul\_eq**

[\[source\]](#)

```

lemma repGaugeGroupI_eq_iff_mul_eq {g1 g2 : GaugeGroupI} :
  repGaugeGroupI g1 = repGaugeGroupI g2 ↔ ∀ i i' j j',
  g1.toU1.1 * g1.toSU3.1 i' i * g1.toSU2.1 j' j =
  g2.toU1.1 * g2.toSU3.1 i' i * g2.toSU2.1 j' j

```

**lemma mem\_repGaugeGroupI\_ker\_iff\_eq**

[\[source\]](#)

```

lemma mem_repGaugeGroupI_ker_iff_eq {g : GaugeGroupI} :
  g ∈ repGaugeGroupI.ker ↔ ∃ a b : ℂ, g.toSU2.1 = a • 1 ∧ g.toSU3.1 = b • 1 ∧
  a * b * g.toU1.1 = 1

```

**lemma gaugeGroup\_subgroup\_ℤ₆\_le\_ker\_repGaugeGroupI**

[\[source\]](#)

```

lemma gaugeGroup_subgroup_ℤ₆_le_ker_repGaugeGroupI :
  GaugeGroupQuot.subgroup .ℤ₆ ≤ repGaugeGroupI.ker

```

**lemma gaugeGroup\_subgroup\_le\_ker\_repGaugeGroupI**

[\[source\]](#)

```

lemma gaugeGroup_subgroup_le_ker_repGaugeGroupI (Q : GaugeGroupQuot) :
  Q.subgroup ≤ repGaugeGroupI.ker

```

**def repGaugeGroup**

[\[source\]](#)

*The action of the Standard Model gauge group, potentially quotiented by a discrete factor on quark fields.*

```

noncomputable def repGaugeGroup : (Q : GaugeGroupQuot) →
  Representation ℂ (GaugeGroup Q) QuarkDoublet
| .I => repGaugeGroupI
| .ℤ₆ => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .ℤ₆)
| .ℤ₂ => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .ℤ₂)
| .ℤ₃ => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .ℤ₃)

```

## 2.43 Physlib.Particles.StandardModel.Fermions.UpSinglet

[Physlib/Particles/StandardModel/Fermions/UpSinglet.lean](#) on GitHub.

**structure UpSinglet**

[\[source\]](#)

*The vector space of an up-type singlet quark field in the Standard Model. These live in the  $(3, 1)_{\{4\}}$  representation of the gauge group.*

```

@[ext]
structure UpSinglet where

```

```
val : Fermion.RightHandedWeyl ⊗[ℂ] EuclideanSpace ℂ (Fin 3)
```

**def valEquiv**

[\[source\]](#)

*The linear equivalence between UpSinglet and its underlying tensor product space.*

```
def valEquiv : UpSinglet ≈ Fermion.RightHandedWeyl ⊗[ℂ] EuclideanSpace ℂ (Fin 3) where
```

**instance AddCommGroup UpSinglet**

[\[source\]](#)

```
instance : AddCommGroup UpSinglet
```

**instance Module ℂ UpSinglet**

[\[source\]](#)

```
instance : Module ℂ UpSinglet
```

**def valLinEquiv**

[\[source\]](#)

*The linear equivalence between UpSinglet and its underlying tensor product space.*

```
def valLinEquiv : UpSinglet ≈ℓ[ℂ]
  Fermion.RightHandedWeyl ⊗[ℂ] EuclideanSpace ℂ (Fin 3) where
```

**lemma valLinEquiv\_apply**

[\[source\]](#)

```
@[simp]
lemma valLinEquiv_apply (q : UpSinglet) : valLinEquiv q = q.val
```

**lemma valLinEquiv\_symm\_apply**

[\[source\]](#)

```
lemma valLinEquiv_symm_apply (m : Fermion.RightHandedWeyl ⊗[ℂ] EuclideanSpace ℂ (Fin 3)) :
  valLinEquiv.symm m = ⟨m⟩
```

**lemma val\_add**

[\[source\]](#)

```
@[simp]
lemma val_add (q1 q2 : UpSinglet) : (q1 + q2).val = q1.val + q2.val
```

**lemma val\_smul**

[\[source\]](#)

```
@[simp]
lemma val_smul (r : ℂ) (q : UpSinglet) : (r • q).val = r • q.val
```

**def repLorentzGroup**

[\[source\]](#)

*The representation of the Lorentz group on the space of up-type quark fields.*

```
noncomputable def repLorentzGroup : Representation ℂ (SL(2,ℂ)) UpSinglet where
```

**def repGaugeGroupI**[\[source\]](#)

*The action of the full Standard Model gauge group on up-type quark fields.*

```
noncomputable def repGaugeGroupI : Representation ℂ GaugeGroupI UpSinglet where
```

**lemma repGaugeGroupI\_tmul**[\[source\]](#)

```
lemma repGaugeGroupI_tmul (g : GaugeGroupI) (ψ : Fermion.RightHandedWeyl)
  (v : EuclideanSpace ℂ (Fin 3)) :
  repGaugeGroupI g (ψ ⊗t v) = (g.toU1 ^ 4 • ψ ⊗t (g.toSU3.1.toEuclideanLin v))
```

**lemma repGaugeGroupI\_tmul\_basis\_eq\_sum**[\[source\]](#)

*The action of the full gauge group on a tensor product of basis elements, expanded as a sum over the columns of the SU(3) matrix.*

```
lemma repGaugeGroupI_tmul_basis_eq_sum (g : GaugeGroupI) (k : Fin 2) (i : Fin 3) :
  repGaugeGroupI g (RightHandedWeyl.basis k ⊗t [ℂ] EuclideanSpace.basisFun (Fin 3) ℂ i) =
  ∑ i' : Fin 3, (g.toU1.1 ^ 4 * g.toSU3.1 i' i)
  • ((RightHandedWeyl.basis k ⊗t [ℂ] EuclideanSpace.basisFun (Fin 3) ℂ i') : UpSinglet)
```

**lemma repGaugeGroupI\_eq\_iff\_mul\_eq**[\[source\]](#)

```
lemma repGaugeGroupI_eq_iff_mul_eq {g1 g2 : GaugeGroupI} :
  repGaugeGroupI g1 = repGaugeGroupI g2 ↔ ∀ i i',
  g1.toU1.1 ^ 4 * g1.toSU3.1 i' i = g2.toU1.1 ^ 4 * g2.toSU3.1 i' i
```

**lemma mem\_repGaugeGroupI\_ker\_iff\_eq**[\[source\]](#)

```
lemma mem_repGaugeGroupI_ker_iff_eq {g : GaugeGroupI} :
  g ∈ repGaugeGroupI.ker ↔ ∃ a : ℂ, g.toSU3.1 = a • 1 ∧ a * g.toU1.1 ^ 4 = 1
```

**lemma gaugeGroup\_subgroup\_ℤ₆\_le\_ker\_repGaugeGroupI**[\[source\]](#)

```
lemma gaugeGroup_subgroup_ℤ₆_le_ker_repGaugeGroupI :
  GaugeGroupQuot.subgroup .ℤ₆ ≤ repGaugeGroupI.ker
```

**lemma gaugeGroup\_subgroup\_le\_ker\_repGaugeGroupI**[\[source\]](#)

```
lemma gaugeGroup_subgroup_le_ker_repGaugeGroupI (Q : GaugeGroupQuot) :
  Q.subgroup ≤ repGaugeGroupI.ker
```

**def repGaugeGroup**[\[source\]](#)

*The action of the Standard Model gauge group, potentially quotiented by a discrete factor on quark fields.*

```
noncomputable def repGaugeGroup : (Q : GaugeGroupQuot) →
  Representation ℂ (GaugeGroup Q) UpSinglet
| .I => repGaugeGroupI
| .ℤ₆ => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .ℤ₆)
| .ℤ₂ => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .ℤ₂)
| .ℤ₃ => QuotientGroup.lift _ repGaugeGroupI (gaugeGroup_subgroup_le_ker_repGaugeGroupI .ℤ₃)
```

## 2.44 Physlib.QuantumMechanics.FreeParticle.Basic

[Physlib/QuantumMechanics/FreeParticle/Basic.lean](#) on GitHub.

### structure FreeParticle

[\[source\]](#)

*A free, spinless quantum particle with mass  $m > 0$  in Space  $d$ .*

```
structure FreeParticle (d : ℕ) where
  m : ℝ
  hm : 0 < m
```

### lemma m\_pos

[\[source\]](#)

```
@[simp]
lemma m_pos : 0 < Q.m
```

### lemma m\_nonneg

[\[source\]](#)

```
@[simp]
lemma m_nonneg : 0 ≤ Q.m
```

### lemma m\_ne\_zero

[\[source\]](#)

```
@[simp]
lemma m_ne_zero : Q.m ≠ 0
```

### abbrev HS

[\[source\]](#)

*The Hilbert space for the free particle.*

```
@[nolint unusedArguments]
abbrev HS (_ : FreeParticle d) : Type _
```

### def hamiltonian

[\[source\]](#)

*The Hamiltonian,  $p^2/(2m)$ .*

```
def hamiltonian : Q.HS →l [ℂ] Q.HS
```

### informal\_lemma hamiltonian\_essentially\_self\_adjoint

[\[source\]](#)

*The Hamiltonian for the free particle is essentially self-adjoint. This follows immediately from the ess. self-adjointness of the momentum-square operator.*

```
informal_lemma hamiltonian_essentially_self_adjoint where
  deps := [``FreeParticle]
  tag := "QM-FP-hamESA"
```

### informal\_definition toQuantumSystem

[\[source\]](#)

*The free particle as a quantum system (self-adjoint Hamiltonian acting on a Hilbert space).*

```
informal_definition toQuantumSystem where
  deps := [``FreeParticle]
  tag := "QM-FP-sys"
```

## 2.45 Physlib.QuantumMechanics.HarmonicOscillator.Basic

[Physlib/QuantumMechanics/HarmonicOscillator/Basic.lean](#) on GitHub.

**structure HarmonicOscillator**

[\[source\]](#)

*The  $d$ -dimensional quantum harmonic oscillator.*

```
structure HarmonicOscillator (d : ℕ) where
  m : ℝ
  hm : 0 < m
  ω : Fin d → ℝ
  hω : ∀ i, 0 < ω i
```

**lemma m\_pos**

[\[source\]](#)

```
@[simp]
lemma m_pos : 0 < Q.m
```

**lemma m\_nonneg**

[\[source\]](#)

```
@[simp]
lemma m_nonneg : 0 ≤ Q.m
```

**lemma m\_ne\_zero**

[\[source\]](#)

```
@[simp]
lemma m_ne_zero : Q.m ≠ 0
```

**lemma ω\_pos**

[\[source\]](#)

```
@[simp]
lemma ω_pos : 0 < Q.ω i
```

**lemma ω\_nonneg**

[\[source\]](#)

```
@[simp]
lemma ω_nonneg : 0 ≤ Q.ω i
```

**lemma ω\_ne\_zero**

[\[source\]](#)

```
@[simp]
lemma ω_ne_zero : Q.ω i ≠ 0
```

**def  $\xi$**  [\[source\]](#)

*The characteristic length  $\xi_i = \sqrt{\hbar} / (\sqrt{Q.m} * \sqrt{Q.\omega_i})$ .*

```
def  $\xi$  :  $\mathbb{R}$ 
```

**lemma  $\xi_{eq}$**  [\[source\]](#)

```
lemma  $\xi_{eq}$  :  $Q.\xi_i = \sqrt{\hbar} / (\sqrt{Q.m} * \sqrt{Q.\omega_i})$ 
```

**lemma  $\xi_{pos}$**  [\[source\]](#)

```
@[simp]
```

```
lemma  $\xi_{pos}$  :  $0 < Q.\xi_i$ 
```

**lemma  $\xi_{nonneg}$**  [\[source\]](#)

```
@[simp]
```

```
lemma  $\xi_{nonneg}$  :  $0 \leq Q.\xi_i$ 
```

**lemma  $\xi_{ne\_zero}$**  [\[source\]](#)

```
@[simp]
```

```
lemma  $\xi_{ne\_zero}$  :  $Q.\xi_i \neq 0$ 
```

**lemma  $\xi_{sq}$**  [\[source\]](#)

```
lemma  $\xi_{sq}$  :  $(Q.\xi_i)^2 = \hbar / (Q.m * Q.\omega_i)$ 
```

**lemma  $\xi_{inv}$**  [\[source\]](#)

```
lemma  $\xi_{inv}$  :  $(Q.\xi_i)^{-1} = \sqrt{Q.m} * \sqrt{Q.\omega_i} / \sqrt{\hbar}$ 
```

**lemma  $\xi_{inv}'$**  [\[source\]](#)

```
lemma  $\xi_{inv}'$  :  $(Q.\xi_i)^{-1} = Q.m * Q.\omega_i * Q.\xi_i / \hbar$ 
```

**def potentialMatrix** [\[source\]](#)

*The positive-definite matrix defining the quadratic potential function.*

```
def potentialMatrix : Matrix (Fin d) (Fin d)  $\mathbb{R}$ 
```

**lemma potentialMatrix\_eq** [\[source\]](#)

```
lemma potentialMatrix_eq :  $Q.potentialMatrix = diagonal ((2^{-1} * Q.m) \bullet Q.\omega^2)$ 
```

**lemma potentialMatrix\_isHermitian** [\[source\]](#)

```
lemma potentialMatrix_isHermitian :  $Q.potentialMatrix.IsHermitian$ 
```



**lemma potentialMatrix\_mulVec**[\[source\]](#)

```
@[simp]
lemma potentialMatrix_mulVec (v : Fin d → ℝ) :
  Q.potentialMatrix *v v = (2-1 * Q.m) • (Q.ω ^ 2 * v)
```

**def potentialQuadraticForm**[\[source\]](#)

*The positive-definite quadratic form associated to the potential matrix.*

```
def potentialQuadraticForm : QuadraticForm ℝ (Fin d → ℝ)
```

**def potentialFunction**[\[source\]](#)

*The quadratic potential function,  $\frac{1}{2}m \cdot \sum_i \omega_i^2 \cdot x_i^2$ .*

```
def potentialFunction : Space d → ℝ
```

**lemma potentialFunction\_eq**[\[source\]](#)

```
lemma potentialFunction_eq : Q.potentialFunction = Q.potentialQuadraticForm ◦ Space.val
```

**informal\_lemma potentialFunction\_aestronglyMeasurable**[\[source\]](#)

*The potential function for the harmonic oscillator is a.e. strongly measurable.*

```
informal_lemma potentialFunction_aestronglyMeasurable where
  deps := [``HarmonicOscillator]
  tag := "QM-H0-potAESM"
```

**def IsIsotropic**[\[source\]](#)

*A Harmonic oscillator is isotropic if all natural frequencies are equal.*

```
def IsIsotropic : Prop
```

**lemma isIsotropic\_def**[\[source\]](#)

```
lemma isIsotropic_def : Q.IsIsotropic ↔ ∀ i j, Q.ω i = Q.ω j
```

**lemma isIsotropic\_of\_one**[\[source\]](#)

```
lemma isIsotropic_of_one (Q : HarmonicOscillator 1) : Q.IsIsotropic
```

**abbrev HS**[\[source\]](#)

*The Hilbert space for the quantum harmonic oscillator.*

```
@[nolint unusedArguments]
abbrev HS (_ : HarmonicOscillator d) : Type _
```

**def kineticOperator**[\[source\]](#)*The kinetic energy operator,  $p^2/2m$ .*

```
def kineticOperator : Q.HS →l.[ℂ] Q.HS
```

**def potentialOperator**[\[source\]](#)*The potential operator which maps  $\psi$  to  $Q.potentialFunction \cdot \psi$ .*

```
def potentialOperator : Q.HS →l.[ℂ] Q.HS
```

**informal\_lemma potentialOperator\_isSelfAdjoint**[\[source\]](#)*The potential operators for the harmonic oscillator is self-adjoint.*

```
informal_lemma potentialOperator_isSelfAdjoint where
  deps := [``HarmonicOscillator.potentialFunction_aestronglyMeasurable]
  tag := "QM-H0-potSA"
```

**def hamiltonian**[\[source\]](#)*The Hamiltonian for the harmonic oscillator.*

```
def hamiltonian : Q.HS →l.[ℂ] Q.HS
```

**lemma hamiltonain\_eq**[\[source\]](#)

```
lemma hamiltonain_eq : Q.hamiltonian = Q.kineticOperator + Q.potentialOperator
```

**informal\_lemma hamiltonian\_essentially\_self\_adjoint**[\[source\]](#)*The Hamiltonian for the harmonic oscillator is essentially self-adjoint.*

```
informal_lemma hamiltonian_essentially_self_adjoint where
  deps := [``HarmonicOscillator.hamiltonian]
  tag := "QM-H0-hamESA"
```

**informal\_definition toQuantumSystem**[\[source\]](#)*The  $d$ -dimensional harmonic oscillator as a quantum system (self-adjoint Hamiltonian acting on a Hilbert space).*

```
informal_definition toQuantumSystem where
  deps := [``HarmonicOscillator.hamiltonian_essentially_self_adjoint]
  tag := "QM-H0-sys"
```

**2.46 Physlib.QuantumMechanics.HarmonicOscillator.OneDimension.Basic**

[Physlib/QuantumMechanics/HarmonicOscillator/OneDimension/Basic.lean](#) on GitHub.

**lemma m\_pos**[\[source\]](#)

```
@[simp]
lemma m_pos : 0 < Q.m
```

**lemma m\_nonneg**[\[source\]](#)

```
@[simp]
lemma m_nonneg : 0 ≤ Q.m
```

**lemma ω\_pos**[\[source\]](#)

```
@[simp]
lemma ω_pos : 0 < Q.ω
```

**lemma ω\_nonneg**[\[source\]](#)

```
@[simp]
lemma ω_nonneg : 0 ≤ Q.ω
```

**lemma ω\_ne\_zero**[\[source\]](#)

```
@[simp]
lemma ω_ne_zero : Q.ω ≠ 0
```

**2.47 Physlib.QuantumMechanics.HilbertSpaces.SpaceD.Basic**

[Physlib/QuantumMechanics/HilbertSpaces/SpaceD/Basic.lean](#) on GitHub.

**lemma toBra\_apply\_apply**[\[source\]](#)

```
@[simp]
lemma toBra_apply_apply : toBra ψ φ = ⟨⟨ψ, φ⟩⟩_ℂ
```

**lemma toBra\_symm\_apply**[\[source\]](#)

```
@[simp]
lemma toBra_symm_apply (f : StrongDual ℂ (SpaceDHilbertSpace d μ)) : ⟨⟨toBra.symm f, ψ⟩⟩_ℂ = f ψ
```

**lemma memHS\_coe**[\[source\]](#)

*Elements of the Hilbert space satisfy the property MemHS.*

```
lemma memHS_coe : MemHS ψ μ
```

**lemma zero**[\[source\]](#)

```
@[simp]
lemma MemHS.zero : MemHS (0 : Space d → ℂ) μ
```

**lemma neg** [\[source\]](#)

**lemma** MemHS.neg (hf : MemHS f  $\mu$ ) : MemHS (-f)  $\mu$

**lemma memHS\_neg\_iff** [\[source\]](#)

**lemma** memHS\_neg\_iff : MemHS (-f)  $\mu \leftrightarrow$  MemHS f  $\mu$

**lemma add** [\[source\]](#)

**lemma** MemHS.add (hf : MemHS f  $\mu$ ) (hg : MemHS g  $\mu$ ) : MemHS (f + g)  $\mu$

**lemma sub** [\[source\]](#)

**lemma** MemHS.sub (hf : MemHS f  $\mu$ ) (hg : MemHS g  $\mu$ ) : MemHS (f - g)  $\mu$

**lemma const\_smul** [\[source\]](#)

**lemma** MemHS.const\_smul (c :  $\mathbb{C}$ ) (hf : MemHS f  $\mu$ ) : MemHS (c • f)  $\mu$

**lemma memHS\_const\_smul\_iff** [\[source\]](#)

**lemma** memHS\_const\_smul\_iff {c :  $\mathbb{C}$ } (hc : c  $\neq$  0) : MemHS (c • f)  $\mu \leftrightarrow$  MemHS f  $\mu$

**lemma ae\_eq** [\[source\]](#)

**lemma** MemHS.ae\_eq (hfg : f =<sup>m</sup>[ $\mu$ ] g) (hf : MemHS f  $\mu$ ) : MemHS g  $\mu$

**lemma memHS\_congr\_ae** [\[source\]](#)

**lemma** memHS\_congr\_ae (hfg : f =<sup>m</sup>[ $\mu$ ] g) : MemHS f  $\mu \leftrightarrow$  MemHS g  $\mu$

**lemma congr\_norm** [\[source\]](#)

**lemma** MemHS.congr\_norm  
(hf : MemHS f  $\mu$ ) (hg : AESTronglyMeasurable g  $\mu$ ) (hfg :  $\forall^n x \in \partial\mu, \|f x\| = \|g x\|$ ) : MemHS g  $\mu$

**lemma memHS\_congr\_norm** [\[source\]](#)

**lemma** memHS\_congr\_norm (hf : AESTronglyMeasurable f  $\mu$ ) (hg : AESTronglyMeasurable g  $\mu$ )  
(hfg :  $\forall^n x \in \partial\mu, \|f x\| = \|g x\|$ ) : MemHS f  $\mu \leftrightarrow$  MemHS g  $\mu$

**lemma mono** [\[source\]](#)

**lemma** MemHS.mono (hf : MemHS f  $\mu$ ) (hg : AESTronglyMeasurable g  $\mu$ ) (hfg :  $\forall^n x \in \partial\mu, \|g x\| \leq \|f x\|$ ) :  
MemHS g  $\mu$

**lemma mono\_measure**[\[source\]](#)

```
lemma MemHS.mono_measure (h :  $\mu' \leq \mu$ ) (hf : MemHS f  $\mu$ ) : MemHS f  $\mu'$ 
```

**lemma restrict**[\[source\]](#)

```
lemma MemHS.restrict ( $\Omega$  : Set (Space d)) (hf : MemHS f  $\mu$ ) : MemHS f ( $\mu$ .restrict  $\Omega$ )
```

**lemma mono\_restrict**[\[source\]](#)

```
lemma MemHS.mono_restrict { $\Omega$   $\Omega'$  : Set (Space d)} (h :  $\Omega' \leq \Omega$ ) (hf : MemHS f ( $\mu$ .restrict  $\Omega$ )) :  
  MemHS f ( $\mu$ .restrict  $\Omega'$ )
```

**lemma indicator**[\[source\]](#)

```
lemma MemHS.indicator { $\Omega$  : Set (Space d)} (h $\Omega$  : MeasurableSet  $\Omega$ ) (hf : MemHS f  $\mu$ ) :  
  MemHS ( $\Omega$ .indicator f)  $\mu$ 
```

**lemma indicator\_of\_restrict**[\[source\]](#)

*If  $f$  is a member of the Hilbert space with measure restricted to  $\Omega$  then the representative  $\Omega$ .indicator  $f$  which vanishes outside  $\Omega$  is a member of SpaceDHilbertSpace  $d$   $\mu$ .*

```
lemma MemHS.indicator_of_restrict  
  { $\Omega$  : Set (Space d)} (h $\Omega$  : MeasurableSet  $\Omega$ ) (hf : MemHS f ( $\mu$ .restrict  $\Omega$ )) :  
  MemHS ( $\Omega$ .indicator f)  $\mu$ 
```

**lemma mk\_neg**[\[source\]](#)

```
@[simp]  
lemma mk_neg : mk hf.neg = -mk hf
```

**lemma mk\_sub**[\[source\]](#)

```
@[simp]  
lemma mk_sub : mk (hf.sub hg) = mk hf - mk hg
```

**lemma coeFn\_mk**[\[source\]](#)

```
lemma coeFn_mk : mk hf == $[\mu]$  f
```

**lemma coeFn\_zero**[\[source\]](#)

```
lemma coeFn_zero :  $\uparrow(\theta$  : SpaceDHilbertSpace  $d$   $\mu$ ) == $[\mu]$   $\theta$ 
```

**lemma coeFn\_neg**[\[source\]](#)

```
lemma coeFn_neg :  $\uparrow(-\psi)$  == $[\mu]$   $-\psi$ 
```

**lemma coeFn\_add**[\[source\]](#)

```
lemma coeFn_add : ↑(ψ.val + φ.val) =ᵐ[μ] ψ + φ
```

**lemma coeFn\_sub**[\[source\]](#)

```
lemma coeFn_sub : ↑(ψ.val - φ.val) =ᵐ[μ] ψ - φ
```

**lemma coeFn\_smul**[\[source\]](#)

```
lemma coeFn_smul : ↑(c • ψ) =ᵐ[μ] c • ψ
```

**abbrev toTemperedDistribution**[\[source\]](#)

*The tempered distribution associated to a state.*

```
abbrev toTemperedDistribution [μ.HasTemperateGrowth]
  (ψ : SpaceDHilbertSpace d μ) :  $\mathcal{S}'(\text{Space } d, \mathbb{C})$ 
```

**abbrev toTemperedDistributionCLM**[\[source\]](#)

*The embedding of states into tempered distributions as a continuous linear map.*

```
abbrev toTemperedDistributionCLM (d : ℕ) (μ : Measure (Space d) := volume)
  [μ.HasTemperateGrowth] : SpaceDHilbertSpace d μ →L[ $\mathbb{C}$ ]  $\mathcal{S}'(\text{Space } d, \mathbb{C})$ 
```

**abbrev SpaceDHilbertSpaceOn**[\[source\]](#)

*The  $L^2$ -space  $\text{SpaceDHilbertSpace } d \ (\mu.\text{restrict } \Omega)$ .*

*Elements are equivalence classes of functions which agree  $\mu$ -a.e. on  $\Omega$  and which have  $\int x \text{ in } \Omega, \|f x\|^2 \partial \mu$  finite.*

```
abbrev SpaceDHilbertSpaceOn {d : ℕ} (Ω : Set (Space d)) (μ : Measure (Space d) := volume)
```

**def subspaceProjection**[\[source\]](#)

*The linear map projecting  $\text{SpaceDHilbertSpace } d \ \mu$  onto  $\text{SpaceDHilbertSpaceOn } \Omega \ \mu$ .*

```
def subspaceProjection : SpaceDHilbertSpace d μ →L[ $\mathbb{C}$ ] SpaceDHilbertSpaceOn Ω μ where
```

**lemma subspaceProjection\_apply**[\[source\]](#)

```
lemma subspaceProjection_apply : subspaceProjection Ω μ ψ =ᵐ[μ.restrict Ω] ψ
```

**lemma subspaceProjection\_norm\_le**[\[source\]](#)

```
lemma subspaceProjection_norm_le : ||subspaceProjection Ω μ ψ|| ≤ ||ψ||
```

**def subspaceIncl**[\[source\]](#)

*The linear isometry including  $\text{SpaceDHilbertSpaceOn } \Omega \mu$  as a sub-Hilbert space of  $\text{SpaceDHilbertSpace } d \mu$ , defined by mapping  $\psi$  to  $\Omega.\text{indicator } \psi$ .*

```
def subspaceIncl : SpaceDHilbertSpaceOn  $\Omega \mu \rightarrow_{\text{L}} [\mathbb{C}] \text{SpaceDHilbertSpace } d \mu$  where
```

**lemma subspaceIncl\_apply**[\[source\]](#)

```
lemma subspaceIncl_apply : subspaceIncl h $\Omega \mu$   $\phi =^m [\mu] \Omega.\text{indicator } \phi$ 
```

**lemma leftInverse\_subspaceProjection**[\[source\]](#)

```
lemma leftInverse_subspaceProjection :  
  LeftInverse (subspaceProjection  $\Omega \mu$ ) (subspaceIncl h $\Omega \mu$ )
```

**lemma subspaceProjection\_subspaceIncl\_apply**[\[source\]](#)

```
@[simp]  
lemma subspaceProjection_subspaceIncl_apply : subspaceProjection  $\Omega \mu$  (subspaceIncl h $\Omega \mu$   $\phi$ ) =  $\phi$ 
```

**lemma subspaceProjection\_surjective**[\[source\]](#)

```
lemma subspaceProjection_surjective : Surjective (subspaceProjection  $\Omega \mu$ )
```

**2.48 Physlib.QuantumMechanics.HilbertSpaces.SpaceD.DirichletSubmodule**

[Physlib/QuantumMechanics/HilbertSpaces/SpaceD/DirichletSubmodule.lean](#) on GitHub.

**def DirichletSchwartzMap**[\[source\]](#)

*The Schwartz maps which vanish on frontier  $\Omega$ .*

```
def DirichletSchwartzMap : Submodule  $\mathbb{C} \mathcal{S}(\text{Space } d, \mathbb{C})$  where
```

**abbrev DirichletSubmoduleOn**[\[source\]](#)

*The submodule of the Hilbert space on  $\Omega$  consisting of the equivalence classes of Schwartz maps which vanish on frontier  $\Omega$ .*

```
abbrev DirichletSubmoduleOn : Submodule  $\mathbb{C} (\text{SpaceDHilbertSpaceOn } \Omega \mu)$ 
```

**lemma mem\_iff**[\[source\]](#)

```
lemma mem_iff { $\psi$  : SpaceDHilbertSpaceOn  $\Omega \mu$ } :  
   $\psi \in \text{DirichletSubmoduleOn } \Omega \mu \leftrightarrow$   
   $\exists f : \text{DirichletSchwartzMap } \Omega, \text{subspaceProjection } \Omega \mu (\text{schwartzIncl } \mu f) = \psi$ 
```

**lemma le\_schwartzSubmoduleOn**[\[source\]](#)

```
lemma le_schwartzSubmoduleOn : DirichletSubmoduleOn  $\Omega \mu \leq \text{SchwartzSubmoduleOn } \Omega \mu$ 
```

## 2.49 Physlib.QuantumMechanics.HilbertSpaces.SpaceD.Fourier

[Physlib/QuantumMechanics/HilbertSpaces/SpaceD/Fourier.lean](#) on GitHub.

**def** `fourierUnitary` [source]

*The  $L^2$  Fourier transform as a unitary on `SpaceDHilbertSpace d`.*

```
noncomputable def fourierUnitary (d : ℕ) :
  SpaceDHilbertSpace d  $\approx_{L^2}$  [ℂ] SpaceDHilbertSpace d
```

**lemma** `fourierUnitary_apply` [source]

*`fourierUnitary d` acts as the  $L^2$  Fourier transform  $\mathcal{F}$ .*

```
@[simp]
lemma fourierUnitary_apply (ψ : SpaceDHilbertSpace d) : fourierUnitary d ψ =  $\mathcal{F}$  ψ
```

**lemma** `fourierUnitary_symm_apply` [source]

*$(\text{fourierUnitary } d).\text{symm}$  acts as the inverse  $L^2$  Fourier transform  $\mathcal{F}^-$ .*

```
@[simp]
lemma fourierUnitary_symm_apply (ψ : SpaceDHilbertSpace d) : (fourierUnitary d).symm ψ =  $\mathcal{F}^-$  ψ
```

**lemma** `schwartzIncl_fourier_eq` [source]

*Applying `fourierUnitary d` to the  $L^2$  class of a Schwartz map  $f$  gives the  $L^2$  class of the Schwartz Fourier transform  $\mathcal{F} f$ .*

```
lemma schwartzIncl_fourier_eq (f :  $\mathcal{S}(\text{Space } d, \mathbb{C})$ ) :
   $\mathcal{F}$  (schwartzIncl volume f) = schwartzIncl volume ( $\mathcal{F} f$ )
```

**lemma** `schwartzIncl_fourierInv_eq` [source]

*Applying  $\mathcal{F}^-$  to the  $L^2$  class of a Schwartz map  $f$  gives the  $L^2$  class of the inverse Schwartz Fourier transform  $\mathcal{F}^- f$ .*

```
lemma schwartzIncl_fourierInv_eq (f :  $\mathcal{S}(\text{Space } d, \mathbb{C})$ ) :
   $\mathcal{F}^-$  (schwartzIncl volume f) = schwartzIncl volume ( $\mathcal{F}^- f$ )
```

**lemma** `fourierInv_schwartzIncl_fourier` [source]

*Pulling the  $L^2$  class of  $\mathcal{F} f$  back through the Fourier unitary recovers the  $L^2$  class of  $f$ .*

```
@[simp]
lemma fourierInv_schwartzIncl_fourier (f :  $\mathcal{S}(\text{Space } d, \mathbb{C})$ ) :
   $\mathcal{F}^-$  (schwartzIncl volume ( $\mathcal{F} f$ )) = schwartzIncl volume f
```

**lemma** `fourierUnitary_map_schwartzSubmodule` [source]

*The Fourier unitary maps the Schwartz submodule onto itself.*



```
lemma fourierUnitary_map_schwartzSubmodule :
  (SchwartzSubmodule d).map (fourierUnitary d).toLinearMap = SchwartzSubmodule d
```

## 2.50 Physlib.QuantumMechanics.HilbertSpaces.SpaceD.PolyBddSchwartzSubmodule

[Physlib/QuantumMechanics/HilbertSpaces/SpaceD/PolyBddSchwartzSubmodule.lean](#) on GitHub.

**def PolyBddSchwartzMap** [\[source\]](#)

*A function is a bounded Schwartz map if it is both Schwartz and bounded by powers of  $\|x\|$ .*

```
def PolyBddSchwartzMap (d : ℕ) (a : ℕ∞) : Submodule ℂ S(Space d, ℂ) where
```

**abbrev PolyBddSchwartzSubmodule** [\[source\]](#)

*The submodule of SpaceDHilbertSpace d corresponding to bounded Schwartz maps.*

```
abbrev PolyBddSchwartzSubmodule
  (d : ℕ) (a : ℕ∞) (μ : Measure (Space d) := volume) [μ.HasTemperateGrowth] :
  Submodule ℂ (SpaceDHilbertSpace d μ)
```

**instance CoeOut (PolyBddSchwartzMap d a) S(Space d, ℂ)** [\[source\]](#)

```
instance : CoeOut (PolyBddSchwartzMap d a) S(Space d, ℂ)
```

**instance CoeFun (PolyBddSchwartzMap d a) (fun \_ ↦ Space d → ℂ)** [\[source\]](#)

```
instance : CoeFun (PolyBddSchwartzMap d a) (fun _ ↦ Space d → ℂ)
```

**lemma PolyBddSchwartzMap\_zero\_eq\_top** [\[source\]](#)

```
lemma PolyBddSchwartzMap_zero_eq_top (d : ℕ) : PolyBddSchwartzMap d 0 = ⊤
```

**lemma PolyBddSchwartzMap\_antitone** [\[source\]](#)

```
lemma PolyBddSchwartzMap_antitone (d : ℕ) {a b : ℕ∞} (h : a ≤ b) :
  PolyBddSchwartzMap d b ≤ PolyBddSchwartzMap d a
```

**lemma le\_SchwartzSubmodule** [\[source\]](#)

```
lemma le_SchwartzSubmodule (d : ℕ) (a : ℕ∞) : PolyBddSchwartzSubmodule d a ≤ SchwartzSubmodule
  d
```

## 2.51 Physlib.QuantumMechanics.HilbertSpaces.SpaceD.SchwartzSubmodule

[Physlib/QuantumMechanics/HilbertSpaces/SpaceD/SchwartzSubmodule.lean](#) on GitHub.

**abbrev SchwartzSubmodule**[\[source\]](#)

*The submodule of  $\text{SpaceDHilbertSpace } d$  corresponding to Schwartz maps.*

```
abbrev SchwartzSubmodule (d : ℕ) (μ : Measure (Space d) := volume) [μ.HasTemperateGrowth] :
  Submodule ℂ (SpaceDHilbertSpace d μ)
```

```
instance CoeFun (SchwartzSubmodule d μ) fun _ ↦ Space d → ℂ
```

[\[source\]](#)

```
instance : CoeFun (SchwartzSubmodule d μ) fun _ ↦ Space d → ℂ
```

**lemma toTemperedDistribution\_schwartzIncl\_eq**[\[source\]](#)

```
lemma toTemperedDistribution_schwartzIncl_eq :
  toTemperedDistribution (schwartzIncl μ g) = g.toTemperedDistributionCLM (Space d) ℂ μ
```

**abbrev SchwartzSubmodule0n**[\[source\]](#)

*The projection of  $\text{SchwartzSubmodule } d \mu$  onto  $\text{SpaceDHilbertSpace0n } \Omega \mu$ .*

```
abbrev SchwartzSubmodule0n
  {d : ℕ} (Ω : Set (Space d)) (μ : Measure (Space d) := volume) [μ.HasTemperateGrowth] :
  Submodule ℂ (SpaceDHilbertSpace0n Ω μ)
```

**lemma mem\_iff**[\[source\]](#)

```
lemma mem_iff {ψ : SpaceDHilbertSpace0n Ω μ} :
  ψ ∈ SchwartzSubmodule0n Ω μ ↔ ∃ φ : SchwartzSubmodule d μ, subspaceProjection Ω μ φ = ψ
```

**2.52 Physlib.QuantumMechanics.HilbertSpaces.SpaceD.SobolevSubmodule**

[Physlib/QuantumMechanics/HilbertSpaces/SpaceD/SobolevSubmodule.lean](#) on GitHub.

**def SobolevSubmodule**[\[source\]](#)

*The Sobolev space  $H^s$  as a submodule of  $\text{SpaceDHilbertSpace } d$ : the  $L^2$  classes whose associated tempered distribution satisfies  $\text{MemSobolev } s \geq 2$ .*

```
def SobolevSubmodule (d : ℕ) (s : ℝ) : Submodule ℂ (SpaceDHilbertSpace d) where
```

**lemma mem\_sobolevSubmodule\_iff**[\[source\]](#)

*Membership in  $H^s$  is the Sobolev condition on the associated tempered distribution.*

```
lemma mem_sobolevSubmodule_iff {s : ℝ} {ψ : SpaceDHilbertSpace d} :
  ψ ∈ SobolevSubmodule d s ↔ MemSobolev s ≥ 2 (toTemperedDistribution ψ)
```

**lemma schwartzIncl\_mem**[\[source\]](#)

*Schwartz maps lie in every Sobolev space  $H^s$ .*

```
lemma SobolevSubmodule.schwartzIncl_mem (s : ℝ) (g : S(Space d, ℂ)) :
  schwartzIncl volume g ∈ SobolevSubmodule d s
```

**lemma schwartzSubmodule\_le\_sobolevSubmodule**[\[source\]](#)

*The Schwartz submodule is contained in every Sobolev space  $H^s$ .*

```
lemma schwartzSubmodule_le_sobolevSubmodule (s : ℝ) :
  SchwartzSubmodule d ≤ SobolevSubmodule d s
```

**lemma dense**[\[source\]](#)

*Every Sobolev space  $H^s$  is dense in SpaceDHilbertSpace d, containing the dense Schwartz submodule.*

```
lemma SobolevSubmodule.dense (s : ℝ) :
  Dense (SobolevSubmodule d s : Set (SpaceDHilbertSpace d))
```

**lemma antitone**[\[source\]](#)

*The Sobolev spaces shrink as the regularity index grows:  $H^s \leq H^{s'}$  for  $s' \leq s$ .*

```
lemma SobolevSubmodule.antitone (d : ℕ) : Antitone (SobolevSubmodule d)
```

**2.53 Physlib.QuantumMechanics.InfiniteSquareWell.Basic**

[Physlib/QuantumMechanics/InfiniteSquareWell/Basic.lean](#) on GitHub.

**structure InfiniteSquareWell**[\[source\]](#)

*A spinless quantum particle with mass  $m > 0$  confined to a cuboid in Space d.*

*The bounds of the well are specified by two functions lower upper : Fin d → ℝ satisfying  $\forall i, \text{lower } i < \text{upper } i$ .*

```
structure InfiniteSquareWell (d : ℕ) where
  m : ℝ
  hm : 0 < m
  lower : Fin d → ℝ
  upper : Fin d → ℝ
  h_bounds : ∀ i, lower i < upper i
```

**lemma m\_pos**[\[source\]](#)

```
@[simp]
lemma m_pos : 0 < Q.m
```

**lemma m\_nonneg**[\[source\]](#)

```
@[simp]
lemma m_nonneg : 0 ≤ Q.m
```

**lemma m\_ne\_zero**[\[source\]](#)

```
@[simp]
lemma m_ne_zero : Q.m ≠ 0
```

**def well** [\[source\]](#)

*The domain of the infinite square well as a Cartesian product of closed intervals.*

```
def well : Set (Space d)
```

**def measure** [\[source\]](#)

*The measure associated with the domain of the infinite square well.*

```
def measure : Measure (Space d)
```

**abbrev HS** [\[source\]](#)

*The Hilbert space for the infinite square well.*

```
abbrev HS : Type _
```

**informal\_definition hamiltonian** [\[source\]](#)

*The Hamiltonian for the infinite square well is  $(2m)^{-1}\text{momentumSqOperator}$  with respect to `InfiniteSquareWell.measure`. This requires first generalizing `momentumSqOperator` to `Space d` measures other than volume.*

```
informal_definition hamiltonian where
  deps := [``InfiniteSquareWell]
  tag := "QM-ISW-ham"
```

**informal\_lemma hamiltonian\_essentially\_self\_adjoint** [\[source\]](#)

*The Hamiltonian for the infinite square well is essentially self-adjoint.*

```
informal_lemma hamiltonian_essentially_self_adjoint where
  deps := [``InfiniteSquareWell]
  tag := "QM-ISW-hamESA"
```

**informal\_definition toQuantumSystem** [\[source\]](#)

*The particle in an infinite square well as a quantum system (self-adjoint Hamiltonian acting on a Hilbert space).*

```
informal_definition toQuantumSystem where
  deps := [``InfiniteSquareWell]
  tag := "QM-ISW-sys"
```

## 2.54 Physlib.QuantumMechanics.Operators.Multiplication

[Physlib/QuantumMechanics/Operators/Multiplication.lean](#) on GitHub.

**lemma mulOperator\_domain\_eq\_top** [\[source\]](#)

*The multiplication operator of a  $\mu$ -a.e. bounded function has full domain.*

```

lemma mulOperator_domain_eq_top {μ : Measure (Space d)}
  {f : Space d → ℂ} (hf : AESTronglyMeasurable f μ) {c : ℝ} (hfc : ∀ x ∂μ, ||f x|| ≤ c) :
  (M μ f).domain = τ

```

### **lemma mulOperator\_domain\_antitone**

[\[source\]](#)

*The domains of multiplication operators shrink with increasing function norm.*

```

lemma mulOperator_domain_antitone {μ : Measure (Space d)}
  {f g : Space d → ℂ} (hg : AESTronglyMeasurable g μ) (h : ∀ x ∂μ, ||g x|| ≤ ||f x||) :
  (M μ f).domain ≤ (M μ g).domain

```

### **lemma mulOperator\_domain\_eq\_of\_congr\_norm**

[\[source\]](#)

*The multiplication operators corresponding to functions of  $\mu$ -a.e. equal norm have the same domain.*

```

lemma mulOperator_domain_eq_of_congr_norm {μ : Measure (Space d)} {f g : Space d → ℂ}
  (hf : AESTronglyMeasurable f μ) (hg : AESTronglyMeasurable g μ) (h : ∀ x ∂μ, ||f x|| = ||g x||) :
  (M μ f).domain = (M μ g).domain

```

### **lemma mulOperator\_isClosed**

[\[source\]](#)

*Multiplication operators of  $\mu$ -a.e. strongly measurable functions are closed.*

```

lemma mulOperator_isClosed {μ : Measure (Space d)} [IsFiniteMeasureOnCompacts μ]
  {f : Space d → ℂ} (hf : AESTronglyMeasurable f μ) :
  (M μ f).IsClosed

```

### **lemma mulOperator\_zero**

[\[source\]](#)

*The multiplication operator of the zero function is the zero operator (domain  $\tau$ ).*

```

@[simp]
lemma mulOperator_zero (μ : Measure (Space d)) : M μ 0 = 0

```

### **lemma mulOperator\_eq\_of\_congr\_ae**

[\[source\]](#)

*$\mu$ -a.e. equal functions give rise to the same multiplication operator.*

```

lemma mulOperator_eq_of_congr_ae {μ : Measure (Space d)} {f g : Space d → ℂ} (h : f ==[μ] g) :
  M μ f = M μ g

```

### **lemma mulOperator\_const\_smul\_ge**

[\[source\]](#)

*Scalar multiplication and mulOperator commute except possibly for  $c = 0$  where the domains of  $0 \cdot M \mu f$  and  $M \mu 0 = 0$  may not agree.*

*See mulOperator\_const\_smul\_eq for equality when  $c \neq 0$ .*

```

lemma mulOperator_const_smul_ge (μ : Measure (Space d)) (f : Space d → ℂ) (c : ℂ) :
  c • M μ f ≤ M μ (c • f)

```

**lemma mulOperator\_const\_smul\_eq**[\[source\]](#)

*Scalar multiplication and mulOperator commute for  $c \neq 0$ .*

```
@[simp]
lemma mulOperator_const_smul_eq (μ : Measure (Space d)) (f : Space d → ℂ) {c : ℂ} (hc : c ≠ 0)
:
  M μ (c • f) = c • M μ f
```

**lemma mulOperator\_neg**[\[source\]](#)

*Negation and mulOperator commute.*

```
@[simp]
lemma mulOperator_neg (μ : Measure (Space d)) (f : Space d → ℂ) : M μ (-f) = -M μ f
```

**lemma mulOperator\_add\_ge**[\[source\]](#)

*$M μ (f + g)$  extends  $M μ f + M μ g$ .*

*In general the domains do not match:  $ψ ∈ (M μ f + M μ g).domain$  amounts to  $MemHS (f • ψ) μ$  and  $MemHS (g • ψ) μ$  whereas  $ψ ∈ (M μ (f + g)).domain$  is equivalent to the weaker condition  $MemHS ((f + g) • ψ) μ$ .*

*See mulOperator\_add\_eq and mulOperator\_add\_eq' for sufficient conditions to ensure equality.*

```
lemma mulOperator_add_ge (μ : Measure (Space d)) (f g : Space d → ℂ) :
  M μ f + M μ g ≤ M μ (f + g)
```

**lemma mulOperator\_add\_eq**[\[source\]](#)

*$(M μ g).domain = τ$  is a sufficient condition to ensure equality in mulOperator\_add\_ge.*

```
@[simp]
lemma mulOperator_add_eq
{μ : Measure (Space d)} (f : Space d → ℂ) {g : Space d → ℂ} (h : (M μ g).domain = τ) :
  M μ (f + g) = M μ f + M μ g
```

**lemma mulOperator\_add\_eq'**[\[source\]](#)

*Having both  $\|f x\| \leq c * \|f x + g x\|$  and  $\|g x\| \leq c' * \|f x + g x\|$   $\mu$ -a.e. is a sufficient condition to ensure equality in mulOperator\_add\_ge.*

```
lemma mulOperator_add_eq' {μ : Measure (Space d)}
{f g : Space d → ℂ} (hf : AESTronglyMeasurable f μ) (hg : AESTronglyMeasurable g μ)
(c c' : ℝ) (hf' : ∀m x ∂μ, \|f x\| ≤ c * \|f x + g x\|) (hg' : ∀m x ∂μ, \|g x\| ≤ c' * \|f x + g x\|) :
  M μ (f + g) = M μ f + M μ g
```

**lemma mulOperator\_sub\_ge**[\[source\]](#)

*$M μ (f - g)$  extends  $M μ f - M μ g$ .*

In general the domains do not match:  $\psi \in (\mathcal{M} \mu f - \mathcal{M} \mu g).domain$  amounts to  $MemHS (f \cdot \psi) \mu$  and  $MemHS (g \cdot \psi) \mu$  whereas  $\psi \in (\mathcal{M} \mu (f - g)).domain$  is equivalent to the weaker condition  $MemHS ((f - g) \cdot \psi) \mu$ .

See `mulOperator_sub_eq` and `mulOperator_sub_eq'` for sufficient conditions to ensure equality.

```
lemma mulOperator_sub_ge (μ : Measure (Space d)) (f g : Space d → ℂ) :
  M μ f - M μ g ≤ M μ (f - g)
```

**lemma mulOperator\_sub\_eq**

[\[source\]](#)

$(\mathcal{M} \mu g).domain = \tau$  is a sufficient condition to ensure equality in `mulOperator_sub_ge`.

@[simp]

**lemma** mulOperator\_sub\_eq

```
{μ : Measure (Space d)} {f : Space d → ℂ} {g : Space d → ℂ} (h : (M μ g).domain = τ) :
  M μ (f - g) = M μ f - M μ g
```

**lemma mulOperator\_sub\_eq'**

[\[source\]](#)

Having both  $\|f x\| \leq c * \|f x - g x\|$  and  $\|g x\| \leq c' * \|f x - g x\|$   $\mu$ -a.e. is a sufficient condition to ensure equality in `mulOperator_sub_ge`.

```
lemma mulOperator_sub_eq' {μ : Measure (Space d)}
  {f g : Space d → ℂ} (hf : AESTronglyMeasurable f μ) (hg : AESTronglyMeasurable g μ)
  (c c' : ℝ) (hf' : ∀m x ∂μ, \|f x\| ≤ c * \|f x - g x\|) (hg' : ∀m x ∂μ, \|g x\| ≤ c' * \|f x - g x\|) :
  M μ (f - g) = M μ f - M μ g
```

**lemma mulOperator\_smul\_ge**

[\[source\]](#)

$\mathcal{M} \mu (f \cdot g)$  extends  $\mathcal{M} \mu f * \mathcal{M} \mu g$ .

In general the domains do not match:  $\psi \in (\mathcal{M} \mu f * \mathcal{M} \mu g).domain$  amounts to  $MemHS (g \cdot \psi) \mu$  and  $MemHS (f \cdot g \cdot \psi) \mu$  whereas  $\psi \in (\mathcal{M} \mu (f \cdot g)).domain$  only requires  $MemHS (f \cdot g \cdot \psi) \mu$ .

See `mulOperator_smul_eq` for a sufficient condition to ensure equality.

```
lemma mulOperator_smul_ge (μ : Measure (Space d)) (f g : Space d → ℂ) :
  M μ f ∘r M μ g ≤ M μ (f · g)
```

**lemma mulOperator\_smul\_eq**

[\[source\]](#)

$(\mathcal{M} \mu g).domain = \tau$  is a sufficient condition to ensure equality in `mulOperator_smul_ge`.

**lemma** mulOperator\_smul\_eq

```
{μ : Measure (Space d)} {f : Space d → ℂ} {g : Space d → ℂ} (h : (M μ g).domain = τ) :
  M μ f ∘r M μ g = M μ (f · g)
```

## 2.55 Physlib.QuantumMechanics.Operators.OneDimension.Momentum

[Physlib/QuantumMechanics/Operators/OneDimension/Momentum.lean](#) on GitHub.

**lemma momentumOperatorUnbounded\_isSymmetric**[\[source\]](#)

```
lemma momentumOperatorUnbounded_isSymmetric : momentumOperatorUnbounded.IsSymmetric
```

**2.56 Physlib.QuantumMechanics.Operators.OneDimension.Parity**

[Physlib/QuantumMechanics/Operators/OneDimension/Parity.lean](#) on GitHub.

**lemma parityOperatorUnbounded\_isSymmetric**[\[source\]](#)

```
lemma parityOperatorUnbounded_isSymmetric :
  parityOperatorUnbounded.IsSymmetric
```

**2.57 Physlib.QuantumMechanics.Operators.OneDimension.Position**

[Physlib/QuantumMechanics/Operators/OneDimension/Position.lean](#) on GitHub.

**lemma positionOperatorUnbounded\_isSymmetric**[\[source\]](#)

```
lemma positionOperatorUnbounded_isSymmetric :
  positionOperatorUnbounded.IsSymmetric
```

**2.58 Physlib.QuantumMechanics.Operators.OneDimension.Unbounded**

[Physlib/QuantumMechanics/Operators/OneDimension/Unbounded.lean](#) on GitHub.

**def IsSymmetric**[\[source\]](#)

*The condition for an unbounded operator to be *\*symmetric\** (equivalently, formally self-adjoint):  $\langle U \psi_1, \psi_2 \rangle = \langle \psi_1, U \psi_2 \rangle$  for all  $\psi_1 \psi_2$ . This is the Hermitian pairing on the underlying space  $S$ ; on its own it does *\*\*not\*\** imply genuine self-adjointness ( $A^\dagger = A$ , including equality of domains), which for an unbounded operator on a proper dense core is strictly stronger.*

```
def IsSymmetric : Prop
```

**2.59 Physlib.QuantumMechanics.Operators.SpectralTheory.SelfAdjoint**

[Physlib/QuantumMechanics/Operators/SpectralTheory/SelfAdjoint.lean](#) on GitHub.

**lemma mem\_resolventSet\_of\_im\_ne\_zero**[\[source\]](#)

*Every non-real  $z$  lies in the resolvent set of a self-adjoint operator.*

```
lemma mem_resolventSet_of_im_ne_zero {z : ℂ} (hz : z.im ≠ 0) : z ∈ ρ T
```

**lemma sub\_smul\_surjective**[\[source\]](#)

*A self-adjoint operator has  $T - z \cdot 1$  surjective for every non-real  $z$ : off the real axis a self-adjoint operator has  $z$  in its resolvent set, so  $T - z \cdot 1$  has full range.*

```
lemma sub_smul_surjective {z : ℂ} (hz : z.im ≠ 0) : Function.Surjective (T - z • 1).toFun
```



**lemma unitaryConj\_isSelfAdjoint**[\[source\]](#)

*Unitary conjugation preserves self-adjointness: if  $A$  is a self-adjoint operator on  $H$  and  $u : H \approx_{\mathbb{C}} H'$  is unitary, then  $u A u^{-1}$  is self-adjoint on  $H'$ . Symmetry, dense domain, and the two deficiency surjectivities of  $A$  all transfer through  $u$ .*

```
lemma unitaryConj_isSelfAdjoint (u : H ≈ℂ H') {A : H →ℂ H} (hA : IsSelfAdjoint A) :
  IsSelfAdjoint (A.unitaryConj u)
```

**2.60 Physlib.QuantumMechanics.Operators.SpectralTheory.Symmetric**

[Physlib/QuantumMechanics/Operators/SpectralTheory/Symmetric.lean](#) on GitHub.

**lemma isEssentiallySelfAdjoint\_of\_defectNumber\_eq\_zero**[\[source\]](#)

*The basic criterion for essential self-adjointness: a symmetric, densely-defined operator whose defect numbers at  $I$  and  $-I$  both vanish is essentially self-adjoint.*

```
lemma isEssentiallySelfAdjoint_of_defectNumber_eq_zero
  (hdense : T.HasDenseDomain) (hpos : T.defectNumber I = 0) (hneg : T.defectNumber (-I) = 0) :
  T.IsEssentiallySelfAdjoint
```

**2.61 Physlib.QuantumMechanics.Operators.Unbounded**

[Physlib/QuantumMechanics/Operators/Unbounded.lean](#) on GitHub.

**lemma isStarNormal\_def**[\[source\]](#)

```
lemma isStarNormal_def [CompleteSpace H] : IsStarNormal T ↔ T† * T = T * T†
```

**lemma adjoint\_domain\_of\_continuous**[\[source\]](#)

```
lemma adjoint_domain_of_continuous [CompleteSpace H] (h : Continuous U) : U†.domain = T
```

**lemma adjoint\_add\_continuous**[\[source\]](#)

```
lemma HasDenseDomain.adjoint_add_continuous [CompleteSpace H]
  (h1 : T1.HasDenseDomain) (h2 : Continuous T2) (h : T1.domain ≤ T2.domain) :
  (T1 + T2)† = T1† + T2†
```

**lemma adjoint\_sub\_continuous**[\[source\]](#)

```
lemma HasDenseDomain.adjoint_sub_continuous [CompleteSpace H]
  (h1 : T1.HasDenseDomain) (h2 : Continuous T2) (h : T1.domain ≤ T2.domain) :
  (T1 - T2)† = T1† - T2†
```

**def LinearPMap.unitaryConj**[\[source\]](#)

*The conjugation  $u A u^{-1}$  of a partially-defined operator  $A : H \rightarrow_{\mathbb{C}} H$  by a unitary  $u : H \approx_{\mathbb{C}} H'$ , with domain  $\text{domain } u (A.\text{domain}) = u^{-1} (A.\text{domain})$  and action  $y \mapsto u (A (u^{-1} y))$ . Since  $u$  and  $u^{-1}$  are  $\mathbb{C}$ -linear, the result is again  $\mathbb{C}$ -linear.*

```
def unitaryConj : H' →ℂ H' where
```

**lemma mem\_unitaryConj\_domain\_iff**[\[source\]](#)

*Membership in the conjugated domain:  $x \in D(u A u^{-1}) \leftrightarrow u^{-1} x \in D(A)$ .*

```
lemma mem_unitaryConj_domain_iff {x : H'} :
  x ∈ (unitaryConj u A).domain ↔ u.symm x ∈ A.domain
```

**lemma unitaryConj\_apply**[\[source\]](#)

*The defining formula  $(u A u^{-1}) x = u (A (u^{-1} x))$ .*

```
lemma unitaryConj_apply (x : (unitaryConj u A).domain) :
  unitaryConj u A x = u (A (u.symm (x : H'), (mem_unitaryConj_domain_iff u A).mp x.2))
```

**lemma map\_mem\_unitaryConj\_domain**[\[source\]](#)

*$u$  maps  $D(A)$  into  $D(u A u^{-1})$ .*

```
lemma map_mem_unitaryConj_domain (y : A.domain) : u (y : H) ∈ (unitaryConj u A).domain
```

**lemma unitaryConj\_apply\_map**[\[source\]](#)

*The action on the image domain:  $(u A u^{-1})(u y) = u (A y)$  for  $y \in D(A)$ .*

```
lemma unitaryConj_apply_map (y : A.domain) :
  unitaryConj u A (u (y : H), map_mem_unitaryConj_domain u A y) = u (A y)
```

**lemma IsFormalAdjoint.unitaryConj**[\[source\]](#)

*Unitary conjugation preserves formal adjointness. If  $A$  is a formal adjoint of  $B$ , then  $u A u^{-1}$  is a formal adjoint of  $u B u^{-1}$ . Unitary conjugation preserves symmetry when  $A = B$ .*

```
lemma IsFormalAdjoint.unitaryConj {B : H →l [C] H} (h : A.IsFormalAdjoint B) :
  (unitaryConj u A).IsFormalAdjoint (unitaryConj u B)
```

**lemma unitaryConj\_dense\_domain**[\[source\]](#)

*If  $A$  has dense domain, then so does  $u A u^{-1}$ : the domain  $u^{-1} \cdot^{-1} (A.domain)$  is the preimage of a dense set under a homeomorphism.*

```
lemma HasDenseDomain.unitaryConj_dense_domain (hdense : A.HasDenseDomain) :
  (unitaryConj u A).HasDenseDomain
```

**lemma unitaryConj\_sub\_smul\_surjective**[\[source\]](#)

*If  $A - z$  is surjective for a scalar  $z : \mathbb{C}$ , then so is  $u A u^{-1} - z$ .*

```
lemma unitaryConj_sub_smul_surjective {z : ℂ} (h : Function.Surjective (A - z • 1).toFun) :
  Function.Surjective (unitaryConj u A - z • 1).toFun
```

**lemma closure\_le\_adjoint**[\[source\]](#)

```
lemma IsSymmetric.closure_le_adjoint [CompleteSpace H] (h : T.IsSymmetric) (h' : T.
  HasDenseDomain) :
  T.closure ≤ T†
```

**lemma isEssentiallySelfAdjoint\_iff**[\[source\]](#)

```
lemma IsSymmetric.isEssentiallySelfAdjoint_iff [CompleteSpace H]
  (h : T.IsSymmetric) (h' : T.HasDenseDomain) :
  T.IsEssentiallySelfAdjoint ↔ T†.domain = T.closure.domain
```

**lemma closure**[\[source\]](#)

*The closure of a symmetric densely-defined operator is symmetric:  $T^{\dagger\dagger}$  is a symmetric closed extension of  $T$ , so it extends  $T.closure$ , whose symmetry then descends.*

```
lemma IsSymmetric.closure [CompleteSpace H] (hsym : T.IsSymmetric) (hdense : T.HasDenseDomain)
  :
  T.closure.IsSymmetric
```

**lemma isSelfAdjoint\_of\_range\_eq\_top**[\[source\]](#)

*Self-adjointness from surjectivity of  $T \pm i$ : a symmetric, densely-defined operator  $T$  for which  $T + I \cdot 1$  and  $T - I \cdot 1$  both have full range is self-adjoint.*

```
lemma IsSymmetric.isSelfAdjoint_of_range_eq_top [CompleteSpace H] (hsym : T.IsSymmetric)
  (hdense : T.HasDenseDomain)
  (hadd : (T + I · 1).toFun.range = T) (hsub : (T - I · 1).toFun.range = T) :
  IsSelfAdjoint T
```

**2.62 Physlib.QuantumMechanics.QuantumSystem.Basic**

[Physlib/QuantumMechanics/QuantumSystem/Basic.lean](#) on GitHub.

**structure QuantumSystem**[\[source\]](#)

*A quantum system is identified by its Hilbert space and self-adjoint Hamiltonian operator.*

```
structure QuantumSystem where
  HS : Type*
  [instNormed : NormedAddCommGroup HS]
  [instInner : InnerProductSpace ℂ HS]
  [instComplete : CompleteSpace HS]
  H : HS →ℂ HS
  H_self_adjoint : IsSelfAdjoint H
```

**instance NormedAddCommGroup Q.HS**[\[source\]](#)

```
instance (Q : QuantumSystem) : NormedAddCommGroup Q.HS
```

**instance InnerProductSpace  $\mathbb{C}$  Q.HS**

[\[source\]](#)

```
instance (Q : QuantumSystem) : InnerProductSpace  $\mathbb{C}$  Q.HS
```

**instance CompleteSpace Q.HS**

[\[source\]](#)

```
instance (Q : QuantumSystem) : CompleteSpace Q.HS
```

**def mk\_esa**

[\[source\]](#)

*Create a quantum system from a Hamiltonian operator which is merely essentially self-adjoint by taking its closure.*

```
def mk_esa {HS : Type*} [NormedAddCommGroup HS] [InnerProductSpace  $\mathbb{C}$  HS] [CompleteSpace HS]
  { $\mathcal{H}$  : HS  $\rightarrow$   $\mathbb{C}$ } [HS] (h $\mathcal{H}$  : IsEssentiallySelfAdjoint  $\mathcal{H}$ ) : QuantumSystem
```

**instance instZero**

[\[source\]](#)

```
instance instZero : Zero QuantumSystem
```

**def UnitaryRelation**

[\[source\]](#)

*The relation on quantum systems where  $Q_1$  is related to  $Q_2$  if there exists a linear isometry equivalence  $e : Q_1.HS \approx_{\mathbb{C}} Q_2.HS$  between the respective Hilbert spaces satisfying  $e \circ Q_1.\mathcal{H} \circ e.symm = Q_2.\mathcal{H}$ .*

```
def UnitaryRelation (Q1 Q2 : QuantumSystem) : Prop
```

**lemma unitaryRelation\_refl**

[\[source\]](#)

*The relation UnitaryRelation is reflexive.*

```
lemma unitaryRelation_refl (Q : QuantumSystem) : UnitaryRelation Q Q
```

**lemma unitaryRelation\_symm**

[\[source\]](#)

*The relation UnitaryRelation is symmetric.*

```
lemma unitaryRelation_symm {Q1 Q2 : QuantumSystem} (h12 : UnitaryRelation Q1 Q2) :
  UnitaryRelation Q2 Q1
```

**lemma unitaryRelation\_trans**

[\[source\]](#)

*The relation UnitaryRelation is transitive.*

```
lemma unitaryRelation_trans {Q1 Q2 Q3 : QuantumSystem} (h12 : UnitaryRelation Q1 Q2)
  (h23 : UnitaryRelation Q2 Q3) : UnitaryRelation Q1 Q3
```

**lemma unitaryRelation\_equiv**

[\[source\]](#)

*The relation UnitaryRelation is an equivalence relation.*

```
lemma unitaryRelation_equiv : Equivalence UnitaryRelation where
```

**instance QuantumSystemSetoid**[\[source\]](#)

*The setoid of quantum systems with UnitaryRelation for equivalence relation.*

```
instance QuantumSystemSetoid : Setoid QuantumSystem
```

**2.63 Physlib.QuantumMechanics.RectangularBarrier.Basic**

[Physlib/QuantumMechanics/RectangularBarrier/Basic.lean](#) on GitHub.

**structure RectangularBarrier**[\[source\]](#)

*A quantum particle with mass  $m > 0$  on Space 1 subject to a rectangular potential barrier.*

*The potential is  $V_0$  on the interval  $I_{cc}$  lower upper and zero elsewhere.*

```
structure RectangularBarrier where
```

```
  m : ℝ
  hm : 0 < m
  lower : ℝ
  upper : ℝ
  h_bounds : lower < upper
  V_0 : ℝ
```

**lemma m\_pos**[\[source\]](#)

```
@[simp]
lemma m_pos : 0 < Q.m
```

**lemma m\_nonneg**[\[source\]](#)

```
@[simp]
lemma m_nonneg : 0 ≤ Q.m
```

**lemma m\_ne\_zero**[\[source\]](#)

```
@[simp]
lemma m_ne_zero : Q.m ≠ 0
```

**def potentialFunction**[\[source\]](#)

*The piece-wise constant potential, equal to  $Q.V_0$  for  $x.val \theta \in I_{cc} \ Q.lower \ Q.upper$  and zero otherwise.*

```
def potentialFunction : Space 1 → ℝ
```

**lemma potentialFunction\_eq**[\[source\]](#)

```
lemma potentialFunction_eq :
  Q.potentialFunction = fun x ↦ (Icc Q.lower Q.upper).indicator (fun _ ↦ Q.V_0) (x θ)
```

**informal\_lemma potentialFunction\_aestronglyMeasurable**[\[source\]](#)

*The piecewise-constant potential of the rectangular barrier is a.e. strongly measurable.*

```
informal_lemma potentialFunction_aestronglyMeasurable where
  deps := [``RectangularBarrier]
  tag := "QM-RB-aesm"
```

**abbrev HS**[\[source\]](#)

*The Hilbert space for the 1d rectangular barrier.*

```
@[nolint unusedArguments]
abbrev HS (_ : RectangularBarrier) : Type _
```

**def kineticOperator**[\[source\]](#)

*The kinetic energy operator,  $p^2/2m$ .*

```
def kineticOperator : Q.HS →l.[C] Q.HS
```

**def potentialOperator**[\[source\]](#)

*The potential energy operator, defined by multiplication by  $Q.potentialFunction$ .*

```
def potentialOperator : Q.HS →l.[C] Q.HS
```

**informal\_lemma potentialOperator\_isSelfAdjoint**[\[source\]](#)

*The potential operator for the rectangular barrier is self-adjoint.*

```
informal_lemma potentialOperator_isSelfAdjoint where
  deps := [``RectangularBarrier]
  tag := "QM-RB-sa"
```

**informal\_definition hamiltonian**[\[source\]](#)

*The Hamiltonian for the rectangular barrier.*

```
informal_definition hamiltonian where
  deps := [``RectangularBarrier]
  tag := "QM-RB-ham"
```

**informal\_lemma hamiltonian\_essentially\_self\_adjoint**[\[source\]](#)

*The Hamiltonian for the rectangular barrier is essentially self-adjoint.*

```
informal_lemma hamiltonian_essentially_self_adjoint where
  deps := [``RectangularBarrier.hamiltonian]
  tag := "QM-RB-hamESA"
```

**informal\_definition toQuantumSystem**[\[source\]](#)

*The rectangular barrier as a quantum system (self-adjoint Hamiltonian acting on a Hilbert space).*

```
informal_definition toQuantumSystem where
  deps := [ `RectangularBarrier.hamiltonian_essentially_self_adjoint ]
  tag := "QM-RB-sys"
```

**2.64 Physlib.Relativity.Fermions.Dirac.Basic**

[Physlib/Relativity/Fermions/Dirac/Basic.lean](#) on GitHub.

**structure Dirac**[\[source\]](#)

*A Dirac fermion, consisting of a left handed Weyl fermion and a dual right handed Weyl fermion.*

```
structure Dirac where
  left : LeftHandedWeyl
  dualRight : DualRightHandedWeyl
```

**def decomposeEquiv**[\[source\]](#)

*The decomposition of a Dirac fermion into its left handed and dual right handed components.*

```
def decomposeEquiv : Dirac ≈ LeftHandedWeyl × DualRightHandedWeyl where
```

**instance AddCommGroup Dirac**[\[source\]](#)

```
instance : AddCommGroup Dirac
```

**instance Module ℂ Dirac**[\[source\]](#)

```
instance : Module ℂ Dirac
```

**lemma left\_add**[\[source\]](#)

```
@[simp]
lemma left_add (d₁ d₂ : Dirac) : (d₁ + d₂).left = d₁.left + d₂.left
```

**lemma dualRight\_add**[\[source\]](#)

```
@[simp]
lemma dualRight_add (d₁ d₂ : Dirac) : (d₁ + d₂).dualRight = d₁.dualRight + d₂.dualRight
```

**lemma left\_smul**[\[source\]](#)

```
@[simp]
lemma left_smul (c : ℂ) (d : Dirac) : (c • d).left = c • d.left
```

**lemma dualRight\_smul**[\[source\]](#)`@[simp]``lemma dualRight_smul (c : ℂ) (d : Dirac) : (c • d).dualRight = c • d.dualRight`**def decomposeLinEquiv**[\[source\]](#)*The linear equivalence between Dirac and LeftHandedWeyl × DualRightHandedWeyl.*`def decomposeLinEquiv : Dirac ≈[ℂ] LeftHandedWeyl × DualRightHandedWeyl where`**def chiralBasis**[\[source\]](#)*The chiral basis of the Dirac fermions.*`def chiralBasis : Basis (Fin 4) ℂ Dirac`**lemma chiralBasis\_cast\_add**[\[source\]](#)`lemma chiralBasis_cast_add (i : Fin 2) : chiralBasis (Fin.castAdd 2 i) =  
 (LeftHandedWeyl.basis i, 0)`**lemma chiralBasis\_nat\_add**[\[source\]](#)`lemma chiralBasis_nat_add (i : Fin 2) : chiralBasis (Fin.natAdd 2 i) =  
 (0, DualRightHandedWeyl.basis i)`**def rep**[\[source\]](#)*The representation of  $SL(2, \mathbb{C})$  on Dirac fermions.*`def rep : Representation ℂ SL(2,ℂ) Dirac where`**lemma rep\_apply\_mk**[\[source\]](#)`lemma rep_apply_mk (g : SL(2,ℂ)) (ψ : LeftHandedWeyl) (χ : DualRightHandedWeyl) :  
 rep g ⟨ψ, χ⟩ = ⟨LeftHandedWeyl.rep g ψ, DualRightHandedWeyl.rep g χ⟩`**def decomposeRepEquiv**[\[source\]](#)*The equivalence between the representation on Dirac and the representation on LeftHandedWeyl × DualRightHandedWeyl.*`def decomposeRepEquiv : rep.Equiv ((LeftHandedWeyl.rep).prod (DualRightHandedWeyl.rep)) where`**2.65 Physlib.Relativity.Fermions.Weyl.DualLeftHanded**[Physlib/Relativity/Fermions/Weyl/DualLeftHanded.lean](#) on GitHub.**lemma toFin2C\_eq\_val**[\[source\]](#)`lemma toFin2C_eq_val (ψ : DualLeftHandedWeyl) : ψ.toFin2C = ψ.val`



**def basis** [source]

*The standard basis on dual-left-handed Weyl fermions.*

```
def basis : Basis (Fin 2) ℂ DualLeftHandedWeyl
```

**lemma basis\_apply** [source]

```
lemma basis_apply (i j : Fin 2) : (basis i).1 j = if j = i then 1 else 0
```

**lemma eq\_sum\_basis** [source]

```
lemma eq_sum_basis (ψ : DualLeftHandedWeyl) : ψ = ∑ i, ψ.1 i • basis i
```

**lemma basis\_val** [source]

```
lemma basis_val (i : Fin 2) : (basis i).val = Pi.single i 1
```

**def rep** [source]

*The vector space  $\mathbb{C}^2$  carrying the representation of  $SL(2, \mathbb{C})$  given by  $M \rightarrow (M^{-1})^T$ .  
In index notation corresponds to a left-handed Weyl fermion with indices  $\psi_a$ .*

```
def rep : Representation ℂ SL(2, ℂ) DualLeftHandedWeyl where
```

**lemma rep\_apply\_eq\_sum\_basis** [source]

```
lemma rep_apply_eq_sum_basis (M : SL(2, ℂ)) (ψ : DualLeftHandedWeyl) :  
  rep M ψ = ∑ i, (∑ j, M.1⁻¹ j i * ψ.1 j) • basis i
```

**lemma rep\_apply\_basis** [source]

```
lemma rep_apply_basis (M : SL(2, ℂ)) (i : Fin 2) :  
  rep M (basis i) = ∑ j, M.1⁻¹ i j • basis j
```

**lemma rep\_toMatrix** [source]

```
lemma rep_toMatrix (M : SL(2, ℂ)) : (LinearMap.toMatrix basis basis) (rep M) = (M.1⁻¹)ᵀ
```

**lemma rep\_apply\_basis\_repr** [source]

```
lemma rep_apply_basis_repr (M : SL(2, ℂ)) (i j : Fin 2) :  
  basis.repr (rep M (basis i)) j = M.1⁻¹ i j
```

## 2.66 Physlib.Relativity.Fermions.Weyl.DualRightHanded

[Physlib/Relativity/Fermions/Weyl/DualRightHanded.lean](#) on GitHub.

**lemma toFin2ℂ\_eq\_val** [source]

```
lemma toFin2ℂ_eq_val (ψ : DualRightHandedWeyl) : ψ.toFin2ℂ = ψ.val
```

**def basis** [\[source\]](#)

*The standard basis on dual-right-handed Weyl fermions.*

```
def basis : Basis (Fin 2) ℂ DualRightHandedWeyl
```

**lemma basis\_apply** [\[source\]](#)

```
lemma basis_apply (i j : Fin 2) : (basis i).1 j = if j = i then 1 else 0
```

**lemma eq\_sum\_basis** [\[source\]](#)

```
lemma eq_sum_basis (ψ : DualRightHandedWeyl) : ψ = ∑ i, ψ.1 i • basis i
```

**lemma basis\_val** [\[source\]](#)

```
lemma basis_val (i : Fin 2) : (basis i).val = Pi.single i 1
```

**def rep** [\[source\]](#)

*The vector space  $\mathbb{C}^2$  carrying the representation of  $SL(2, \mathbb{C})$  given by  $M \rightarrow (M^{-1})^t$ .  
In index notation this corresponds to a Weyl fermion with index  $\psi_{\dot{a}}$ .*

```
def rep : Representation ℂ SL(2, ℂ) DualRightHandedWeyl where
```

**lemma rep\_apply** [\[source\]](#)

```
lemma rep_apply (M : SL(2, ℂ)) (ψ : DualRightHandedWeyl) :  
  rep M ψ = ⟨(M.1-1).conjTranspose *v ψ.1⟩
```

**lemma rep\_apply\_eq\_sum\_basis** [\[source\]](#)

```
lemma rep_apply_eq_sum_basis (M : SL(2, ℂ)) (ψ : DualRightHandedWeyl) :  
  rep M ψ = ∑ i, (∑ j, (M.1-1).conjTranspose i j * ψ.1 j) • basis i
```

**lemma rep\_apply\_basis** [\[source\]](#)

```
lemma rep_apply_basis (M : SL(2, ℂ)) (i : Fin 2) :  
  rep M (basis i) = ∑ j, (M.1-1).conjTranspose j i • basis j
```

**lemma rep\_toMatrix** [\[source\]](#)

```
lemma rep_toMatrix (M : SL(2, ℂ)) :  
  (LinearMap.toMatrix basis basis) (rep M) = (M.1-1).conjTranspose
```

**lemma rep\_apply\_basis\_repr** [\[source\]](#)

```
lemma rep_apply_basis_repr (M : SL(2, ℂ)) (i j : Fin 2) :  
  basis_repr (rep M (basis i)) j = star (M.1-1 i j)
```

## 2.67 Physlib.Relativity.Fermions.Weyl.Duals

[Physlib/Relativity/Fermions/Weyl/Duals.lean](#) on GitHub.

**def LeftHandedWeyl.dual** [\[source\]](#)

*The morphism between the representation leftHanded and the representation dualLeftHanded defined by multiplying an element of leftHanded by the matrix  $\varepsilon^{a\theta a_1} = !![0, 1; -1, 0]$ .*

**def** LeftHandedWeyl.dual : LeftHandedWeyl.rep.IntertwiningMap DualLeftHandedWeyl.rep **where**

**lemma LeftHandedWeyl.dual\_hom\_apply** [\[source\]](#)

**lemma** LeftHandedWeyl.dual\_hom\_apply ( $\psi$  : LeftHandedWeyl) :  
 LeftHandedWeyl.dual  $\psi$  =  
 DualLeftHandedWeyl.toFin2CEquiv.symm (!! [0, 1; -1, 0] \*<sub>v</sub>  $\psi$ .toFin2C)

**def DualLeftHandedWeyl.dual** [\[source\]](#)

*The morphism from dualLeftHanded to leftHanded defined by multiplying an element of DualLeftHandedWeyl by the matrix  $\varepsilon_{a_1 a_2} = !![0, -1; 1, 0]$ .*

**def** DualLeftHandedWeyl.dual : DualLeftHandedWeyl.rep.IntertwiningMap LeftHandedWeyl.rep **where**

**lemma DualLeftHandedWeyl.dual\_hom\_apply** [\[source\]](#)

**lemma** DualLeftHandedWeyl.dual\_hom\_apply ( $\psi$  : DualLeftHandedWeyl) :  
 DualLeftHandedWeyl.dual  $\psi$  =  
 LeftHandedWeyl.toFin2CEquiv.symm (!! [0, -1; 1, 0] \*<sub>v</sub>  $\psi$ .toFin2C)

**def LeftHandedWeyl.dualEquiv** [\[source\]](#)

*The equivalence between the representation leftHanded and the representation dualLeftHanded defined by multiplying an element of leftHanded by the matrix  $\varepsilon^{a\theta a_1} = !![0, 1; -1, 0]$ .*

**def** LeftHandedWeyl.dualEquiv : LeftHandedWeyl.rep.Equiv DualLeftHandedWeyl.rep

**lemma dualEquiv\_hom\_hom\_apply** [\[source\]](#)

*leftHandedDualEquiv acting on an element  $\psi$  : leftHanded corresponds to multiplying  $\psi$  by the matrix  $!![0, 1; -1, 0]$ .*

**lemma** LeftHandedWeyl.dualEquiv\_hom\_hom\_apply ( $\psi$  : LeftHandedWeyl) :  
 LeftHandedWeyl.dualEquiv  $\psi$  =  
 DualLeftHandedWeyl.toFin2CEquiv.symm (!! [0, 1; -1, 0] \*<sub>v</sub>  $\psi$ .toFin2C)

**lemma dualEquiv\_inv\_hom\_apply** [\[source\]](#)

*The inverse of leftHandedDualEquiv acting on an element  $\psi$  : dualLeftHanded corresponds to multiplying  $\psi$  by the matrix  $!![0, -1; 1, 0]$ .*

```
lemma LeftHandedWeyl.dualEquiv_inv_hom_apply (ψ : DualLeftHandedWeyl) :
  LeftHandedWeyl.dualEquiv.symm ψ =
  LeftHandedWeyl.toFin2CEquiv.symm (!![0, -1; 1, 0] *v ψ.toFin2C)
```

**informal\_definition RightHandedWeyl.dualEquiv**

[\[source\]](#)

*The linear equivalence between `rightHandedWeyl` and `DualRightHandedWeyl` given by multiplying an element of `rightHandedWeyl` by the matrix  $\varepsilon^{a\theta a_1} = ![0, 1; -1, 0]$ .*

```
informal_definition RightHandedWeyl.dualEquiv where
  deps := [``RightHandedWeyl, ``DualRightHandedWeyl]
  tag := "6VZR4"
```

**informal\_lemma dualEquiv\_equivariant**

[\[source\]](#)

*The linear equivalence `rightHandedWeylDualEquiv` is equivariant with respect to the action of  $SL(2, \mathbb{C})$  on `rightHandedWeyl` and `DualRightHandedWeyl`.*

```
informal_lemma RightHandedWeyl.dualEquiv_equivariant where
  deps := [``RightHandedWeyl.dualEquiv]
  tag := "6VZSG"
```

## 2.68 Physlib.Relativity.Fermions.Weyl.LeftHanded

[Physlib/Relativity/Fermions/Weyl/LeftHanded.lean](#) on GitHub.

**lemma toFin2C\_eq\_val**

[\[source\]](#)

```
lemma toFin2C_eq_val (ψ : LeftHandedWeyl) : ψ.toFin2C = ψ.val
```

**def basis**

[\[source\]](#)

*The standard basis on left-handed Weyl fermions.*

```
def basis : Basis (Fin 2) ℂ LeftHandedWeyl
```

**lemma basis\_apply**

[\[source\]](#)

```
lemma basis_apply (i j : Fin 2) : (basis i).1 j = if j = i then 1 else 0
```

**lemma eq\_sum\_basis**

[\[source\]](#)

```
lemma eq_sum_basis (ψ : LeftHandedWeyl) : ψ = ∑ i, ψ.1 i • basis i
```

**lemma basis\_val**

[\[source\]](#)

```
lemma basis_val (i : Fin 2) : (basis i).val = Pi.single i 1
```

**def rep** [\[source\]](#)

*The vector space  $\mathbb{C}^2$  carrying the fundamental representation of  $SL(2, \mathbb{C})$ . In index notation corresponds to a Weyl fermion with indices  $\psi^a$ .*

```
def rep : Representation ℂ SL(2, ℂ) LeftHandedWeyl where
```

**lemma rep\_apply** [\[source\]](#)

```
lemma rep_apply (M : SL(2, ℂ)) (ψ : LeftHandedWeyl) : rep M ψ = ⟨M.1 *v ψ.1⟩
```

**lemma rep\_apply\_eq\_sum\_basis** [\[source\]](#)

```
lemma rep_apply_eq_sum_basis (M : SL(2, ℂ)) (ψ : LeftHandedWeyl) :
  rep M ψ = ∑ i, (∑ j, M.1 i j * ψ.1 j) • basis i
```

**lemma rep\_apply\_basis** [\[source\]](#)

```
lemma rep_apply_basis (M : SL(2, ℂ)) (i : Fin 2) :
  rep M (basis i) = ∑ j, M.1 j i • basis j
```

**lemma rep\_toMatrix** [\[source\]](#)

```
lemma rep_toMatrix (M : SL(2, ℂ)) : (LinearMap.toMatrix basis basis) (rep M) = M.1
```

**lemma rep\_apply\_basis\_repr** [\[source\]](#)

```
lemma rep_apply_basis_repr (M : SL(2, ℂ)) (i j : Fin 2) :
  basis.repr (rep M (basis i)) j = M.1 j i
```

## 2.69 Physlib.Relativity.Fermions.Weyl.RightHanded

[Physlib/Relativity/Fermions/Weyl/RightHanded.lean](#) on GitHub.

**lemma toFin2ℂ\_eq\_val** [\[source\]](#)

```
lemma toFin2ℂ_eq_val (ψ : RightHandedWeyl) : ψ.toFin2ℂ = ψ.val
```

**def basis** [\[source\]](#)

*The standard basis on right-handed Weyl fermions.*

```
def basis : Basis (Fin 2) ℂ RightHandedWeyl
```

**lemma basis\_apply** [\[source\]](#)

```
lemma basis_apply (i j : Fin 2) : (basis i).1 j = if j = i then 1 else 0
```

**lemma eq\_sum\_basis** [\[source\]](#)

```
lemma eq_sum_basis (ψ : RightHandedWeyl) : ψ = ∑ i, ψ.1 i • basis i
```

**lemma basis\_val**[\[source\]](#)

```
lemma basis_val (i : Fin 2) : (basis i).val = Pi.single i 1
```

**def rep**[\[source\]](#)

*The vector space  $\mathbb{C}^2$  carrying the conjugate representation of  $SL(2, \mathbb{C})$ . In index notation corresponds to a Weyl fermion with indices  $\psi^{\dot{a}}$ .*

```
def rep : Representation  $\mathbb{C}$  SL(2,  $\mathbb{C}$ ) RightHandedWeyl where
```

**lemma rep\_apply**[\[source\]](#)

```
lemma rep_apply (M : SL(2,  $\mathbb{C}$ )) (ψ : RightHandedWeyl) : rep M ψ = (M.1.map star *v ψ.1)
```

**lemma rep\_apply\_eq\_sum\_basis**[\[source\]](#)

```
lemma rep_apply_eq_sum_basis (M : SL(2,  $\mathbb{C}$ )) (ψ : RightHandedWeyl) :  
  rep M ψ =  $\sum i, (\sum j, M.1.map star i j * ψ.1 j) \cdot \text{basis } i$ 
```

**lemma rep\_apply\_basis**[\[source\]](#)

```
lemma rep_apply_basis (M : SL(2,  $\mathbb{C}$ )) (i : Fin 2) :  
  rep M (basis i) =  $\sum j, M.1.map star j i \cdot \text{basis } j$ 
```

**lemma rep\_toMatrix**[\[source\]](#)

```
lemma rep_toMatrix (M : SL(2,  $\mathbb{C}$ )) : (LinearMap.toMatrix basis basis) (rep M) = M.1.map star
```

**lemma rep\_apply\_basis\_repr**[\[source\]](#)

```
lemma rep_apply_basis_repr (M : SL(2,  $\mathbb{C}$ )) (i j : Fin 2) :  
  basis_repr (rep M (basis i)) j = star (M.1 j i)
```

**2.70 Physlib.Relativity.LorentzGroup.Basic**

[Physlib/Relativity/LorentzGroup/Basic.lean](#) on GitHub.

**lemma range\_toGL**[\[source\]](#)

*The range of `toGL`, as a set in  $GL(\text{Fin } 1 \oplus \text{Fin } d) \mathbb{R}$ , is exactly the preimage of the Lorentz group under the coercion  $GL(\text{Fin } 1 \oplus \text{Fin } d) \mathbb{R} \rightarrow \text{Matrix } \_ \_ \mathbb{R}$ : an element of the general linear group lies in the image precisely when its underlying matrix is a Lorentz transformation.*

```
lemma range_toGL :  
  Set.range (@toGL d).toFun =  
    Units.val-1' (LorentzGroup d : Set (Matrix (Fin 1  $\oplus$  Fin d) (Fin 1  $\oplus$  Fin d)  $\mathbb{R}$ ))
```

**lemma toGL\_isClosedEmbedding**[\[source\]](#)

The homomorphism `toGL` is a closed embedding of the Lorentz group into  $GL(\text{Fin } 1 \oplus \text{Fin } d, \mathbb{R})$ . This strengthens `toGL_embedding`: the image is closed because it is the preimage, under the continuous coercion  $GL(\text{Fin } 1 \oplus \text{Fin } d, \mathbb{R}) \rightarrow \text{Matrix } \_ \_ \mathbb{R}$ , of the closed set `LorentzGroup d` (`LorentzGroup.isClosed`).

```
lemma toGL_isClosedEmbedding : Topology.IsClosedEmbedding (@toGL d).toFun where
```

**def toGLSubgroup**[\[source\]](#)

The Lorentz group realised as a subgroup of  $GL(\text{Fin } 1 \oplus \text{Fin } d, \mathbb{R})$ , namely the range of the embedding `toGL`.

```
def toGLSubgroup (d : ℕ) : Subgroup (GL (Fin 1 ⊕ Fin d) ℝ)
```

**lemma mem\_toGLSubgroup**[\[source\]](#)

```
@[simp]
```

```
lemma mem_toGLSubgroup {x : GL (Fin 1 ⊕ Fin d) ℝ} :  
  x ∈ toGLSubgroup d ↔  
    (x : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ) ∈ LorentzGroup d
```

**lemma coe\_toGLSubgroup**[\[source\]](#)

```
lemma coe_toGLSubgroup :  
  (toGLSubgroup d : Set (GL (Fin 1 ⊕ Fin d) ℝ)) =  
    Units.val-1 (LorentzGroup d : Set (Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ))
```

**def toClosedSubgroup**[\[source\]](#)

The Lorentz group realised as a *\*closed\** subgroup of  $GL(\text{Fin } 1 \oplus \text{Fin } d, \mathbb{R})$ . The closedness is exactly the content of `toGL_isClosedEmbedding`, and is the hypothesis that the (not-yet-formalised) closed subgroup theorem would consume to produce a Lie group structure.

```
def toClosedSubgroup (d : ℕ) : ClosedSubgroup (GL (Fin 1 ⊕ Fin d) ℝ) where
```

**2.71 Physlib.Relativity.LorentzGroup.Restricted.FromBoostRotation**

[Physlib/Relativity/LorentzGroup/Restricted/FromBoostRotation.lean](#) on GitHub.

**theorem exists\_boost\_mul\_rotation**[\[source\]](#)

Every element of the restricted Lorentz group can be written as a product of a generalized boost and a rotation.

```
theorem exists_boost_mul_rotation {d} (Λ : LorentzGroup.restricted d) :  
  ∃ (v : Lorentz.Velocity d) (R : Rotations d),  
    (Λ : LorentzGroup d) = generalizedBoost 0 v * (R : LorentzGroup d)
```

**2.72 Physlib.Relativity.Tensors.Basic**

[Physlib/Relativity/Tensors/Basic.lean](#) on GitHub.

**lemma permT\_injective**[\[source\]](#)*permT is injective.*

```

lemma permT_injective {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
  {σ : Fin m → Fin n} (h : IsReindexing c c1 σ) :
  Function.Injective (permT (S := S) σ h)

```

**2.73 Physlib.Relativity.Tensors.ComplexTensor.Metrics.Basic**

Physlib/Relativity/Tensors/ComplexTensor/Metrics/Basic.lean on GitHub.

**lemma leftMetric\_eq\_leftHandedWeyl\_basis**[\[source\]](#)

```

lemma leftMetric_eq_leftHandedWeyl_basis : εL =
  - fromPairT (LeftHandedWeyl.basis 0 ⊗ₜ[C] LeftHandedWeyl.basis 1)
  + fromPairT (LeftHandedWeyl.basis 1 ⊗ₜ[C] LeftHandedWeyl.basis 0)

```

**lemma dualLeftMetric\_eq\_dualLeftHandedWeyl\_basis**[\[source\]](#)

```

lemma dualLeftMetric_eq_dualLeftHandedWeyl_basis : εL' =
  fromPairT (DualLeftHandedWeyl.basis 0 ⊗ₜ[C] DualLeftHandedWeyl.basis 1)
  - fromPairT (DualLeftHandedWeyl.basis 1 ⊗ₜ[C] DualLeftHandedWeyl.basis 0)

```

**lemma rightMetric\_eq\_rightHandedWeyl\_basis**[\[source\]](#)

```

lemma rightMetric_eq_rightHandedWeyl_basis : εR =
  - fromPairT (RightHandedWeyl.basis 0 ⊗ₜ[C] RightHandedWeyl.basis 1)
  + fromPairT (RightHandedWeyl.basis 1 ⊗ₜ[C] RightHandedWeyl.basis 0)

```

**lemma dualRightMetric\_eq\_dualRightHandedWeyl\_basis**[\[source\]](#)

```

lemma dualRightMetric_eq_dualRightHandedWeyl_basis : εR' =
  fromPairT (DualRightHandedWeyl.basis 0 ⊗ₜ[C] DualRightHandedWeyl.basis 1)
  - fromPairT (DualRightHandedWeyl.basis 1 ⊗ₜ[C] DualRightHandedWeyl.basis 0)

```

**2.74 Physlib.Relativity.Tensors.ComplexTensor.Units.Basic**

Physlib/Relativity/Tensors/ComplexTensor/Units/Basic.lean on GitHub.

**lemma dualLeftLeftUnit\_eq\_tensor\_basis**[\[source\]](#)

```

lemma dualLeftLeftUnit_eq_tensor_basis : δL' =
  ∑ i, Tensor.basis (S := complexLorentzTensor)
    ![Color.downL, Color.upL] (fun | 0 => i | 1 => i)

```

**lemma leftDualLeftUnit\_eq\_tensor\_basis**[\[source\]](#)

```

lemma leftDualLeftUnit_eq_tensor_basis : δL =
  ∑ i, Tensor.basis (S := complexLorentzTensor)
    ![Color.upL, Color.downL] (fun | 0 => i | 1 => i)

```



**lemma dualRightRightUnit\_eq\_tensor\_basis**[\[source\]](#)

```
lemma dualRightRightUnit_eq_tensor_basis : δR' =
  ∑ i, Tensor.basis (S := complexLorentzTensor)
    ![Color.downR, Color.upR] (fun | 0 => i | 1 => i)
```

**lemma rightDualRightUnit\_eq\_tensor\_basis**[\[source\]](#)

```
lemma rightDualRightUnit_eq_tensor_basis : δR =
  ∑ i, Tensor.basis (S := complexLorentzTensor)
    ![Color.upR, Color.downR] (fun | 0 => i | 1 => i)
```

**2.75 Physlib.Relativity.Tensors.Conjugation.Basic**

[Physlib/Relativity/Tensors/Conjugation/Basic.lean](#) on GitHub.

**lemma conjEquiv\_basis\_repr**[\[source\]](#)

```
lemma conjEquiv_basis_repr {c : C} (v : V c) (φ : basisIdx (S.bar c)) :
  (b (S.bar c)).repr (S.conjEquiv v) φ
  = star ((b c).repr v (Equiv.cast (S.barIdx_eq c) φ))
```

**lemma conjEquiv\_equivariant**[\[source\]](#)

```
lemma conjEquiv_equivariant {c : C} (g : G) (v : V c) :
  S.conjEquiv (rep c g v) = rep (S.bar c) g (S.conjEquiv v)
```

**lemma conjT\_apply**[\[source\]](#)

```
lemma conjT_apply {n : ℕ} {c : Fin n → C} (t : S.Tensor c) :
  S.conjT t = ofComponents (fun i => S.bar (c i))
    (fun b => star (componentMap c t (S.componentReindex c b)))
```

**lemma conjT\_basis**[\[source\]](#)

```
lemma conjT_basis {n : ℕ} {c : Fin n → C} (i : ComponentIdx (S := S.toTensorSpecies) c) :
  S.conjT (basis c i) = basis (fun i => S.bar (c i)) ((S.componentReindex c).symm i)
```

**lemma conjT\_pure**[\[source\]](#)

```
lemma conjT_pure {n : ℕ} {c : Fin n → C} (p : Tensor.Pure S.toTensorSpecies c) :
  S.conjT p.toTensor = Pure.toTensor (fun i => S.conjEquiv (p i))
```

**lemma conjT\_equivariant**[\[source\]](#)

```
lemma conjT_equivariant {n : ℕ} {c : Fin n → C} (g : G) (t : S.Tensor c) :
  S.conjT (g • t) = g • S.conjT t
```

**2.76 Physlib.Relativity.Tensors.Contraction.CrossToEnd**

[Physlib/Relativity/Tensors/Contraction/CrossToEnd.lean](#) on GitHub.

**def crossToEnd**[\[source\]](#)

*Slot-addressed cross contraction: contract slot  $i$  of  $t1$  against slot  $j$  of  $t2$ . The survivors are  $t1$ 's slots except  $i$  followed by  $t2$ 's slots except  $j$ , so the output color is a clean  $\text{Fin } (nA + nB) \rightarrow C$  and a chain composes without recasts.*

```

noncomputable def crossToEnd {nA nB : ℕ} {cA : Fin (nA + 1) → C} {cB : Fin (nB + 1) → C}
  (i : Fin (nA + 1)) (j : Fin (nB + 1)) (hc : S.τ (cA i) = cB j) :
  Tensor S cA →l[k] Tensor S cB →l[k]
  Tensor S (Fin.append (cA ∘ i.succAbove) (cB ∘ j.succAbove))

```

**lemma crossToEnd\_two**[\[source\]](#)

*Cross-contracting the last slot of a rank-2 tensor  $A$  with the  $0$ -slot of a rank-2 tensor  $B$  is a plain  $\text{contrT}$  of their product on the middle slots  $1, 2$ , up to a color recast. Discharging the slot arithmetic once lets a  $\text{contrT}$ -level identity lift to  $\text{crossToEnd}$  by one rewrite.*

```

lemma crossToEnd_two {cA cB : Fin 2 → C} (h : S.τ (cA (Fin.last 1)) = cB 0)
  (A : Tensor S cA) (B : Tensor S cB) :
  crossToEnd (Fin.last 1) (0 : Fin 2) h A B
  = permT (id : Fin 2 → Fin 2) (IsReindexing.on_id.mpr (fun i => by fin_cases i <=> rfl))
    (contrT 2 (1 : Fin 4) (2 : Fin 4))
    (by
      refine (by decide, ?_)
      rw [show (1 : Fin 4) = Fin.castAdd 2 (1 : Fin 2) from rfl,
          show (2 : Fin 4) = Fin.natAdd 2 (0 : Fin 2) from rfl,
          Fin.append_left, Fin.append_right]
      exact h)
    (prodT A B)

```

**lemma crossToEnd\_equivariant**[\[source\]](#)

*Cross contraction is  $G$ -equivariant, each of  $\text{prodT}$ ,  $\text{contrT}$  and  $\text{permT}$  being so.*

```

@[simp]
lemma crossToEnd_equivariant {nA nB : ℕ} {cA : Fin (nA + 1) → C} {cB : Fin (nB + 1) → C}
  (i : Fin (nA + 1)) (j : Fin (nB + 1)) (hc : S.τ (cA i) = cB j) (g : G)
  (t1 : Tensor S cA) (t2 : Tensor S cB) :
  crossToEnd i j hc (g • t1) (g • t2) = g • crossToEnd i j hc t1 t2

```

**lemma crossToEnd\_assoc\_rankTwo**[\[source\]](#)

*Rebracket the chain  $A \text{--}(iA \cdot 0)\text{--} B \text{--}(\text{last} \cdot 0)\text{--} C$  for rank-two  $B$  and  $C$ : contracting  $A \cdot B$  first and then  $\cdot C$  equals contracting  $B \cdot C$  first and then  $A \cdot$ , up to the color recast  $\text{permT}$  id.*

*Not full associativity: only  $A$ 's attachment slot  $iA$  is free.  $B$ 's rank two is what makes the statement possible, the second contraction being addressed at  $\text{Fin.last } nA$ , the slot  $B$ 's unique survivor occupies in  $A \cdot B$ . Other slots are reached by conjugating with the  $\text{permT}$  commutators.*

*With  $A := T$ ,  $B := g$ ,  $C := g^{-1}$  this reassociates  $(T \cdot g) \cdot g^{-1}$  to  $T \cdot (g \cdot g^{-1})$ , after which  $g \cdot g^{-1}$  collapses to the unit tensor: the step behind raising then lowering a named index.*

```

lemma crossToEnd_assoc_rankTwo {nA : ℕ}
  {cA : Fin (nA + 1) → C} {cB cC : Fin 2 → C}
  (iA : Fin (nA + 1))
  (hc1 : S.τ (cA iA) = cB 0)
  (hc2 : S.τ (cB (Fin.last 1)) = cC 0)
  (tA : Tensor S cA) (tB : Tensor S cB) (tC : Tensor S cC) :
  crossToEnd (Fin.last nA) 0
    (by
      rw [show (Fin.last nA : Fin (nA + 1)) = Fin.natAdd nA (Fin.last 0) from
        Fin.ext (by simp), Fin.append_right, Function.comp_apply, Fin.succAbove_zero,
        Fin.succ_last]
        exact hc2)
      (crossToEnd iA 0 hc1 tA tB) tC =
  permT id
    (IsReindexing.on_id.mpr fun i => by
      refine Fin.addCases (fun a => ?_) (fun a => ?_) i
      · simp only [Fin.append_left, Function.comp_apply, Fin.succAbove_last,
        show a.castSucc = Fin.castAdd 1 a from rfl, Fin.append_left]
      · simp only [Fin.append_right, Function.comp_apply, Fin.succAbove_zero]
        fin_cases a
        rfl)
    (crossToEnd iA 0
      (by
        rw [show (0 : Fin (1 + 1)) = Fin.castAdd 1 (0 : Fin 1) from rfl, Fin.append_left,
          Function.comp_apply, Fin.succAbove_last, Fin.castSucc_zero]
          exact hc1)
        tA
        (crossToEnd (Fin.last 1) 0 hc2 tB tC))

```

### **lemma crossToEnd\_permT\_left**

[\[source\]](#)

*Left commutator: contracting slot  $i$  of  $\text{permT } \sigma \ t1$  equals contracting slot  $\sigma \ i$  of  $t1$ , then relabelling the survivors by  $\sigma'$ . The square  $h\sigma'$  records how  $\sigma'$  carries the complement of  $i$  to the complement of  $\sigma \ i$ , and implies its bijectivity.*

```

lemma crossToEnd_permT_left {nA nB : ℕ} {cA cA' : Fin (nA + 1) → C}
  {cB : Fin (nB + 1) → C}
  (i : Fin (nA + 1)) (j : Fin (nB + 1))
  (σ : Fin (nA + 1) → Fin (nA + 1)) (σ' : Fin nA → Fin nA)
  (hσ : IsReindexing cA cA' σ)
  (hσ' : (σ i).succAbove ∘ σ' = σ ∘ i.succAbove)
  (hc : S.τ (cA' i) = cB j)
  (t1 : Tensor S cA) (t2 : Tensor S cB) :
  crossToEnd i j hc (permT σ hσ t1) t2 =
  permT (Fin.append (Fin.castAdd nB ∘ σ') (Fin.natAdd nA))
    (IsReindexing.append_congr_left (cB ∘ j.succAbove)
      (IsReindexing.succAbove_of_succAbove_eq i hσ hσ'))
    (crossToEnd (σ i) j (by rw [hσ.2 i]; exact hc) t1 t2)

```

### **lemma crossToEnd\_permT\_right**

[\[source\]](#)

*Right commutator, the mirror of crossToEnd\_permT\_left: contracting slot  $j$  of  $\text{permT } \sigma \ t2$  equals contracting slot  $\sigma \ j$  of  $t2$ , then relabelling the survivors by  $\sigma'$ .*

```

lemma crossToEnd_permT_right {nA nB : ℕ} {cA : Fin (nA + 1) → C}
  {cB cB' : Fin (nB + 1) → C}

```

```

(i : Fin (nA + 1)) (j : Fin (nB + 1))
(σ : Fin (nB + 1) → Fin (nB + 1)) (σ' : Fin nB → Fin nB)
(hσ : IsReindexing cB cB' σ)
(hσ' : (σ j).succAbove ∘ σ' = σ ∘ j.succAbove)
(hc : S.τ (cA i) = cB' j)
(t1 : Tensor S cA) (t2 : Tensor S cB) :
crossToEnd i j hc t1 (permT σ hσ t2) =
  permT (Fin.append (Fin.castAdd nB) (Fin.natAdd nA ∘ σ'))
    (IsReindexing.append_congr_right (cA ∘ i.succAbove)
      (IsReindexing.succAbove_of_succAbove_eq j hσ hσ'))
    (crossToEnd i (σ j) (by rw [hσ.2 j]; exact hc) t1 t2)

```

## 2.77 Physlib.Relativity.Tensors.Contraction.CrossToSlot

[Physlib/Relativity/Tensors/Contraction/CrossToSlot.lean](#) on GitHub.

### lemma crossToSlot\_cycle

[\[source\]](#)

*The survivor color of `crossToEnd i j` against a rank-2 tensor of color `cM` is `c` with slot `i` replaced by the surviving color `cM (j.succAbove 0)`, once the survivors are rotated back by the inverse of the cycle `[i, last]`.*

```

lemma IsReindexing.crossToSlot_cycle {nA : ℕ} {c : Fin (nA + 1) → C} {cM : Fin 2 → C}
(i : Fin (nA + 1)) (j : Fin 2) :
IsReindexing
  (Fin.append (c ∘ i.succAbove) (cM ∘ j.succAbove))
  (Function.update c i (cM (j.succAbove 0)))
  ↑(Fin.cycleIcc i (Fin.last nA)).symm

```

### def crossToSlot

[\[source\]](#)

*Contract slot `i` of `t` against slot `j` of the rank-2 tensor `M`, whose `j`-slot color must equal the dual color `τ (c i)` (propositionally, via `hc`), and rotate `M`'s surviving slot back into position `i`. The result is `t` with slot `i` relabelled to `M`'s surviving color.*

*This is raising and lowering a named index; the metric case is `j = 0, M = metricTensor (τ (c i))`. Because `hc` is propositional, a rank-2 tensor whose slot color matches only up to an equality enters directly, with no transport.*

```

noncomputable def crossToSlot {nA : ℕ} {c : Fin (nA + 1) → C} {cM : Fin 2 → C}
(i : Fin (nA + 1)) (j : Fin 2) (hc : S.τ (c i) = cM j) (M : S.Tensor cM) :
S.Tensor c →l[k] S.Tensor (Function.update c i (cM (j.succAbove 0)))

```

### lemma crossToSlot\_eq\_crossToEnd

[\[source\]](#)

*The bridge between the two conventions: `crossToSlot i j hc M t` is `crossToEnd i j hc t M` with the survivor rotated from the last slot back to slot `i` by the inverse cycle.*

```

lemma crossToSlot_eq_crossToEnd {nA : ℕ} {c : Fin (nA + 1) → C} {cM : Fin 2 → C}
(i : Fin (nA + 1)) (j : Fin 2) (hc : S.τ (c i) = cM j) (M : S.Tensor cM) (t : S.Tensor c) :
crossToSlot i j hc M t =
  permT ↑(Fin.cycleIcc i (Fin.last nA)).symm (IsReindexing.crossToSlot_cycle i j)
    (crossToEnd i j hc t M)

```

**def crossToSlotInv**[\[source\]](#)

*Contract slot  $i$  of a tensor whose color there is  $d$  against  $M'$ , then absorb the color cast the two `Function.update`s generate, landing back on  $c$ . This is the returning half of a raise-then-lower round trip; absorbing the cast here is what keeps the round trip cast-free at both ends.*

```
noncomputable def crossToSlotInv {nA : ℕ} {c : Fin (nA + 1) → C} {b d e : C} (i : Fin (nA + 1))
  (he : c i = e) (hb : S.τ d = b) (M' : S.Tensor ![b, e]) :
  S.Tensor (Function.update c i d) →l[k] S.Tensor c
```

**lemma crossToSlot\_permT\_right\_id**[\[source\]](#)

*An identity reindexing of the rank-2 tensor becomes the corresponding identity reindexing of the contracted output: the `crossToSlot`-level case of `crossToEnd_permT_right` where the rank-2 tensor's colors are only propositionally recast.*

```
lemma crossToSlot_permT_right_id {nA : ℕ} {c : Fin (nA + 1) → C} {cM cM' : Fin 2 → C}
  (i : Fin (nA + 1)) (j : Fin 2) (hc : S.τ (c i) = cM' j)
  (M : Tensor S cM) (hM : IsReindexing cM cM' (id : Fin 2 → Fin 2)) (t : Tensor S c) :
  crossToSlot i j hc (permT (id : Fin 2 → Fin 2) hM M) t =
  permT (id : Fin (nA + 1) → Fin (nA + 1))
    (IsReindexing.on_id.mpr (fun a => by
      by_cases ha : a = i
      · subst ha
        simpa using hM.2 (j.succAbove 0)
      · simp [Function.update_of_ne ha]))
    (crossToSlot i j (hc.trans (hM.2 j).symm) M t)
```

**lemma crossToSlot\_equivariant**[\[source\]](#)

*Cross contraction into a slot is  $G$ -equivariant, both constituents (`crossToEnd`, `permT`) being so.*

```
@[simp]
lemma crossToSlot_equivariant {nA : ℕ} {c : Fin (nA + 1) → C} {cM : Fin 2 → C}
  (i : Fin (nA + 1)) (j : Fin 2) (hc : S.τ (c i) = cM j) (g : G)
  (M : Tensor S cM) (t : Tensor S c) :
  crossToSlot i j hc (g • M) (g • t) = g • crossToSlot i j hc M t
```

**2.78 Physlib.Relativity.Tensors.Contraction.Products**

[Physlib/Relativity/Tensors/Contraction/Products.lean](#) on GitHub.

**lemma prodT\_contrT\_fst**[\[source\]](#)

*A contraction internal to the left factor commutes past the outer product with a right spectator. The mirror of `prodT_contrT_snd`, obtained from it by `prodT_swap`, so the right-hand side contracts the swapped product `prodT t1 t` and carries a block swap.*

```
lemma prodT_contrT_fst {n n1 : ℕ} {c : Fin (n + 1 + 1) → C}
  {c1 : Fin n1 → C}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j ∧ S.τ (c i) = c j)
  (t : Tensor S c) (t1 : Tensor S c1) :
  prodT (contrT n i j hij t) t1 =
  permT _ IsReindexing.append_swap
```

```

(permT id (IsReindexing.append_succSuccAbove_natAdd i j) <|
contrT _
  (Fin.natAdd n1 i)
  (Fin.natAdd n1 j)
  (by simp using hij) <|
prodT t1 t)

```

## 2.79 Physlib.Relativity.Tensors.Contraction.Pure

[Physlib/Relativity/Tensors/Contraction/Pure.lean](#) on GitHub.

### lemma permP\_dropPair\_dropPair\_congr

[\[source\]](#)

*Two permP-dropPair-dropPair towers over a common pure tensor agree once their composite slot maps agree pointwise. Comparing two orders of dropping two pairs of slots this way keeps every slot map an opaque variable, so the comparison closes by unification rather than by whnf-reducing the color and index-map composites.*

```

lemma permP_dropPair_dropPair_congr {nP nOut : ℕ}
  {c : Fin (nP + 1 + 1 + 1 + 1) → C} {cOut : Fin nOut → C} (p : Pure S c)
  (aL bL : Fin (nP + 1 + 1 + 1 + 1)) (habL : aL ≠ bL)
  (a2L b2L : Fin (nP + 1 + 1)) (hab2L : a2L ≠ b2L)
  (σ1L : Fin nOut → Fin nP)
  (h1L : IsReindexing ((c ∘ Fin.succSuccAbove aL bL) ∘ Fin.succSuccAbove a2L b2L) cOut σ1L)
  (aR bR : Fin (nP + 1 + 1 + 1 + 1)) (habR : aR ≠ bR)
  (a2R b2R : Fin (nP + 1 + 1)) (hab2R : a2R ≠ b2R)
  (σ1R : Fin nOut → Fin nP)
  (h1R : IsReindexing ((c ∘ Fin.succSuccAbove aR bR) ∘ Fin.succSuccAbove a2R b2R) cOut σ1R)
  (hslot : ∀ m : Fin nOut,
    Fin.succSuccAbove aL bL (Fin.succSuccAbove a2L b2L (σ1L m)) =
    Fin.succSuccAbove aR bR (Fin.succSuccAbove a2R b2R (σ1R m))) :
  permP σ1L h1L (dropPair a2L b2L hab2L (dropPair aL bL habL p)) =
  permP σ1R h1R (dropPair a2R b2R hab2R (dropPair aR bR habR p))

```

### lemma permP\_dropPair\_permP\_congr

[\[source\]](#)

*Two permP-dropPair-permP towers over a common pure tensor agree once their composite slot maps agree pointwise. The one-contraction, inner-relabelled sibling of permP\_dropPair\_dropPair\_congr.*

```

lemma permP_dropPair_permP_congr {nP nOut mL mR : ℕ}
  {c : Fin nP → C} {cL : Fin (mL + 1 + 1) → C} {cR : Fin (mR + 1 + 1) → C}
  {cOut : Fin nOut → C} (p : Pure S c)
  (σ0L : Fin (mL + 1 + 1) → Fin nP) (h0L : IsReindexing c cL σ0L)
  (aL bL : Fin (mL + 1 + 1)) (habL : aL ≠ bL)
  (σ1L : Fin nOut → Fin mL)
  (h1L : IsReindexing (cL ∘ Fin.succSuccAbove aL bL) cOut σ1L)
  (σ0R : Fin (mR + 1 + 1) → Fin nP) (h0R : IsReindexing c cR σ0R)
  (aR bR : Fin (mR + 1 + 1)) (habR : aR ≠ bR)
  (σ1R : Fin nOut → Fin mR)
  (h1R : IsReindexing (cR ∘ Fin.succSuccAbove aR bR) cOut σ1R)
  (hslot : ∀ m : Fin nOut,
    σ0L (Fin.succSuccAbove aL bL (σ1L m)) = σ0R (Fin.succSuccAbove aR bR (σ1R m))) :
  permP σ1L h1L (dropPair aL bL habL (permP σ0L h0L p)) =
  permP σ1R h1R (dropPair aR bR habR (permP σ0R h0R p))

```

**lemma contrPCoeff\_congr**[\[source\]](#)

*contrPCoeff is insensitive to its proof arguments once the two contracted slots agree. Lets a coefficient comparison rewrite the slots without a congr search through the proof fields.*

```
lemma contrPCoeff_congr {n : ℕ} {c : Fin n → C} (p : Pure S c)
  {i i' j j' : Fin n} (hi : i = i') (hj : j = j')
  {hij : i ≠ j ∧ S.τ (c i) = c j} {hij' : i' ≠ j' ∧ S.τ (c i') = c j'} :
  p.contrPCoeff i j hij = p.contrPCoeff i' j' hij'
```

**2.80 Physlib.Relativity.Tensors.Contraction.SuccSuccAbove**

[Physlib/Relativity/Tensors/Contraction/SuccSuccAbove.lean](#) on GitHub.

**lemma succSuccAbove\_castAdd\_natAdd\_apply\_castAdd**[\[source\]](#)

*Reinserting a left-block survivor  $a$  after removing the  $i$ -th slot of the left block and the  $j$ -th slot of the right block of  $\text{Fin } ((nA + 1) + (nB + 1))$ , the removal read at the contracted length  $(nA + nB) + 1 + 1$ . Unlike `succSuccAbove_natAdd_apply_castAdd` the two holes straddle the two blocks, so the statement carries the reshaping `Fin.cast`s.*

```
lemma succSuccAbove_castAdd_natAdd_apply_castAdd {nA nB : ℕ} (i : Fin (nA + 1)) (j : Fin (nB + 1))
  (a : Fin nA) :
  Fin.cast (show (nA + nB) + 1 + 1 = (nA + 1) + (nB + 1) by omega)
    ((Fin.cast (show (nA + 1) + (nB + 1) = (nA + nB) + 1 + 1 by omega)
      (Fin.castAdd (nB + 1) i)).succSuccAbove
      (Fin.cast (show (nA + 1) + (nB + 1) = (nA + nB) + 1 + 1 by omega)
        (Fin.natAdd (nA + 1) j)) (Fin.castAdd nB a))
    = Fin.castAdd (nB + 1) (i.succAbove a)
```

**lemma succSuccAbove\_castAdd\_natAdd\_apply\_natAdd**[\[source\]](#)

*Reinserting a right-block survivor, the mirror of `Fin.succSuccAbove_castAdd_natAdd_apply_castAdd`.*

```
lemma succSuccAbove_castAdd_natAdd_apply_natAdd {nA nB : ℕ} (i : Fin (nA + 1)) (j : Fin (nB + 1))
  (a : Fin nB) :
  Fin.cast (show (nA + nB) + 1 + 1 = (nA + 1) + (nB + 1) by omega)
    ((Fin.cast (show (nA + 1) + (nB + 1) = (nA + nB) + 1 + 1 by omega)
      (Fin.castAdd (nB + 1) i)).succSuccAbove
      (Fin.cast (show (nA + 1) + (nB + 1) = (nA + nB) + 1 + 1 by omega)
        (Fin.natAdd (nA + 1) j)) (Fin.natAdd nA a))
    = Fin.natAdd (nA + 1) (j.succAbove a)
```

**2.81 Physlib.Relativity.Tensors.Contraction.UnitTensorContraction**

[Physlib/Relativity/Tensors/Contraction/UnitTensorContraction.lean](#) on GitHub.

**lemma crossToEnd\_prodT\_unitTensor**[\[source\]](#)

*Cross-contracting the last slot of the product  $E \otimes B$  (with  $B$  rank one) against the unit tensor for that slot's color returns  $E \otimes B$  unchanged, the contracted slot carried to the end. The rank-one spectator case that seeds `crossToEnd_unitTensor_slot`.*

```

private lemma crossToEnd_prodT_unitTensor {nV : ℕ} {cE : Fin nV → C} {cB : C}
  (E : Tensor S cE) (B : Tensor S ![cB]) :
  crossToEnd (Fin.last nV) (0 : Fin 2)
    (by rw [show (Fin.last nV : Fin (nV + 1)) = Fin.natAdd nV 0 from Fin.ext (by simp)]);
  simp :
    S.τ ((Fin.append cE ![cB]) (Fin.last nV)) = ![S.τ cB, cB] 0)
    (prodT E B) (unitTensor cB) =
  permT id (IsReindexing.on_id.mpr (fun i => by
    refine Fin.addCases (fun j => ?_) (fun j => ?_) i
    · simp only [Fin.append_left, Function.comp_apply, Fin.succAbove_last]
      exact (Fin.append_left cE ![cB] j).symm
    · simp only [Fin.append_right, Function.comp_apply, Fin.succAbove_zero]
      fin_cases j
      rfl)) (prodT E B)

```

### lemma crossToEnd\_unitTensor\_slot

[\[source\]](#)

The boundary case of `crossToEnd_unitTensor`, proved by spectator decomposition along the last slot (`eq_sum_evalT`). The last slot is threaded as a variable `i` with `hilast : i = last nA` so that `crossToEnd_unitTensor` can apply it at the image of `i` under its transposition without a dependent rewrite of the slot index.

```

private lemma crossToEnd_unitTensor_slot {nA : ℕ} {cA : Fin (nA + 1) → C} {c : C}
  {i : Fin (nA + 1)} (hilast : i = Fin.last nA) (hc : cA i = c) (t1 : Tensor S cA) :
  crossToEnd i (0 : Fin 2) (by simp [hc] : S.τ (cA i) = ![S.τ c, c] 0) t1 (unitTensor c) =
  permT (Fin.append i.succAbove (fun _ : Fin 1 => i)) (IsReindexing.move_last i hc) t1

```

### lemma crossToEnd\_unitTensor

[\[source\]](#)

Contracting slot `i` of `t1` against the unit tensor for that slot's color returns `t1`, the contracted slot relabelled by `move_last i`. The unit tensor acts as an identity for `crossToEnd` at any named slot.

```

lemma crossToEnd_unitTensor {nA : ℕ} {cA : Fin (nA + 1) → C} {c : C}
  (i : Fin (nA + 1)) (hc : cA i = c) (t1 : Tensor S cA) :
  crossToEnd i (0 : Fin 2) (by simp [hc] : S.τ (cA i) = ![S.τ c, c] 0) t1 (unitTensor c) =
  permT (Fin.append i.succAbove (fun _ : Fin 1 => i)) (IsReindexing.move_last i hc) t1

```

### lemma unitTensor\_pair

[\[source\]](#)

The color list `crossToEnd` produces from a matched rank-2 pair  $M : ![a, d], M' : ![b, e]$  is the color list  $![S.τ e, e]$  of `unitTensor e`, via the identity slot map: the reindexing carried by the collapse hypothesis  $M \cdot M' = \delta$  of the round trips below. Only the surviving colors `a` and `e` enter, and `a` equals `S.τ e` only propositionally, so a pair whose colors match propositionally enters with no transport.

```

lemma IsReindexing.unitTensor_pair {a b d e : C} (h : S.τ e = a) :
  IsReindexing (![S.τ e, e] : Fin 2 → C)
    (Fin.append ((![a, d] : Fin 2 → C) ∘ (Fin.last 1).succAbove)
      ((![b, e] : Fin 2 → C) ∘ (0 : Fin 2).succAbove))
    (id : Fin 2 → Fin 2)

```

### lemma crossToEnd\_round\_trip\_of\_unit\_slot

[\[source\]](#)



*Round trip of two contractions against a matched pair, in result-to-end form. Contracting slot  $i$  of  $t$  against  $M_1$ , then contracting the surviving  $M_1$ -index (deposited at slot  $j$ ) against  $M_2$ , returns  $t$  up to the cycle that moves the round-tripped slot from the end back to  $i$ , whenever  $M_1$  and  $M_2$  collapse to the unit tensor. The last slot is threaded as a variable  $j$  with  $hj : j = \text{last } nA$  so that a caller can apply it at a propositionally-last slot without a dependent rewrite of the slot index.*

```

lemma crossToEnd_round_trip_of_unit_slot {nA : ℕ} {c : Fin (nA + 1) → C} {d : C}
  (i j : Fin (nA + 1)) (hj : j = Fin.last nA)
  (M1 : Tensor S ![S.τ (c i), d]) (M2 : Tensor S ![S.τ d, c i])
  (hM : crossToEnd (Fin.last 1) (0 : Fin 2) rfl M1 M2 =
    permT (id : Fin 2 → Fin 2) (IsReindexing.unitTensor_pair rfl)
      (unitTensor (S := S) (c i)))
  (t : Tensor S c) :
  crossToEnd j (0 : Fin 2)
    (by subst hj
      rw [show (Fin.last nA : Fin (nA + 1)) = Fin.natAdd nA (0 : Fin 1) from
        Fin.ext (by simp), Fin.append_right]; rfl)
    (crossToEnd i (0 : Fin 2) rfl t M1) M2 =
  permT (Fin.append i.succAbove (fun _ : Fin 1 => i))
    (by
      subst hj
      exact (by
        rw [Fin.append_succAbove_const_eq_cycleIcc i]
        exact (Fin.cycleIcc i (Fin.last nA)).bijective,
        fun x => by
          refine Fin.addCases (fun a => ?_) (fun a => ?_) x
          · simp only [Function.comp_apply, Fin.append_left, Fin.succAbove_last,
            show (Fin.castSucc a : Fin (nA + 1)) = Fin.castAdd 1 a from rfl]
          · fin_cases a
            simp [Fin.append_right, Fin.succAbove_zero]))
    t

```

**lemma crossToSlot\_round\_trip\_of\_unit**

[\[source\]](#)

*Round trip for crossToSlot at an arbitrary slot, with the colors in their composite form. The general-color statement crossToSlot\_raise\_lower\_round\_trip is obtained from this by substituting its three color equalities.*

```

private lemma crossToSlot_round_trip_of_unit {nA : ℕ} {c : Fin (nA + 1) → C} {d : C}
  (i : Fin (nA + 1))
  (M1 : Tensor S ![S.τ (c i), d]) (M2 : Tensor S ![S.τ d, c i])
  (hM : crossToEnd (Fin.last 1) (0 : Fin 2) rfl M1 M2 =
    permT (id : Fin 2 → Fin 2) (IsReindexing.unitTensor_pair rfl)
      (unitTensor (S := S) (c i)))
  (t : Tensor S c) :
  crossToSlot (S := S) i (0 : Fin 2)
    (by rw [Function.update_self] : S.τ (Function.update c i d i) = S.τ d) M2
    (crossToSlot (S := S) i (0 : Fin 2) rfl M1 t) =
  permT (id : Fin (nA + 1) → Fin (nA + 1)) (IsReindexing.update_update_of_eq i rfl) t

```

**lemma crossToSlot\_raise\_lower\_round\_trip**

[\[source\]](#)

*Raising then lowering a named index, with every color threaded propositionally. For a rank-2 metric  $M : ![a, d]$  and an inverse  $M' : ![b, e]$  collapsing to the unit tensor ( $M \cdot M' = \delta$ ), at any rank and any slot  $i$ : contracting slot  $i$  with  $M$ , then contracting*

the result with  $M'$ , is the identity up to the `Function.update` color reindex.

No color is pinned to a composite.  $ha : \tau (c\ i) = a$  places the metric's contracted slot,  $hb : \tau d = b$  the inverse's, and  $he : c\ i = e$  the surviving one, so a caller whose metric and inverse sit at literal colors supplies them directly, with no transporting `permT` on either factor. Lowering then raising is this same statement with  $M$  and  $M'$  exchanged and the reversed collapse  $M' \cdot M = \delta$ ; neither collapse follows from the other here.

```
lemma crossToSlot_raise_lower_round_trip {nA : ℕ} {c : Fin (nA + 1) → C} {a b d e : C}
  (i : Fin (nA + 1)) (he : c i = e) (ha : S.τ (c i) = a) (hb : S.τ d = b)
  (M : Tensor S ![a, d]) (M' : Tensor S ![b, e])
  (hM : crossToEnd (Fin.last 1) (0 : Fin 2) hb M M' =
    permT (id : Fin 2 → Fin 2)
      (IsReindexing.unitTensor_pair ((congrArg S.τ he.symm).trans ha))
      (unitTensor (S := S) e))
  (t : Tensor S c) :
  crossToSlot (S := S) i (0 : Fin 2)
    (by rw [Function.update_self]; exact hb : S.τ (Function.update c i d i) = b) M'
    (crossToSlot (S := S) i (0 : Fin 2) ha M t) =
    permT (id : Fin (nA + 1) → Fin (nA + 1)) (IsReindexing.update_update_of_eq i he) t
```

**lemma crossToSlotInv\_crossToSlot**

[\[source\]](#)

Lowering undoes raising: `crossToSlot_raise_lower_round_trip` with the color cast absorbed into `crossToSlotInv`.

```
lemma crossToSlotInv_crossToSlot (t : Tensor S c) :
  crossToSlotInv i he hb M' (crossToSlot i (0 : Fin 2) ha M t) = t
```

**lemma crossToSlot\_crossToSlotInv**

[\[source\]](#)

Raising undoes lowering. The reversed collapse  $M' \cdot M = \delta$  is the round trip at the color list `Function.update c i d` with the pair exchanged, which gives the lowering half a left inverse and hence injectivity; that upgrades `crossToSlotInv_crossToSlot` to a two-sided inverse.

```
lemma crossToSlot_crossToSlotInv (t : Tensor S (Function.update c i d)) :
  crossToSlot i (0 : Fin 2) ha M (crossToSlotInv i he hb M' t) = t
```

**def crossToSlotEquiv**

[\[source\]](#)

Raising and lowering a named index against a mutually inverse pair, as a linear equivalence: the two color assignments  $c$  and `Function.update c i d` carry the same tensors.  $M$  and  $M'$  are matched only propositionally and  $d$  is unrelated to  $c\ i$ , so this covers a metric pairing two unrelated colors as well as a metric and its inverse at  $S.\tau (c\ i)$ .

```
noncomputable def crossToSlotEquiv : Tensor S c ≈l[k] Tensor S (Function.update c i d)
```

## 2.82 Physlib.Relativity.Tensors.Dual

[Physlib/Relativity/Tensors/Dual.lean](#) on GitHub.

**def toDualMapAtIndex**[\[source\]](#)

*The linear map between  $S.\text{Tensor } c$  and  $S.\text{Tensor } (\text{Function.update } c \ i \ (S.\tau \ (c \ i)))$  formed by contracting the index  $i$  with the metric tensor.*

```
noncomputable def toDualMapAtIndex : {n : ℕ} → {c : Fin n → C} → (i : Fin n) →
  S.Tensor c →l[k] S.Tensor (Function.update c i (S.τ (c i)))
| 0, _, i => i.elim0
| _ + 1, c, i => crossToSlot i (0 : Fin 2) rfl (metricTensor (S := S) (S.τ (c i)))
```

**lemma crossToEnd\_dual\_metricTensor\_metricTensor**[\[source\]](#)

*The metric tensor at  $S.\tau \ c$  contracted with the metric tensor at  $c$  is the unit tensor at  $c$ .*

```
lemma crossToEnd_dual_metricTensor_metricTensor {c : C} :
  crossToEnd (Fin.last 1) (0 : Fin 2) (S.τ_τ_apply c) (metricTensor (S := S) (S.τ c))
    (metricTensor (S := S) c) =
  permT (id : Fin 2 → Fin 2) (IsReindexing.unitTensor_pair rfl) (unitTensor (S := S) c)
```

**lemma crossToEnd\_metricTensor\_metricTensor\_eq\_dual\_unit**[\[source\]](#)

*The metric tensor at  $c$  contracted with the metric tensor at  $S.\tau \ c$  is the unit tensor at  $S.\tau \ c$ .*

```
lemma crossToEnd_metricTensor_metricTensor_eq_dual_unit {c : C} :
  crossToEnd (Fin.last 1) (0 : Fin 2) rfl (metricTensor (S := S) c)
    (metricTensor (S := S) (S.τ c)) =
  permT (id : Fin 2 → Fin 2) (IsReindexing.unitTensor_pair (S.τ_τ_apply c))
    (unitTensor (S := S) (S.τ c))
```

**lemma toDualMapAtIndex\_toDualMapAtIndex**[\[source\]](#)

*Dualising the index  $i$  twice returns the original tensor, up to the reindexing of the colors.*

```
lemma toDualMapAtIndex_toDualMapAtIndex {n : ℕ} {c : Fin n → C}
  (i : Fin n) (t : S.Tensor c) :
  toDualMapAtIndex (S := S) i (toDualMapAtIndex (S := S) i t) =
  permT (id : Fin n → Fin n)
    (IsReindexing.update_update_of_eq i (by simp [τ_τ_apply])) t
```

**def fromDualMapAtIndex**[\[source\]](#)

*The linear map between  $S.\text{Tensor } (\text{Function.update } c \ i \ (S.\tau \ (c \ i)))$  and  $S.\text{Tensor } c$  formed by contracting the index  $i$  with the metric tensor. It is the inverse of  $\text{toDualMapAtIndex}$ .*

```
noncomputable def fromDualMapAtIndex : {n : ℕ} → {c : Fin n → C} → (i : Fin n) →
  S.Tensor (Function.update c i (S.τ (c i))) →l[k] S.Tensor c
| 0, _, i => i.elim0
| _ + 1, c, i => crossToSlotInv i rfl (S.τ_τ_apply (c i)) (metricTensor (S := S) (c i))
```

**lemma fromDualMapAtIndex\_toDualMapAtIndex**[\[source\]](#)

```
@[simp]
lemma fromDualMapAtIndex_toDualMapAtIndex {n : ℕ} {c : Fin n → C} (i : Fin n) (t : S.Tensor c)
:
  fromDualMapAtIndex (S := S) i (toDualMapAtIndex (S := S) i t) = t
```

### lemma toDualMapAtIndex\_fromDualMapAtIndex

[\[source\]](#)

```
@[simp]
lemma toDualMapAtIndex_fromDualMapAtIndex {n : ℕ} {c : Fin n → C} (i : Fin n)
(t : S.Tensor (Function.update c i (S.τ (c i)))) :
  toDualMapAtIndex (S := S) i (fromDualMapAtIndex (S := S) i t) = t
```

### lemma toDualMapAtIndex\_equivariant

[\[source\]](#)

*Dualising the index i commutes with the action of G.*

```
@[simp]
lemma toDualMapAtIndex_equivariant {n : ℕ} {c : Fin n → C} (i : Fin n) (g : G) (t : S.Tensor c)
:
  toDualMapAtIndex (S := S) i (g • t) = g • toDualMapAtIndex (S := S) i t
```

### def toDualAtIndex

[\[source\]](#)

*The linear equivalence between  $S.Tensor\ c$  and  $S.Tensor\ (Function.update\ c\ i\ (S.τ\ (c\ i)))$  formed by contracting the index  $i$  with the metric tensor.*

```
noncomputable def toDualAtIndex : {n : ℕ} → {c : Fin n → C} → (i : Fin n) →
  S.Tensor c ≈[k] S.Tensor (Function.update c i (S.τ (c i)))
| 0, _, i => i.elim0
| _ + 1, c, i =>
  crossToSlotEquiv i rfl rfl (S.τ_τ_apply (c i)) (metricTensor (S := S) (S.τ (c i)))
  (metricTensor (S := S) (c i)) crossToEnd_dual_metricTensor_metricTensor
  crossToEnd_metricTensor_metricTensor_eq_dual_unit
```

### lemma toDualAtIndex\_apply

[\[source\]](#)

```
@[simp]
lemma toDualAtIndex_apply {n : ℕ} {c : Fin n → C} (i : Fin n) (t : S.Tensor c) :
  toDualAtIndex (S := S) i t = toDualMapAtIndex (S := S) i t
```

### lemma toDualAtIndex\_symm\_apply

[\[source\]](#)

```
@[simp]
lemma toDualAtIndex_symm_apply {n : ℕ} {c : Fin n → C} (i : Fin n)
(t : S.Tensor (Function.update c i (S.τ (c i)))) :
  (toDualAtIndex (S := S) i).symm t = fromDualMapAtIndex (S := S) i t
```

## 2.83 Physlib.Relativity.Tensors.Elab

[Physlib/Relativity/Tensors/Elab.lean](#) on GitHub.

**def indexExprIsJiggle**[\[source\]](#)

*Bool which is true if an index is a jiggle  $\tau(i)$ .*

```
def indexExprIsJiggle (stx : Syntax) : Bool
```

**def indexRemoveTau**[\[source\]](#)

*Removes a  $\tau(\cdot)$  wrapper from an index, leaving any other index unchanged. Pure, so it can be used to compare indices up to  $\tau$  (e.g. when counting contractions in `withoutContrEval`).*

```
def indexRemoveTau (stx : TSyntax `indexExpr) : TSyntax `indexExpr
```

**def getJigglePos**[\[source\]](#)

*Returns the positions of indices which are "jiggled", i.e., of the form  $\tau(\mu)$ , these are the indices which are to be raised or lowered.*

```
def getJigglePos (ind : List (TSyntax `indexExpr)) : TermElabM (List ℕ)
```

**abbrev TensorExpressionOperator**[\[source\]](#)

*A Tensor expression operator is a map which takes in a pair consisting of a list of indices and a term and outputs a list of indices and a term, after applying a certain operation.*

```
abbrev TensorExpressionOperator
```

**def create**[\[source\]](#)

*The creation of a tensor from a syntax tree.*

```
def TensorExpressionOperator.create (stx : Syntax) :  
  TermElabM (List (TSyntax `indexExpr) × Term)
```

**def jiggle**[\[source\]](#)

*The tensor expression operator which rises or lowers the indices of a tensor based on indices with  $\tau$ -syntax.*

```
def TensorExpressionOperator.jiggle : TensorExpressionOperator
```

**def eval**[\[source\]](#)

*The tensor expression operator which evaluates indices.*

```
def TensorExpressionOperator.eval : TensorExpressionOperator
```

**def contr**[\[source\]](#)

*The tensor expression operator which contracts indices.*

```
def TensorExpressionOperator.contr : TensorExpressionOperator
```

**def prod**[\[source\]](#)

*The tensor expression operator which takes the product of two tensors.*

```
def TensorExpressionOperator.prod : List (TSyntax `indexExpr) × Term →
  List (TSyntax `indexExpr) × Term →
  TermElabM (List (TSyntax `indexExpr) × Term)
```

**def neg**[\[source\]](#)

*The tensor expression operator which negates a tensor.*

```
def TensorExpressionOperator.neg : TensorExpressionOperator
```

**def smul**[\[source\]](#)

*The tensor expression operator which multiplies a tensor by a scalar.*

```
def TensorExpressionOperator.smul (c : Term) : TensorExpressionOperator
```

**def action**[\[source\]](#)

*The tensor expression operator which acts on a tensor by a group element.*

```
def TensorExpressionOperator.action (c : Term) : TensorExpressionOperator
```

**def colorsDefEq**[\[source\]](#)

*Whether T1 and T2 elaborate to tensors of definitionally equal colour (equal type).  
Used to decide whether an identity reindexing between them may be dropped: the  
bare, un-permT'd term is well typed exactly when the two colours agree by rfl. The  
check elaborates speculatively and restores the elaborator state afterwards.*

```
def colorsDefEq (T1 T2 : Term) : TermElabM Bool
```

**def permWrap**[\[source\]](#)

*Wraps T in the reindexing permT ![lPerm] IsReindexing.auto.*

```
def permWrap (lPerm : List ℕ) (T : Term) : TermElabM Term
```

**def add**[\[source\]](#)

*The tensor expression operator which adds two tensors.*

```
def TensorExpressionOperator.add : List (TSyntax `indexExpr) × Term →
  List (TSyntax `indexExpr) × Term →
  TermElabM (List (TSyntax `indexExpr) × Term)
```

**def equal**[\[source\]](#)

*The tensor expression operator which equates two tensors.*

```
def TensorExpressionOperator.equal : List (TSyntax `indexExpr) × Term →
  List (TSyntax `indexExpr) × Term →
  TermElabM (List (TSyntax `indexExpr) × Term)
```

### def stripToTensorSelf

[\[source\]](#)

*Removes redundant `Tensorial.toTensor` coercions from an elaborated tensor expression: every subterm `Tensorial.toTensor t` whose `Tensorial` instance is the canonical instance on `S.Tensor c` (`Tensorial.self`) is replaced by `t` itself, since there the coercion is the identity. Coercions coming from genuine tensorial instances (e.g. on Lorentz vectors) carry real information and are left untouched.*

```
def stripToTensorSelf (e : Expr) : Expr
```

## 2.84 Physlib.Relativity.Tensors.Evaluation

[Physlib/Relativity/Tensors/Evaluation.lean](#) on GitHub.

### lemma evalT\_permT

[\[source\]](#)

*Commuting evaluation with permutations.*

```
lemma evalT_permT {n m : ℕ} {c : Fin (n + 1) → C} {c' : Fin (m + 1) → C}
  {σ : Fin (n + 1) → Fin (m + 1)}
  (h : IsReindexing c' c σ) (i : Fin (n + 1)) (x : basisIdx (c i)) (t : Tensor S c') :
  evalT i x (permT _ h t) = permT _ (h.succAbove i)
    (evalT (σ i) (basisIdxCongr (by simp [h.2]) x) t)
```

### lemma contrT\_evalT

[\[source\]](#)

*Commuting evaluation with contraction. Evaluating index `k` and then contracting the pair `i j` equals contracting the corresponding pair `k.succAbove i`, `k.succAbove j` and then evaluating the residual index, up to the identity reindexing `IsReindexing.succAbove_succSuccAbove_comm`.*

```
lemma contrT_evalT {n : ℕ} {c : Fin (n + 1 + 1 + 1) → C}
  (k : Fin (n + 1 + 1 + 1)) (i j : Fin (n + 1 + 1)) (φ : basisIdx (c k))
  (hij : i ≠ j ∧ S.τ ((c ∘ k.succAbove) i) = (c ∘ k.succAbove) j) (t : Tensor S c) :
  contrT n i j hij (evalT k φ t) =
  permT id (.succAbove_succSuccAbove_comm k i j hij.1)
    (evalT (Fin.predPredAbove (k.succAbove i) (k.succAbove j) (by simp [hij.1]) k (by simp))
      (basisIdxCongr (by simp) φ)
      (contrT (n + 1) (k.succAbove i) (k.succAbove j) (by simp [hij.1], hij.2) t))
```

### lemma evalT\_prodT\_right

[\[source\]](#)

*Evaluating an index in the right factor of a tensor product commutes with forming the product, up to the identity reindexing which identifies the two ways of removing that index from the appended color list.*

```
lemma evalT_prodT_right {n n1 : ℕ} {c : Fin n → C} {c1 : Fin (n1 + 1) → C}
  (i : Fin (n1 + 1)) (x : basisIdx (c1 i)) (t : Tensor S c) (t1 : Tensor S c1) :
  prodT t (evalT i x t1) =
  permT id (IsReindexing.append_succAbove_natAdd (n := n) (n1 := n1) i)
    (evalT (Fin.natAdd (m := n1 + 1) n i) (basisIdxCongr (by simp) x) (prodT t t1))
```

**lemma ext\_of\_evalT**[\[source\]](#)

```
lemma ext_of_evalT {n : ℕ} {c : Fin (n + 1) → C} (t1 t2 : Tensor S c)
  (h : ∀ i φ, evalT i φ t1 = evalT i φ t2) :
  t1 = t2
```

**lemma ext\_of\_evalT\_index**[\[source\]](#)

```
lemma ext_of_evalT_index {n : ℕ} {c : Fin (n + 1) → C} {t1 t2 : Tensor S c}
  (i : Fin (n + 1)) (h : ∀ φ, evalT i φ t1 = evalT i φ t2) :
  t1 = t2
```

**2.85 Physlib.Relativity.Tensors.LeviCivita.Basic**

[Physlib/Relativity/Tensors/LeviCivita/Basic.lean](#) on GitHub.

**def euclidLeviCivita**[\[source\]](#)

*The Euclidean Levi-Civita symbol  $\epsilon_{\{ijkl\}}$  in dimension 4.*

```
def _root_.euclidLeviCivita (g : Fin 4 → Fin 4) : ℝ
```

**lemma leviCivita\_contract\_three**[\[source\]](#)

```
@[sorryful]
lemma leviCivita_contract_three : {ε4 | μ ν ρ σ ⊗ ε4 | τ(μ) τ(ν) τ(ρ) τ(σ) =
  (-6) • unitTensor (S := realLorentzTensor) Color.down | σ τ }T
```

**lemma leviCivita\_contract\_self**[\[source\]](#)

```
@[sorryful, nolint checkType]
lemma leviCivita_contract_self :
  {ε4 | μ ν ρ σ ⊗ ε4 | τ(μ) τ(ν) τ(ρ) τ(σ)}T.toField = - 24
```

**2.86 Physlib.Relativity.Tensors.LeviCivita.Contractions**

[Physlib/Relativity/Tensors/LeviCivita/Contractions.lean](#) on GitHub.

**lemma euclidLeviCivita\_symbol\_contract\_zero**[\[source\]](#)

*\*\*Full Euclidean Levi-Civita contraction\*\*  $\sum_b (\epsilon_4)_b \cdot (\epsilon_4)_b = 24$  at the symbol level: summing the square of every standard-basis component of  $\epsilon_4$  over all four  $\text{Fin } 4$  index slots, paired naively (no metric), counts the  $4! = 24$  permutations. The Lorentz contraction  $\epsilon^{\{\mu\nu\rho\sigma\}} \epsilon_{\{\mu\nu\rho\sigma\}}$  lowers one factor with  $\eta$  and equals  $-24$  instead.*

```
lemma euclidLeviCivita_symbol_contract_zero :
  ∑ g : (Fin 4 → Fin 4), euclidLeviCivita g * euclidLeviCivita g = 24
```

**lemma euclidLeviCivita\_symbol\_contract\_one**[\[source\]](#)

*\*\*Triple Euclidean Levi-Civita contraction\*\*  $\sum_h (\epsilon_4)_{\{a,h\}} \cdot (\epsilon_4)_{\{b,h\}} = 6 \cdot \delta[a,b]$  at the symbol level: contracting three of the four  $\text{Fin } 4$  component slots of  $\epsilon_4$  with the naive Kronecker pairing leaves one free pair  $a, b$  and the factor  $3! = 6$ . The Lorentz form carries an extra  $\det \eta = -1$ .*



```

lemma euclidLeviCivita_symbol_contract_one (a b : Fin 4) :
  ∑ h : Fin 3 → Fin 4, euclidLeviCivita (Fin.cons a h) * euclidLeviCivita (Fin.cons b h)
  = 6 * ((kroneckerDelta a b : ℕ) : ℝ)

```

**lemma euclidLeviCivita\_symbol\_contract\_two**

[\[source\]](#)

*\*\*Double Euclidean Levi-Civita contraction\*\*  $\sum_h (\varepsilon 4)_{\{r,s,h\}} \cdot (\varepsilon 4)_{\{t,w,h\}} = 2 \cdot (\delta[r,t] \cdot \delta[s,w] - \delta[r,w] \cdot \delta[s,t])$  at the symbol level: contracting two of the four Fin 4 component slots of  $\varepsilon 4$  with the naïve Kronecker pairing leaves two free pairs and the factor  $2! = 2$ . The Lorentz form carries an extra  $\det \eta = -1$ .*

```

lemma euclidLeviCivita_symbol_contract_two (r s t w : Fin 4) :
  ∑ h : Fin 2 → Fin 4, euclidLeviCivita (Fin.cons r (Fin.cons s h))
    * euclidLeviCivita (Fin.cons t (Fin.cons w h))
  = 2 * (((kroneckerDelta r t : ℕ) : ℝ) * ((kroneckerDelta s w : ℕ) : ℝ)
    - ((kroneckerDelta r w : ℕ) : ℝ) * ((kroneckerDelta s t : ℕ) : ℝ))

```

## 2.87 Physlib.Relativity.Tensors.RealTensor.Representation.Contraction

[Physlib/Relativity/Tensors/RealTensor/Representation/Contraction.lean](#) on GitHub.

**def Vector.contract**

[\[source\]](#)

*The intertwining map defining the contraction of a contravariant Lorentz vector with a covariant Lorentz vector.*

```

def Vector.contract : (Vector.rep.tprod CoVector.rep).IntertwiningMap
  (Representation.trivial ℝ (LorentzGroup d) ℝ) where

```

**def CoVector.contract**

[\[source\]](#)

*The intertwining map defining the contraction of a covariant Lorentz vector with a contravariant Lorentz vector.*

```

def CoVector.contract : (CoVector.rep.tprod Vector.rep).IntertwiningMap
  (Representation.trivial ℝ (LorentzGroup d) ℝ) where

```

**lemma Vector.contract\_tmul**

[\[source\]](#)

```

lemma Vector.contract_tmul (φ : Vector d) (ψ : CoVector d) :
  Vector.contract (φ ⊗ₜ ψ) = ∑ i, φ i * ψ i

```

**lemma CoVector.contract\_tmul**

[\[source\]](#)

```

lemma CoVector.contract_tmul (φ : CoVector d) (ψ : Vector d) :
  CoVector.contract (φ ⊗ₜ ψ) = ∑ i, φ i * ψ i

```

**lemma Vector.contract\_basis\_left**

[\[source\]](#)

```

lemma Vector.contract_basis_left (μ : Fin 1 ⊗ Fin d) (ψ : CoVector d) :
  Vector.contract (basis μ ⊗ₜ ψ) = ψ μ

```

**lemma CoVector.contract\_basis\_left**[\[source\]](#)

```
lemma CoVector.contract_basis_left (μ : Fin 1 ⊗ Fin d) (φ : Vector d) :
  CoVector.contract (basis μ ⊗ₜ φ) = φ μ
```

**lemma Vector.contract\_basis\_right**[\[source\]](#)

```
lemma Vector.contract_basis_right (φ : Vector d) (μ : Fin 1 ⊗ Fin d) :
  Vector.contract (φ ⊗ₜ basis μ) = φ μ
```

**lemma CoVector.contract\_basis\_right**[\[source\]](#)

```
lemma CoVector.contract_basis_right (ψ : CoVector d) (μ : Fin 1 ⊗ Fin d) :
  CoVector.contract (ψ ⊗ₜ basis μ) = ψ μ
```

**lemma contract\_eq\_coVector\_contract**[\[source\]](#)

```
lemma Vector.contract_eq_coVector_contract (φ : Vector d) (ψ : CoVector d) :
  Vector.contract (φ ⊗ₜ ψ) = CoVector.contract (ψ ⊗ₜ φ)
```

**lemma Vector.contract\_rep**[\[source\]](#)

```
lemma Vector.contract_rep (Λ : LorentzGroup d) (φ : Vector d) (ψ : CoVector d) :
  Vector.contract ((Vector.rep Λ φ) ⊗ₜ (CoVector.rep Λ ψ)) = Vector.contract (φ ⊗ₜ ψ)
```

**lemma CoVector.contract\_rep**[\[source\]](#)

```
lemma CoVector.contract_rep (Λ : LorentzGroup d) (φ : CoVector d) (ψ : Vector d) :
  CoVector.contract ((CoVector.rep Λ φ) ⊗ₜ (Vector.rep Λ ψ)) = CoVector.contract (φ ⊗ₜ ψ)
```

**2.88 Physlib.Relativity.Tensors.Reindexing**

[Physlib/Relativity/Tensors/Reindexing.lean](#) on GitHub.

**lemma append\_castAdd\_natAdd\_eq\_id**[\[source\]](#)

*Splitting Fin (m + n) into its two blocks and reassembling them is the identity.*

```
lemma append_castAdd_natAdd_eq_id {m n : ℕ} :
  Fin.append (Fin.castAdd n) (Fin.natAdd m) = (id : Fin (m + n) → Fin (m + n))
```

**lemma append\_succAbove\_const\_eq\_cycleIcc**[\[source\]](#)

*Moving slot i to the end is the cycle [i, last]: the block map that lists the i.succAbove survivors in order and then i is Fin.cycleIcc i (Fin.last n).*

```
lemma append_succAbove_const_eq_cycleIcc {n : ℕ} (i : Fin (n + 1)) :
  Fin.append i.succAbove (fun _ : Fin 1 => i) = ↑(Fin.cycleIcc i (Fin.last n))
```

**lemma swap**[\[source\]](#)

```
lemma swap {n : ℕ} {c : Fin n → C} (i j : Fin n) :
  IsReindexing c (c ∘ Equiv.swap i j) (Equiv.swap i j)
```

**lemma succAbove\_of\_succAbove\_eq**[\[source\]](#)

The conclusion of `succAbove` from an explicit survivor relabelling: if  $\sigma'$  fills the square  $(\sigma \ i).succAbove \circ \sigma' = \sigma \circ i.succAbove$ , carrying the complement of  $i$  to the complement of  $\sigma \ i$ , then it is a reindexing of the two shortened colour lists. The square forces  $\sigma'$  to be injective, hence bijective, so no bijectivity hypothesis is needed; this is why the statement is restricted to equal lengths.

Where `succAbove` builds the relabelling from  $\sigma$  as a *dite* composite, the map here is the caller's, which is what lets it stand in the statement of a lemma the caller instantiates.

```
lemma succAbove_of_succAbove_eq {n : ℕ} {c c1 : Fin (n + 1) → C}
  {σ : Fin (n + 1) → Fin (n + 1)} {σ' : Fin n → Fin n} (i : Fin (n + 1))
  (h : IsReindexing c c1 σ) (hσ' : (σ i).succAbove ∘ σ' = σ ∘ i.succAbove) :
  IsReindexing (c ∘ (σ i).succAbove) (c1 ∘ i.succAbove) σ'
```

**lemma succSuccAbove\_succAbove\_comm**[\[source\]](#)

```
lemma succSuccAbove_succAbove_comm {n : ℕ} {c : Fin (n + 1 + 1 + 1) → C}
  (k : Fin (n + 1)) (i j : Fin (n + 1 + 1 + 1)) :
```

**lemma move\_last**[\[source\]](#)

Splitting a list of colours  $c : \text{Fin } (n + 1) \rightarrow C$  at an arbitrary slot  $i$ , rather than at the last one as in `append_succ_last`: the block map that lists the  $i.succAbove$  survivors and then  $i$  itself matches  $c$  with the survivors of  $i$  followed by the surviving entry  $c1 \ 1$  of a rank-two list  $c1$  whose second entry is slot  $i$ 's colour.

```
lemma move_last {n : ℕ} {c : Fin (n + 1) → C} {c1 : Fin 2 → C} (i : Fin (n + 1))
  (hc : c i = c1 1) :
  IsReindexing c
    (Fin.append (c ∘ i.succAbove) (c1 ∘ (0 : Fin 2).succAbove))
    (Fin.append i.succAbove (fun _ : Fin 1 => i))
```

**lemma update\_update\_of\_eq**[\[source\]](#)

Updating slot  $i$  of  $c$  to  $d$  and then back to  $e = c \ i$  returns  $c$ , no other slot moving: the colour cast a round trip of two contractions at slot  $i$  generates.

```
lemma update_update_of_eq {n : ℕ} {c : Fin n → C} {d e : C} (i : Fin n) (he : c i = e) :
  IsReindexing c (Function.update (Function.update c i d) i e) (id : Fin n → Fin n)
```

**2.89 Physlib.Relativity.Tensors.TensorSpecies.Basic**

[Physlib/Relativity/Tensors/TensorSpecies/Basic.lean](#) on GitHub.

**lemma basisIdxCongr\_symm**[\[source\]](#)

```
lemma basisIdxCongr_symm {c c1 : C} (h : c = c1) (i : basisIdx c1) :
  (basisIdxCongr h).symm i = basisIdxCongr h.symm i
```

**2.90 Physlib.SpaceAndTime.Time.Derivatives**

[Physlib/SpaceAndTime/Time/Derivatives.lean](#) on GitHub.

**lemma deriv\_add**[\[source\]](#)

```
lemma deriv_add [NormedAddCommGroup M] [NormedSpace ℝ M] (f g : Time → M)
  (hf : DifferentiableAt ℝ f t) (hg : DifferentiableAt ℝ g t) :
  ∂t (fun s => f s + g s) t = ∂t f t + ∂t g t
```

**lemma deriv\_fun\_sum**[\[source\]](#)

```
lemma deriv_fun_sum {ι : Type*} [NormedAddCommGroup M] [NormedSpace ℝ M]
  (s : Finset ι) (a : ι → Time → M) (ha : ∀ i ∈ s, DifferentiableAt ℝ (a i) t) :
  ∂t (fun x => ∑ i ∈ s, a i x) t = ∑ i ∈ s, ∂t (a i) t
```

**lemma deriv\_mul\_const**[\[source\]](#)

```
lemma deriv_mul_const (f : Time → ℝ) (c : ℝ) (hf : DifferentiableAt ℝ f t) :
  ∂t (fun s => f s * c) t = ∂t f t * c
```

**lemma val\_contDiff**[\[source\]](#)

```
@[fun_prop]
lemma val_contDiff {n : WithTop ℕ} : ContDiff ℝ n Time.val
```

**lemma deriv\_space**[\[source\]](#)

```
lemma deriv_space {d : ℕ} {f : Time → Space d}
  (hf : Differentiable ℝ f) (t : Time) (i : Fin d) :
  deriv (fun s => f s i) t = deriv f t i
```

**2.91 Physlib.SpaceAndTime.Time.MatrixDerivatives**

[Physlib/SpaceAndTime/Time/MatrixDerivatives.lean](#) on GitHub.

**lemma matrix\_transpose**[\[source\]](#)

*The transpose of a differentiable matrix-valued function is differentiable (cf. Continuous.matrix\_transpose).*

```
lemma DifferentiableAt.matrix_transpose {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
  {A : E → Matrix (Fin d) (Fin d) ℝ} {t : E} (hA : DifferentiableAt ℝ A t) :
  DifferentiableAt ℝ (fun s => (A s)T) t
```

**lemma deriv\_matrix\_mul**[\[source\]](#)

*Product rule for the time derivative of a product of matrix-valued functions.*

```

lemma deriv_matrix_mul (A B : Time → Matrix (Fin d) (Fin d) ℝ) (t : Time)
  (hA : DifferentiableAt ℝ A t) (hB : DifferentiableAt ℝ B t) :
  ∂t (fun s => A s * B s) t = A t * ∂t B t + ∂t A t * B t

```

**lemma deriv\_matrix\_transpose**

[\[source\]](#)

*The time derivative commutes with transpose.*

```

lemma deriv_matrix_transpose (A : Time → Matrix (Fin d) (Fin d) ℝ) (t : Time)
  (hA : DifferentiableAt ℝ A t) :
  ∂t (fun s => (A s)T) t = (∂t A t)T

```

**lemma deriv\_matrix\_apply**

[\[source\]](#)

*The time derivative commutes with taking a matrix entry.*

```

lemma deriv_matrix_apply (A : Time → Matrix (Fin d) (Fin d) ℝ) (t : Time)
  (hA : DifferentiableAt ℝ A t) (i j : Fin d) :
  ∂t (fun s => A s i j) t = (∂t A t) i j

```

## 2.92 Physlib.Units.Dimension

[Physlib/Units/Dimension.lean](#) on GitHub.

**instance Mul (Dimension B)**

[\[source\]](#)

```

instance : Mul (Dimension B) where

```

**lemma mul\_exponent**

[\[source\]](#)

```

@[simp]
lemma mul_exponent (d1 d2 : Dimension B) (b : B) :
  (d1 * d2).exponent b = d1.exponent b + d2.exponent b

```

**instance One (Dimension B)**

[\[source\]](#)

```

instance : One (Dimension B) where

```

**lemma one\_exponent**

[\[source\]](#)

```

@[simp]
lemma one_exponent (b : B) : (1 : Dimension B).exponent b = 0

```

**instance CommGroup (Dimension B)**

[\[source\]](#)

```

instance : CommGroup (Dimension B) where

```

**lemma inv\_exponent**

[\[source\]](#)

```

@[simp]
lemma inv_exponent (d : Dimension B) (b : B) : d-1.exponent b = -d.exponent b

```

**lemma div\_exponent**[\[source\]](#)

```
@[simp]
lemma div_exponent (d1 d2 : Dimension B) (b : B) :
  (d1 / d2).exponent b = d1.exponent b - d2.exponent b
```

**lemma npow\_exponent**[\[source\]](#)

```
@[simp]
lemma npow_exponent (d : Dimension B) (n : ℕ) (b : B) :
  (d ^ n).exponent b = n • d.exponent b
```

**instance Pow (Dimension B) ℚ**[\[source\]](#)

```
instance : Pow (Dimension B) ℚ where
```

**lemma qpow\_exponent**[\[source\]](#)

```
@[simp]
lemma qpow_exponent (d : Dimension B) (q : ℚ) (b : B) :
  (d ^ q).exponent b = d.exponent b * q
```

**instance DecidableEq (Dimension B)**[\[source\]](#)

*Decidable equality of dimensions over a finite basis B.*

```
instance [Fintype B] : DecidableEq (Dimension B)
```

**def single**[\[source\]](#)

*The base-dimension vector for  $b : B$ : exponent 1 at  $b$ , 0 elsewhere. This is the generic analogue of the named generators  $L[$ ,  $T[$ , ...*

```
def single [DecidableEq B] (b : B) : Dimension B
```

**lemma single\_exponent**[\[source\]](#)

```
@[simp]
lemma single_exponent [DecidableEq B] (b b' : B) :
  (single b).exponent b' = if b' = b then 1 else 0
```

**def extend**[\[source\]](#)

*Change of basis along a map  $f : B \rightarrow B'$  of base dimensions: reindex a dimension over  $B$  into one over  $B'$  by placing each exponent at its image. For an embedding  $f$  (injective) this preserves every exponent (`extend_exponent_apply`), so a dimension in one system re-expresses faithfully in an extending one.*

```
def extend {B' : Type} [Fintype B] [DecidableEq B'] (f : B → B') (d : Dimension B) :
  Dimension B'
```

**lemma extend\_exponent\_apply**[\[source\]](#)

```
@[simp]
lemma extend_exponent_apply {B' : Type} [Fintype B] [DecidableEq B'] {f : B → B'}
  (hf : Function.Injective f) (d : Dimension B) (b : B) :
  (extend f d).exponent (f b) = d.exponent b
```

**def extendHom**[\[source\]](#)

*extend f packaged as a monoid homomorphism of dimensions.*

```
def extendHom {B' : Type} [Fintype B] [DecidableEq B'] (f : B → B') :
  Dimension B →* Dimension B' where
```

**structure Embedding**[\[source\]](#)

*A **\*\*dimension embedding\*\***  $B \hookrightarrow B'$ : an injective monoid homomorphism of dimensions. As a **MonoidHom** it is dimension-preserving (it respects products, inverses and rational powers); injectivity makes it a faithful inclusion of the basis  $B$  into  $B'$ . Cross-basis dimension injections are produced as **Embeddings** so that dimension-preservation holds by construction.*

```
structure Embedding (B B' : Type) where
  toHom : Dimension B →* Dimension B'
  inj : Function.Injective toHom
```

**structure Projection**[\[source\]](#)

*A **\*\*dimension projection\*\***  $B' \twoheadrightarrow B$ : a surjective monoid homomorphism of dimensions. As a **MonoidHom** it is truth-preserving, but it is lossy — it reduces a richer basis  $B'$  onto a coarser basis  $B$ , collapsing the base dimensions that  $B$  does not track.*

```
structure Projection (B' B : Type) where
  toHom : Dimension B' →* Dimension B
  surj : Function.Surjective toHom
```

**def ofBasis**[\[source\]](#)

*An injective **\*basis\*** map  $f : B \rightarrow B'$  induces a dimension embedding, via **extend**. This is the label-level case: it sends each base dimension of  $B$  to a base dimension of  $B'$ , so it is automatically dimension-preserving and faithful.*

```
def Embedding.ofBasis {B B' : Type} [Fintype B] [DecidableEq B']
  (f : B → B') (hf : Function.Injective f) : Embedding B B' where
```

**2.93 Physlib.Units.ISQBridge**

[Physlib/Units/ISQBridge.lean](#) on GitHub.

**def toISQFun**[\[source\]](#)

*The function underlying **toISQHom**: fix length, mass, time and temperature, and send *PhysLib*'s charge generator to the derived ISQ charge  $I \cdot T$  (the current exponent is the charge exponent, and the time exponent absorbs it).*

```
def toISQFun (d : Dimension LTMCTDimensionBase) : Dimension ISQDimensionBase
```

**def toISQHom**

[\[source\]](#)

*The dimension-preserving embedding of PhysLib dimensions into the ISQ dimensions.*

```
def toISQHom : Dimension LTMCTDimensionBase →* Dimension ISQDimensionBase where
```

**lemma toISQHom\_apply**

[\[source\]](#)

*toISQHom applied to a dimension is toISQFun.*

```
lemma toISQHom_apply (d : Dimension LTMCTDimensionBase) : toISQHom d = toISQFun d
```

**def fromISQFun**

[\[source\]](#)

*The function underlying fromISQHom: fix length, mass and temperature, read electric current as charge/time (the charge exponent is the current exponent, and the time exponent subtracts it), and drop amount of substance and luminous intensity.*

```
def fromISQFun (d : Dimension ISQDimensionBase) : Dimension LTMCTDimensionBase
```

**def fromISQHom**

[\[source\]](#)

*The truth-preserving reduction of ISQ dimensions onto PhysLib's.*

```
def fromISQHom : Dimension ISQDimensionBase →* Dimension LTMCTDimensionBase where
```

**lemma fromISQHom\_apply**

[\[source\]](#)

*fromISQHom applied to a dimension is fromISQFun.*

```
lemma fromISQHom_apply (d : Dimension ISQDimensionBase) : fromISQHom d = fromISQFun d
```

**lemma fromISQHom\_comp\_toISQHom**

[\[source\]](#)

*The projection recovers PhysLib's basis from its image under the embedding: fromISQHom ◦ toISQHom = id.*

```
lemma fromISQHom_comp_toISQHom :  
  fromISQHom.comp toISQHom = MonoidHom.id (Dimension LTMCTDimensionBase)
```

**lemma toISQHom\_injective**

[\[source\]](#)

*toISQHom is injective: PhysLib dimensions include faithfully into ISQ.*

```
lemma toISQHom_injective : Function.Injective toISQHom
```



**lemma fromISQHom\_surjective**[\[source\]](#)

*fromISQHom is surjective: every PhysLib dimension is the reduction of some ISQ dimension (namely its own embedding).*

```
lemma fromISQHom_surjective : Function.Surjective fromISQHom
```

**def ltmctToISQ**[\[source\]](#)

*PhysLib's dimensions embed into the ISQ dimensions (charge  $\mapsto I \cdot T$ ).*

```
def ltmctToISQ : Embedding LTMCTDimensionBase ISQDimensionBase where
```

**def isqToLTMCT**[\[source\]](#)

*The ISQ dimensions project onto PhysLib's, reading current as charge/time and forgetting amount of substance and luminous intensity.*

```
def isqToLTMCT : Projection ISQDimensionBase LTMCTDimensionBase where
```

**lemma isqToLTMCT\_comp\_ltmctToISQ**[\[source\]](#)

*The projection is a retraction of the embedding:  $\text{isqToLTMCT} \circ \text{ltmctToISQ} = \text{id}$ . So PhysLib's basis embeds faithfully into ISQ and is recovered by the projection, but not conversely — amount of substance and luminous intensity cannot be recovered.*

```
lemma isqToLTMCT_comp_ltmctToISQ :  
  isqToLTMCT.toHom.comp ltmctToISQ.toHom = MonoidHom.id (Dimension LTMCTDimensionBase)
```

**lemma toISQHom\_C**[\[source\]](#)

*The embedding is physically faithful on charge: PhysLib's charge generator  $C$  maps to the \*derived\* ISQ charge  $I \cdot T$ .*

```
lemma toISQHom_C : toISQHom C = ISQDimensionBase.charge
```

**2.94 Physlib.Units.ISQDimensionBase**

[Physlib/Units/ISQDimensionBase.lean](#) on GitHub.

**inductive ISQDimensionBase**[\[source\]](#)

*The seven ISQ base quantities (ISO/IEC 80000-1; VIM 3rd edition). Unlike LTMCTDimensionBase, electric current is the electromagnetic base quantity (so charge is derived), and amount of substance and luminous intensity are further base quantities.*

```
inductive ISQDimensionBase where  
  | length  
  | mass  
  | time  
  | current  
  | temperature  
  | amount  
  | luminousIntensity
```

**lemma card\_eq\_seven**[\[source\]](#)*The ISQ has seven base quantities.*

```
lemma card_eq_seven : Fintype.card ISQDimensionBase = 7
```

**def charge**[\[source\]](#)*Electric charge is a *\*derived\** dimension in the ISQ:  $Q = I \cdot T$ .*

```
def charge : Dimension ISQDimensionBase
```

**lemma charge\_exponent\_current**[\[source\]](#)*The derived charge dimension has electric-current exponent 1.*

```
lemma charge_exponent_current : charge.exponent .current = 1
```

**lemma charge\_exponent\_time**[\[source\]](#)*The derived charge dimension has time exponent 1.*

```
lemma charge_exponent_time : charge.exponent .time = 1
```

**lemma charge\_exponent\_mass**[\[source\]](#)*The derived charge dimension has mass exponent 0.*

```
lemma charge_exponent_mass : charge.exponent .mass = 0
```

**lemma single\_amount\_ne\_one**[\[source\]](#)*Amount of substance is an independent base dimension (absent from *LTMCTDimensionBase*).*

```
lemma single_amount_ne_one : single (.amount : ISQDimensionBase) ≠ 1
```

**lemma single\_luminousIntensity\_ne\_one**[\[source\]](#)*Luminous intensity is an independent base dimension (absent from *LTMCTDimensionBase*).*

```
lemma single_luminousIntensity_ne_one :
  single (.luminousIntensity : ISQDimensionBase) ≠ 1
```

**2.95 Physlib.Units.LTMCTDimensionBase**[Physlib/Units/LTMCTDimensionBase.lean](#) on GitHub.**inductive LTMCTDimensionBase**[\[source\]](#)*PhysLib's default basis of base dimensions — length, time, mass, charge, temperature.  
Note this is *\*charge\**-based, so it is not the SI/ISQ base-quantity set; see *Dimension*.*

```
inductive LTMCTDimensionBase where
| length
| time
```

```
| mass
| charge
| temperature
```

**def length**[\[source\]](#)

*The length exponent of a LTMCTDimensionBase dimension.*

```
def length (d : Dimension LTMCTDimensionBase) : ℚ
```

**def time**[\[source\]](#)

*The time exponent of a LTMCTDimensionBase dimension.*

```
def time (d : Dimension LTMCTDimensionBase) : ℚ
```

**def mass**[\[source\]](#)

*The mass exponent of a LTMCTDimensionBase dimension.*

```
def mass (d : Dimension LTMCTDimensionBase) : ℚ
```

**def charge**[\[source\]](#)

*The charge exponent of a LTMCTDimensionBase dimension.*

```
def charge (d : Dimension LTMCTDimensionBase) : ℚ
```

**def temperature**[\[source\]](#)

*The temperature exponent of a LTMCTDimensionBase dimension.*

```
def temperature (d : Dimension LTMCTDimensionBase) : ℚ
```

**def ofLTMCTDimensionBase**[\[source\]](#)

*Build a LTMCTDimensionBase dimension from its five exponents, in the order (length, time, mass, charge, temperature).*

```
def ofLTMCTDimensionBase (length time mass charge temperature : ℚ) : Dimension
  LTMCTDimensionBase
```

**lemma ofLTMCTDimensionBase\_length**[\[source\]](#)

```
@[simp]
lemma ofLTMCTDimensionBase_length (l t m c θ : ℚ) :
  (ofLTMCTDimensionBase l t m c θ).length = l
```

**lemma ofLTMCTDimensionBase\_time**[\[source\]](#)

```
@[simp]
lemma ofLTMCTDimensionBase_time (l t m c θ : ℚ) :
  (ofLTMCTDimensionBase l t m c θ).time = t
```

**lemma ofLTMCTDimensionBase\_mass**[\[source\]](#)

```
@[simp]
lemma ofLTMCTDimensionBase_mass (l t m c θ : ℚ) :
  (ofLTMCTDimensionBase l t m c θ).mass = m
```

**lemma ofLTMCTDimensionBase\_charge**[\[source\]](#)

```
@[simp]
lemma ofLTMCTDimensionBase_charge (l t m c θ : ℚ) :
  (ofLTMCTDimensionBase l t m c θ).charge = c
```

**lemma ofLTMCTDimensionBase\_temperature**[\[source\]](#)

```
@[simp]
lemma ofLTMCTDimensionBase_temperature (l t m c θ : ℚ) :
  (ofLTMCTDimensionBase l t m c θ).temperature = θ
```

**lemma L\_eq\_single**[\[source\]](#)

```
lemma L_eq_single : L = single.length
```

**lemma T\_eq\_single**[\[source\]](#)

```
lemma T_eq_single : T = single.time
```

**lemma M\_eq\_single**[\[source\]](#)

```
lemma M_eq_single : M = single.mass
```

**lemma C\_eq\_single**[\[source\]](#)

```
lemma C_eq_single : C = single.charge
```

**lemma θ\_eq\_single**[\[source\]](#)

```
lemma θ_eq_single : θ = single.temperature
```

**2.96 Physlib.Units.ParametricDimensionExamples**

[Physlib/Units/ParametricDimensionExamples.lean](#) on GitHub.

**inductive Info**[\[source\]](#)

*A basis of information-theoretic base dimensions.*

```
inductive Info
| bit
| symbol
```

**def bitDim**[\[source\]](#)*The bit base dimension.***def** bitDim : Dimension Info**def symbolDim**[\[source\]](#)*The symbol base dimension.***def** symbolDim : Dimension Info

## 2.97 Physlib.Units.ParametricUnits

[Physlib/Units/ParametricUnits.lean](#) on GitHub.**structure UnitScale**[\[source\]](#)*A choice of unit for each base dimension of B: a positive-real magnitude per base.  
This is the basis-generic form of UnitChoices.***structure** UnitScale (B : Type) **where**  
scale : B → ℝ≥0  
scale\_pos : ∀ b, 0 < scale b**lemma ratio\_ne\_zero**[\[source\]](#)**lemma** ratio\_ne\_zero (u1 u2 : UnitScale B) (b : B) : u1.scale b / u2.scale b ≠ 0**def dimScale**[\[source\]](#)*The dimension-scaling homomorphism, generic in the basis B: a quantity of dimension d rescales by  $\frac{u1\ b}{u2\ b} ^{d.exponent\ b}$  when changing the unit choice from u1 to u2. This is the basis-generic form of UnitChoices.dimScale.***noncomputable def** dimScale [Fintype B] (u1 u2 : UnitScale B) : Dimension B →\* ℝ≥0 **where****lemma dimScale\_self**[\[source\]](#)**@[simp]**  
**lemma** dimScale\_self [Fintype B] (u : UnitScale B) (d : Dimension B) :  
dimScale u u d = 1**lemma dimScale\_one**[\[source\]](#)**@[simp]**  
**lemma** dimScale\_one [Fintype B] (u1 u2 : UnitScale B) :  
dimScale u1 u2 1 = 1**lemma dimScale\_transitive**[\[source\]](#)*The scaling is transitive (a cocycle in the unit choices).*

```
lemma dimScale_transitive [Fintype B] (u1 u2 u3 : UnitScale B) (d : Dimension B) :
  dimScale u1 u2 d * dimScale u2 u3 d = dimScale u1 u3 d
```

**def toScale**

[\[source\]](#)

*Read a five-field UnitChoices as a UnitScale over LTMCTDimensionBase.*

```
noncomputable def toScale (u : UnitChoices) : UnitScale LTMCTDimensionBase where
```

## 2.98 Physlib.Units.UnitDependent

[Physlib/Units/UnitDependent.lean](#) on GitHub.

**instance MulAction  $\mathbb{R}_{\geq 0}$  (DimSet M d)**

[\[source\]](#)

```
instance (M : Type) [MulAction  $\mathbb{R}_{\geq 0}$  M] [MulUnitDependent M] (d : Dimension LTMCTDimensionBase) :
  MulAction  $\mathbb{R}_{\geq 0}$  (DimSet M d) where
```

**instance CarriesDimension (DimSet M d)**

[\[source\]](#)

```
instance (M : Type) [MulAction  $\mathbb{R}_{\geq 0}$  M] [MulUnitDependent M] (d : Dimension LTMCTDimensionBase) :
  CarriesDimension (DimSet M d) where
```

## 2.99 Physlib.Units.WithDim.Basic

[Physlib/Units/WithDim/Basic.lean](#) on GitHub.

**instance HasDim (WithDim d M)**

[\[source\]](#)

```
instance (d : Dimension LTMCTDimensionBase) (M : Type) : HasDim (WithDim d M) where
```

**instance Inhabited (WithDim d M)**

[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [Inhabited M] : Inhabited (WithDim d M) where
```

**instance Zero (WithDim d M)**

[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [Zero M] : Zero (WithDim d M) where
```

**instance Add (WithDim d M)**

[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [Add M] : Add (WithDim d M) where
```

**instance Neg (WithDim d M)**

[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [Neg M] : Neg (WithDim d M) where
```

**instance Sub (WithDim d M)**

[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [Sub M] : Sub (WithDim d M) where
```

**instance AddSemigroup (WithDim d M)**[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [AddSemigroup M] :
  AddSemigroup (WithDim d M) where
```

**instance AddCommSemigroup (WithDim d M)**[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [AddCommSemigroup M] :
  AddCommSemigroup (WithDim d M) where
```

**instance AddMonoid (WithDim d M)**[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [AddMonoid M] :
  AddMonoid (WithDim d M) where
```

**instance AddCommMonoid (WithDim d M)**[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [AddCommMonoid M] :
  AddCommMonoid (WithDim d M) where
```

**instance AddGroup (WithDim d M)**[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [AddGroup M] :
  AddGroup (WithDim d M) where
```

**instance AddCommGroup (WithDim d M)**[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [AddCommGroup M] :
  AddCommGroup (WithDim d M) where
```

**instance LE (WithDim d M)**[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [LE M] : LE (WithDim d M) where
```

**instance LT (WithDim d M)**[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [LT M] : LT (WithDim d M) where
```

**instance Preorder (WithDim d M)**[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [Preorder M] :
  Preorder (WithDim d M) where
```

**instance PartialOrder (WithDim d M)**[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [PartialOrder M] :
  PartialOrder (WithDim d M) where
```

**instance MulAction  $\mathbb{R}_{\geq 0}$  (WithDim d M)**

[\[source\]](#)

```
instance {B : Type} (d : Dimension B) (M : Type) [MulAction  $\mathbb{R}_{\geq 0}$  M] :
  MulAction  $\mathbb{R}_{\geq 0}$  (WithDim d M) where
```

**instance HMul (WithDim d1  $\mathbb{R}$ ) (WithDim d2  $\mathbb{R}$ ) (WithDim (d1 \* d2)  $\mathbb{R}$ )**

[\[source\]](#)

```
instance {B : Type} {d1 d2 : Dimension B} :
  HMul (WithDim d1  $\mathbb{R}$ ) (WithDim d2  $\mathbb{R}$ ) (WithDim (d1 * d2)  $\mathbb{R}$ ) where
```

**instance DMul (WithDim d1  $\mathbb{R}$ ) (WithDim d2  $\mathbb{R}$ ) (WithDim (d1 \* d2)  $\mathbb{R}$ )**

[\[source\]](#)

```
instance {d1 d2 : Dimension LTMCTDimensionBase} :
  DMul (WithDim d1  $\mathbb{R}$ ) (WithDim d2  $\mathbb{R}$ ) (WithDim (d1 * d2)  $\mathbb{R}$ ) where
```

**instance HDiv (WithDim d1  $\mathbb{R}$ ) (WithDim d2  $\mathbb{R}$ ) (WithDim (d1 \* d2<sup>-1</sup>)  $\mathbb{R}$ )**

[\[source\]](#)

```
noncomputable instance {B : Type} (d1 d2 : Dimension B) :
  HDiv (WithDim d1  $\mathbb{R}$ ) (WithDim d2  $\mathbb{R}$ ) (WithDim (d1 * d2-1)  $\mathbb{R}$ ) where
```

## 2.100 PhyslibAlpha.ClassicalFieldTheory.Local.EulerLagrangeEquation

[PhyslibAlpha/ClassicalFieldTheory/Local/EulerLagrangeEquation.lean](#) on GitHub.

**def SatisfiesEulerLagrange**

[\[source\]](#)

*A field satisfies the local Euler-Lagrange equations for a local lagrangian when the Euler-Lagrange operator vanishes along that field.*

```
def SatisfiesEulerLagrange (L : Lagrangian d m k)
  (f : Space d → EuclideanSpace  $\mathbb{R}$  (Fin m)) : Prop
```

**lemma satisfiesEulerLagrange\_iff**

[\[source\]](#)

```
@[simp]
lemma satisfiesEulerLagrange_iff (L : Lagrangian d m k)
  (f : Space d → EuclideanSpace  $\mathbb{R}$  (Fin m)) :
  SatisfiesEulerLagrange L f ↔ eulerLagrangeOp L f = 0
```

**lemma eulerLagrangeOp\_eq\_zero**

[\[source\]](#)

```
lemma SatisfiesEulerLagrange.eulerLagrangeOp_eq_zero
  {L : Lagrangian d m k} {f : Space d → EuclideanSpace  $\mathbb{R}$  (Fin m)}
  (h : SatisfiesEulerLagrange L f) :
  eulerLagrangeOp L f = 0
```

**lemma of\_eulerLagrangeOp\_eq\_zero**

[\[source\]](#)

```
lemma SatisfiesEulerLagrange.of_eulerLagrangeOp_eq_zero
  {L : Lagrangian d m k} {f : Space d → EuclideanSpace  $\mathbb{R}$  (Fin m)}
  (h : eulerLagrangeOp L f = 0) :
  SatisfiesEulerLagrange L f
```



**theorem isCritical\_iff\_satisfiesEulerLagrange\_of\_contDiff\_and\_smoothInCoordinates** [\[source\]](#)

```

theorem isCritical_iff_satisfiesEulerLagrange_of_contDiff_and_smoothInCoordinates
  (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
  (hL : Lagrangian.SmoothInCoordinates L)
  (hbase : HasFiniteAction L f) :
  IsCritical L f ↔ SatisfiesEulerLagrange L f

```

**theorem isCritical\_iff\_satisfiesEulerLagrange\_of\_admissibleForAction\_and\_smoothInCoordinates** [\[source\]](#)

```

theorem isCritical_iff_satisfiesEulerLagrange_of_admissibleForAction_and_smoothInCoordinates
  (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
  (hLf : IsAdmissibleForAction L f)
  (hL : Lagrangian.SmoothInCoordinates L) :
  IsCritical L f ↔ SatisfiesEulerLagrange L f

```

**2.101 PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.ChargeBalance**

[PhyslibAlpha/Particles/BeyondTheStandardModel/TwoHDM/ChargeBalance.lean](#) on GitHub.

**lemma coeff\_aeval\_diag** [\[source\]](#)

*Rescaling each variable  $X_i$  by a constant  $d_i$  multiplies the coefficient of the monomial  $m$  by  $\prod_i (d_i)^{m_i}$ .*

```

lemma coeff_aeval_diag {σ R : Type*} [CommRing R] (d : σ → R) (f : MvPolynomial σ R)
  (m : σ →₀ ℕ) :
  coeff m (aeval (fun i => C (d i) * X i) f) = (m.prod fun i k => d i ^ k) * coeff m f

```

**lemma coeff\_eq\_zero\_of\_charge\_ne\_zero** [\[source\]](#)

*\*\*Charge balancing.\*\* If each variable  $X_i$  carries an integer charge  $w_i$ ,  $c$  is a phase of infinite order, and the polynomial  $f$  is invariant under the charge rotation  $X_i \mapsto c^{w_i} X_i$ , then every monomial with nonzero net charge has vanishing coefficient.*

```

lemma coeff_eq_zero_of_charge_ne_zero {σ K : Type*} [Field K] (w : σ → ℤ) (c : K) (hc : c ≠ 0)
  (hroot : ∀ n : ℤ, c ^ n = 1 → n = 0) {f : MvPolynomial σ K}
  (hf : aeval (fun i => C (c ^ (w i)) * X i) f = f) {m : σ →₀ ℕ}
  (hm : ∑ i ∈ m.support, (m i : ℤ) * w i ≠ 0) :
  coeff m f = 0

```

**2.102 PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.EffectivePotential**

[PhyslibAlpha/Particles/BeyondTheStandardModel/TwoHDM/EffectivePotential.lean](#) on GitHub.

**abbrev EffectivePotential** [\[source\]](#)

*A general potential of the Higgs field.*

```

abbrev EffectivePotential : Type

```

**def IsInvariant**[\[source\]](#)

*The proposition that the general potential is invariant under the global action of the gauge group.*

```
def IsInvariant (V : EffectivePotential) : Prop
```

**def HasMaxMassDimLE**[\[source\]](#)

*The proposition that the potential  $V$  has a maximum mass dimension less then or equal to  $n$  - also implying it is a polynomial.*

```
def HasMaxMassDimLE (V : EffectivePotential) (n : ℕ) : Prop
```

**lemma exists\_comp\_linear\_poly**[\[source\]](#)

*A polynomial potential, restricted along any real-linear parametrisation  $L$  of field configurations, is a genuine polynomial in the parameters. This is the bookkeeping that lets the potential be evaluated on the field components of a gauge slice.*

```
lemma HasMaxMassDimLE.exists_comp_linear_poly {V : EffectivePotential} {n : ℕ}
  (h : HasMaxMassDimLE V n) {ι : Type*} [Fintype ι] [DecidableEq ι]
  (L : (ι → ℝ) →ℓ[ℝ] TwoHiggsDoublet) :
  ∃ P : MvPolynomial ι ℝ, ∀ a : ι → ℝ, V (L a) = P.eval a
```

**2.103 PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.GaugeSlice**

[PhyslibAlpha/Particles/BeyondTheStandardModel/TwoHDM/GaugeSlice.lean](#) on GitHub.

**def sliceHiggs**[\[source\]](#)

*The upper-triangular slice configuration  $((z, \theta), (w_0, w_1))$ . It specialises to `repHiggs` when the components take their real "canonical frame" values.*

```
def sliceHiggs (z w0 w1 : ℂ) : TwoHiggsDoublet where
```

**def sliceR**[\[source\]](#)

*The slice as a real-linear map from the six real field parameters  $(\text{Re } \phi_{10}, \text{Im } \phi_{10}, \text{Re } \phi_{20}, \text{Im } \phi_{20}, \text{Re } \phi_{21}, \text{Im } \phi_{21})$ .*

```
def sliceR : (Fin 6 → ℝ) →ℓ[ℝ] TwoHiggsDoublet where
```

**lemma repHiggs\_eq\_sliceHiggs**[\[source\]](#)

*The representative family is the real slice.*

```
lemma repHiggs_eq_sliceHiggs (X : Fin 4 → ℝ) :
  repHiggs X = sliceHiggs (X 0) ((X 1 : ℂ) + Complex.I * (X 2 : ℂ)) (X 3)
```

**lemma gaugeCartan\_smul\_sliceHiggs**[\[source\]](#)

*Hypercharge action of the Cartan phase on the slice: it multiplies the first components by  $a$  and the perpendicular second component by  $\bar{a}$ .*

```

lemma gaugeCartan_smul_sliceHiggs (a : unitary ℂ) (z w0 w1 : ℂ) :
  GaugeGroupI.gaugeCartan a • sliceHiggs z w0 w1
  = sliceHiggs ((a : ℂ) * z) ((a : ℂ) * w0) ((star a : ℂ) * w1)

```

**lemma ofU1Subgroup\_smul\_sliceHiggs**

[\[source\]](#)

*Hypercharge action of the residual  $U(1)$  on the slice: it multiplies the perpendicular second component by  $c^6$  and leaves the first components fixed.*

```

lemma ofU1Subgroup_smul_sliceHiggs (c : unitary ℂ) (z w0 w1 : ℂ) :
  GaugeGroupI.ofU1Subgroup c • sliceHiggs z w0 w1
  = sliceHiggs z w0 ((c : ℂ) ^ 6 * w1)

```

**def cartanRotParam**

[\[source\]](#)

*The Cartan hypercharge phase  $u$ , transported to a rotation of the six real field parameters: it phases the first-component pairs by  $u$  and the perpendicular pair by  $\bar{u}$ .*

```

def cartanRotParam (u : unitary ℂ) (a : Fin 6 → ℝ) : Fin 6 → ℝ

```

**lemma gaugeCartan\_smul\_sliceR**

[\[source\]](#)

*Acting by the Cartan phase on a slice configuration is the same as rotating its parameters.*

```

lemma gaugeCartan_smul_sliceR (u : unitary ℂ) (a : Fin 6 → ℝ) :
  GaugeGroupI.gaugeCartan u • sliceR a = sliceR (cartanRotParam u a)

```

**def resRotParam**

[\[source\]](#)

*The residual  $U(1)$  phase  $c$ , transported to a rotation of the perpendicular parameter pair.*

```

def resRotParam (c : unitary ℂ) (a : Fin 6 → ℝ) : Fin 6 → ℝ

```

**lemma ofU1Subgroup\_smul\_sliceR**

[\[source\]](#)

*Acting by the residual  $U(1)$  on a slice configuration rotates only the perpendicular pair.*

```

lemma ofU1Subgroup_smul_sliceR (c : unitary ℂ) (a : Fin 6 → ℝ) :
  GaugeGroupI.ofU1Subgroup c • sliceR a = sliceR (resRotParam c a)

```

## 2.104 PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.GaugeTorus

[PhyslibAlpha/Particles/BeyondTheStandardModel/TwoHDM/GaugeTorus.lean](#) on GitHub.

**def gaugeCartan**

[\[source\]](#)

*The Cartan  $SU(2)$  gauge element  $\text{diag}(a, \bar{a})$ , for  $a$  a phase.*

```

noncomputable def gaugeCartan (a : unitary ℂ) : GaugeGroupI

```

**lemma gaugeCartan\_toU1**[\[source\]](#)

```
@[simp]
lemma gaugeCartan_toU1 (a : unitary ℂ) : (gaugeCartan a).toU1 = 1
```

**lemma gaugeCartan\_toSU2\_coe**[\[source\]](#)

```
@[simp]
lemma gaugeCartan_toSU2_coe (a : unitary ℂ) :
  ((gaugeCartan a).toSU2 : Matrix (Fin 2) (Fin 2) ℂ) = !![(a : ℂ), 0; 0, (star a : ℂ)]
```

**lemma gaugeCartan\_smul\_eq**[\[source\]](#)

*The Cartan element acts as the diagonal matrix  $\text{diag}(a, \bar{a})$ .*

```
lemma gaugeCartan_smul_eq (a : unitary ℂ) (φ : HiggsVec) :
  gaugeCartan a • φ = WithLp.toLp 2 (!![(a : ℂ), 0; 0, (star a : ℂ)] *v φ.ofLp)
```

**2.105 PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.Invariants**

[PhyslibAlpha/Particles/BeyondTheStandardModel/TwoHDM/Invariants.lean](#) on GitHub.

**lemma eval\_aeval\_comp**[\[source\]](#)

*Pushing an evaluation through an aeval substitution.*

```
lemma eval_aeval_comp {R : Type*} [CommRing R] {κ ι : Type*} (x : ι → R)
  (f : κ → MvPolynomial ι R) (G : MvPolynomial κ R) :
  eval x (aeval f G) = eval (fun i => eval x (f i)) G
```

**def cartanSubst**[\[source\]](#)

*The Cartan hypercharge rotation of the slice parameters, as a substitution of the polynomial variables.*

```
noncomputable def cartanSubst (u : unitary ℂ) : Fin 6 → MvPolynomial (Fin 6) ℝ
```

**lemma eval\_cartanSubst**[\[source\]](#)

```
lemma eval_cartanSubst (u : unitary ℂ) (a : Fin 6 → ℝ) :
  (fun k => MvPolynomial.eval a (cartanSubst u k)) = cartanRotParam u a
```

**lemma aeval\_cartanSubst\_eq**[\[source\]](#)

*Gauge (Cartan) invariance of the potential forces the slice polynomial to be invariant under the hypercharge rotation of its variables.*

```
lemma aeval_cartanSubst_eq {V : EffectivePotential} (hI : IsInvariant V)
  {P : MvPolynomial (Fin 6) ℝ} (hP : ∀ a, V (sliceR a) = P.eval a) (u : unitary ℂ) :
  aeval (cartanSubst u) P = P
```

**def residualSubst**[\[source\]](#)

*The residual  $U(1)$  rotation of the perpendicular parameter, as a substitution.*

```
noncomputable def residualSubst (c : unitary ℂ) : Fin 6 → MvPolynomial (Fin 6) ℝ
```

**lemma eval\_residualSubst**[\[source\]](#)

```
lemma eval_residualSubst (c : unitary ℂ) (a : Fin 6 → ℝ) :  
  (fun k => MvPolynomial.eval a (residualSubst c k)) = resRotParam c a
```

**lemma aeval\_residualSubst\_eq**[\[source\]](#)

*Gauge (residual  $U(1)$ ) invariance forces the slice polynomial to be invariant under the perpendicular rotation of its variables.*

```
lemma aeval_residualSubst_eq {V : EffectivePotential} (hI : IsInvariant V)  
{P : MvPolynomial (Fin 6) ℝ} (hP : ∀ a, V (sliceR a) = P.eval a) (c : unitary ℂ) :  
  aeval (residualSubst c) P = P
```

**def hyperchargeEigen**[\[source\]](#)

*Change to hypercharge eigen-coordinates:  $a_k$  in terms of  $z, z^-, w_\theta, w_\theta^-, w_1, w_1^-$  (indices  $0..5$ ). This diagonalises the gauge-torus rotation into a scaling.*

```
noncomputable def hyperchargeEigen : Fin 6 → MvPolynomial (Fin 6) ℂ
```

**def diagCartan**[\[source\]](#)

*The Cartan hypercharge, diagonal in eigen-coordinates: charges  $(1, -1, 1, -1, -1, 1)$ .*

```
noncomputable def diagCartan (u : unitary ℂ) : Fin 6 → MvPolynomial (Fin 6) ℂ
```

**def diagRes**[\[source\]](#)

*The residual  $U(1)$ , diagonal in eigen-coordinates: only the perpendicular pair is charged.*

```
noncomputable def diagRes (c : unitary ℂ) : Fin 6 → MvPolynomial (Fin 6) ℂ
```

**lemma bind<sub>1</sub>\_diagCartan\_hyperchargeEigen**[\[source\]](#)

*Conjugation identity: the diagonal Cartan scaling, pulled back through the eigen-coordinate change, is the (complexified) Cartan rotation substitution.*

```
lemma bind1_diagCartan_hyperchargeEigen (u : unitary ℂ) (k : Fin 6) :  
  bind1 (diagCartan u) (hyperchargeEigen k)  
  = bind1 hyperchargeEigen (map (algebraMap ℝ ℂ) (cartanSubst u k))
```

**lemma bind<sub>1</sub>\_diagRes\_hyperchargeEigen**[\[source\]](#)

*Conjugation identity for the residual  $U(1)$ .*

```

lemma bind1_diagRes_hyperchargeEigen (c : unitary ℂ) (k : Fin 6) :
  bind1 (diagRes c) (hyperchargeEigen k)
    = bind1 hyperchargeEigen (map (algebraMap ℝ ℂ) (residualSubst c k))

```

**def cartanCharge**

[\[source\]](#)

*The Cartan hypercharges of  $z, z^-, w_\theta, w_\theta^-, w_1, w_1^-$ .*

```

def cartanCharge : Fin 6 → ℤ

```

**def hyperCharge**

[\[source\]](#)

*The residual- $U(1)$  hypercharges (only the perpendicular pair is charged).*

```

def hyperCharge : Fin 6 → ℤ

```

**def Qslice**

[\[source\]](#)

*The slice potential, complexified and written in hypercharge eigen-coordinates.*

```

noncomputable def Qslice (P : MvPolynomial (Fin 6) ℝ) : MvPolynomial (Fin 6) ℂ

```

**lemma diagCartan\_eq**

[\[source\]](#)

*The Cartan diagonal in the charge form consumed by the charge-balancing engine.*

```

lemma diagCartan_eq (u : unitary ℂ) :
  diagCartan u = fun i => ℂ ((u : ℂ) ^ (cartanCharge i)) * X i

```

**lemma diagRes\_eq**

[\[source\]](#)

*The residual diagonal in the charge form consumed by the engine.*

```

lemma diagRes_eq (c : unitary ℂ) :
  diagRes c = fun i => ℂ (((c : ℂ) ^ 6) ^ (hyperCharge i)) * X i

```

**lemma bind<sub>1</sub>\_diagCartan\_Qslice**

[\[source\]](#)

*In eigen-coordinates, the Cartan hypercharge acts by the diagonal scaling, and the slice potential is invariant under it.*

```

lemma bind1_diagCartan_Qslice {V : EffectivePotential} (hI : IsInvariant V)
  {P : MvPolynomial (Fin 6) ℝ} (hP : ∀ a, V (sliceR a) = P.eval a) (u : unitary ℂ) :
  bind1 (diagCartan u) (Qslice P) = Qslice P

```

**lemma bind<sub>1</sub>\_diagRes\_Qslice**

[\[source\]](#)

*Likewise for the residual  $U(1)$ .*

```

lemma bind1_diagRes_Qslice {V : EffectivePotential} (hI : IsInvariant V)
  {P : MvPolynomial (Fin 6) ℝ} (hP : ∀ a, V (sliceR a) = P.eval a) (c : unitary ℂ) :
  bind1 (diagRes c) (Qslice P) = Qslice P

```

**lemma exists\_infiniteOrder\_unitary**[\[source\]](#)

*There is a gauge phase of infinite order ( $\exp i$ ), needed to run charge balancing.*

```
lemma exists_infiniteOrder_unitary :
  ∃ ω : unitary ℂ, ∀ n : ℤ, (ω : ℂ) ^ n = 1 → n = 0
```

**lemma coeff\_Qslice\_eq\_zero**[\[source\]](#)

*\*\*Hypercharge balancing.\*\* Every monomial of the slice potential  $Q_{\text{slice}}$   $P$  (in eigen-coordinates) that carries nonzero Cartan or residual hypercharge has vanishing coefficient.*

```
lemma coeff_Qslice_eq_zero {V : EffectivePotential} (hI : IsInvariant V)
  {P : MvPolynomial (Fin 6) ℝ} (hP : ∀ a, V (sliceR a) = P.eval a) (m : Fin 6 → ℕ)
  (hm : (∑ i ∈ m.support, (m i : ℤ) * cartanCharge i ≠ 0) ∨
    (∑ i ∈ m.support, (m i : ℤ) * hyperCharge i ≠ 0)) :
  coeff m (Qslice P) = 0
```

**def neutralBilinear**[\[source\]](#)

*The five hypercharge-neutral quadratic bilinears in eigen-coordinates:  $z z^-$ ,  $w_\theta w_\theta^-$ ,  $z w_\theta^-$ ,  $z^- w_\theta$ ,  $w_1 w_1^-$ .*

```
noncomputable def neutralBilinear : Fin 5 → MvPolynomial (Fin 6) ℂ
```

**lemma charge\_univ\_eq\_support**[\[source\]](#)

*The charge of a monomial, summed over the whole index set, equals the sum over its support.*

```
lemma charge_univ_eq_support (w : Fin 6 → ℤ) (m : Fin 6 → ℕ) :
  ∑ i, (m i : ℤ) * w i = ∑ i ∈ m.support, (m i : ℤ) * w i
```

**lemma chargeSum\_add**[\[source\]](#)

*A charge sum is additive in the monomial.*

```
lemma chargeSum_add (w : Fin 6 → ℤ) (a b : Fin 6 → ℕ) :
  ∑ k, ((a + b) k : ℤ) * w k = (∑ k, (a k : ℤ) * w k) + ∑ k, (b k : ℤ) * w k
```

**lemma chargeSum\_single**[\[source\]](#)

*The charge sum of a single generator is the charge of that variable.*

```
lemma chargeSum_single (w : Fin 6 → ℤ) (i : Fin 6) :
  ∑ k, ((Finsupp.single i (1 : ℕ)) k : ℤ) * w k = w i
```

**lemma monomial\_mem\_adjoin\_neutralBilinear**[\[source\]](#)

*\*\*Generation.\*\* Every hypercharge-neutral monomial is a product of the five bilinears.*

```
lemma monomial_mem_adjoin_neutralBilinear (m : Fin 6 → ℕ)
  (hA : ∑ i, (m i : ℤ) * cartanCharge i = 0) (hB : ∑ i, (m i : ℤ) * hyperCharge i = 0) :
```

```
monomial m (l : ℂ) ∈ Algebra.adjoin ℂ (Set.range neutralBilinear)
```

### lemma Qslice\_mem\_adjoin\_neutralBilinear

[\[source\]](#)

*The slice potential lies in the subalgebra generated by the five bilinears: every monomial that survives is hypercharge-neutral, hence a product of the bilinears.*

```
lemma Qslice_mem_adjoin_neutralBilinear {V : EffectivePotential} (hI : IsInvariant V)
{P : MvPolynomial (Fin 6) ℝ} (hP : ∀ a, V (sliceR a) = P.eval a) :
Qslice P ∈ Algebra.adjoin ℂ (Set.range neutralBilinear)
```

### lemma exists\_aeval\_neutralBilinear

[\[source\]](#)

*Consequently the complexified slice potential is aeval neutralBilinear G for some polynomial G in the five bilinears.*

```
lemma exists_aeval_neutralBilinear {V : EffectivePotential} (hI : IsInvariant V)
{P : MvPolynomial (Fin 6) ℝ} (hP : ∀ a, V (sliceR a) = P.eval a) :
∃ G : MvPolynomial (Fin 5) ℂ, aeval neutralBilinear G = Qslice P
```

### def aRep

[\[source\]](#)

*The slice parameters realising repHiggs X as a point of the slice family.*

```
def aRep (X : Fin 4 → ℝ) : Fin 6 → ℝ
```

### lemma repHiggs\_eq\_sliceR

[\[source\]](#)

```
lemma repHiggs_eq_sliceR (X : Fin 4 → ℝ) : repHiggs X = sliceR (aRep X)
```

### def eigenPoint

[\[source\]](#)

*The hypercharge eigen-point  $(z, z^-, w_0, w_0^-, w_1, w_1^-)$  of repHiggs X: here  $z = X_0$  is real,  $w_0 = X_1 + i X_2$  and  $w_1 = X_3$  is real.*

```
noncomputable def eigenPoint (X : Fin 4 → ℝ) : Fin 6 → ℂ
```

### lemma aeval\_hyperchargeEigen\_eigenPoint

[\[source\]](#)

*The eigen-coordinate change sends the eigen-point of repHiggs X back to its slice parameters.*

```
lemma aeval_hyperchargeEigen_eigenPoint (X : Fin 4 → ℝ) (k : Fin 6) :
aeval (eigenPoint X) (hyperchargeEigen k) = algebraMap ℝ ℂ (aRep X k)
```

### lemma eval\_Qslice\_eigenPoint

[\[source\]](#)

*The complexified slice potential, evaluated at the eigen-point of repHiggs X, returns the real value V (repHiggs X).*

```
lemma eval_Qslice_eigenPoint (P : MvPolynomial (Fin 6) ℝ) (X : Fin 4 → ℝ) :
eval (eigenPoint X) (Qslice P) = algebraMap ℝ ℂ (P.eval (aRep X))
```



**def realPart**[\[source\]](#)

*The real part of a complex polynomial, taken coefficient-wise.*

```
noncomputable def realPart (H : MvPolynomial (Fin 5) ℂ) : MvPolynomial (Fin 5) ℝ
```

**lemma realPart\_C**[\[source\]](#)

```
lemma realPart_C (a : ℂ) : realPart (C a) = C a.re
```

**lemma realPart\_add**[\[source\]](#)

```
lemma realPart_add (p q : MvPolynomial (Fin 5) ℂ) :  
  realPart (p + q) = realPart p + realPart q
```

**lemma realPart\_mul\_X**[\[source\]](#)

```
lemma realPart_mul_X (p : MvPolynomial (Fin 5) ℂ) (i : Fin 5) :  
  realPart (p * X i) = realPart p * X i
```

**lemma realPart\_eval**[\[source\]](#)

*Evaluating a complex polynomial at a real point and taking the real part is the same as evaluating its real part.*

```
lemma realPart_eval (H : MvPolynomial (Fin 5) ℂ) (y : Fin 5 → ℝ) :  
  (eval (fun j => (↑(y j) : ℂ)) H).re = (realPart H).eval y
```

**def sliceBilinear**[\[source\]](#)

*The five real gauge-invariant bilinears, evaluated at repHiggs  $X$ :  $\|\Phi_1\|^2$ ,  $\text{Re}\langle\langle\cdot\rangle\rangle$ ,  $\text{Im}\langle\langle\cdot\rangle\rangle$ ,  $|\Phi_{2_0}|^2$ ,  $|\Phi_{2_1}|^2$ .*

```
def sliceBilinear (X : Fin 4 → ℝ) : Fin 5 → ℝ
```

**def transf**[\[source\]](#)

*The complex substitution expressing each bilinear, at the eigen-point, through the real generators (the off-diagonal pair  $z\ w_{\theta}^-$ ,  $z^-\ w_{\theta}$  mix  $\text{Re}\langle\langle\cdot\rangle\rangle$  and  $\text{Im}\langle\langle\cdot\rangle\rangle$ ).*

```
noncomputable def transf : Fin 5 → MvPolynomial (Fin 5) ℂ
```

**lemma neutralBilinear\_eval\_eigenPoint**[\[source\]](#)

*The bilinears at the eigen-point of repHiggs  $X$  are the real generators, read through transf.*

```
lemma neutralBilinear_eval_eigenPoint (X : Fin 4 → ℝ) (i : Fin 5) :  
  eval (eigenPoint X) (neutralBilinear i)  
    = eval (fun j => (↑(sliceBilinear X j) : ℂ)) (transf i)
```

**lemma exists\_polynomial\_repHiggs\_sliceBilinear**[\[source\]](#)

*\*\*The potential on representatives, in the bilinears.\*\* A gauge-invariant polynomial potential, on the representative family, is a polynomial in the five real gauge-invariant bilinears  $\|\Phi 1\|^2$ ,  $\text{Re}\langle\langle\cdot\rangle\rangle$ ,  $\text{Im}\langle\langle\cdot\rangle\rangle$ ,  $|\Phi 2_0|^2$ ,  $|\Phi 2_1|^2$ .*

```
lemma exists_polynomial_repHiggs_sliceBilinear {V : EffectivePotential} {n : ℕ}
  (hI : IsInvariant V) (h : HasMaxMassDimLE V n) :
  ∃ p : MvPolynomial (Fin 5) ℝ, ∀ X : Fin 4 → ℝ, V (repHiggs X) = p.eval (sliceBilinear X)
```

**def sliceBilinearPoly**[\[source\]](#)

*The five bilinear generators, as polynomials in the four representative parameters.*

```
noncomputable def sliceBilinearPoly : Fin 5 → MvPolynomial (Fin 4) ℝ
```

**def gramPoly**[\[source\]](#)

*The four Gram components, as polynomials in the four representative parameters.*

```
noncomputable def gramPoly : Fin 1 ⊕ Fin 3 → MvPolynomial (Fin 4) ℝ
```

**lemma exists\_clearing\_mem**[\[source\]](#)

*Some power of  $\|\Phi 1\|^2 = X_0^2$  times the value polynomial lies in the Gram subalgebra: multiplying by  $X_0^2$  pairs each  $X_1^2 + X_2^2$  into  $(X_0 X_1)^2 + (X_0 X_2)^2$  and each  $X_3^2$  into the determinant  $X_0^2 X_3^2$ , both of which are Gram polynomials.*

```
lemma exists_clearing_mem (p : MvPolynomial (Fin 5) ℝ) :
  ∃ N : ℕ, (X 0) ^ (2 * N) * aeval sliceBilinearPoly p ∈
  Algebra.adjoin ℝ (Set.range gramPoly)
```

**lemma exists\_gram\_clearing**[\[source\]](#)

*\*\*Denominator clearing.\*\* For the value polynomial p, some power of  $\|\Phi 1\|^2 = X_0^2$  times  $p \circ \text{sliceBilinear}$  is a polynomial in the Gram vector.*

```
lemma exists_gram_clearing (p : MvPolynomial (Fin 5) ℝ) :
  ∃ (A : MvPolynomial (Fin 1 ⊕ Fin 3) ℝ) (N : ℕ), ∀ X : Fin 4 → ℝ,
  (X 0) ^ (2 * N) * p.eval (sliceBilinear X) = A.eval ((repHiggs X).gramVector)
```

**lemma exists\_normSq\_Φ1\_clearing**[\[source\]](#)

*\*\* $\|\Phi 1\|^2$ -clearing, on all configurations.\*\* A power of  $\|\Phi 1\|^2$  times a gauge-invariant polynomial potential is, everywhere, a polynomial in the Gram vector.*

```
lemma exists_normSq_Φ1_clearing {V : EffectivePotential} {n : ℕ}
  (hI : IsInvariant V) (h : HasMaxMassDimLE V n) :
  ∃ (A : MvPolynomial (Fin 1 ⊕ Fin 3) ℝ) (N : ℕ), ∀ φ : TwoHiggsDoublet,
  ((φ.Φ1) ^ 2) ^ N * V φ = A.eval φ.gramVector
```

**def swapSubst**[\[source\]](#)

*The Gram-vector substitution induced by swapping the doublets (sign flip on the imaginary and difference components).*

```
noncomputable def swapSubst : (Fin 1 ⊕ Fin 3) → MvPolynomial (Fin 1 ⊕ Fin 3) ℝ
```

### lemma exists\_normSq\_ϕ2\_clearing

[\[source\]](#)

*\*\*||ϕ2||<sup>2</sup>-clearing, on all configurations.\*\* A power of ||ϕ2||<sup>2</sup> times a gauge-invariant polynomial potential is, everywhere, a polynomial in the Gram vector. Obtained from exists\_normSq\_ϕ1\_clearing for the doublet-swapped potential.*

```
lemma exists_normSq_ϕ2_clearing {V : EffectivePotential} {n : ℕ}
(hI : IsInvariant V) (h : HasMaxMassDimLE V n) :
∃ (B : MvPolynomial (Fin 1 ⊕ Fin 3) ℝ) (M : ℕ), ∀ ϕ : TwoHiggsDoublet,
(||ϕ.ϕ2|| ^ 2) ^ M * V ϕ = B.eval ϕ.gramVector
```

### lemma gramPoly\_injective

[\[source\]](#)

*The four Gram invariants are algebraically independent: the Gram substitution is injective.*

```
lemma gramPoly_injective :
Function.Injective
(aeval gramPoly : MvPolynomial (Fin 1 ⊕ Fin 3) ℝ → MvPolynomial (Fin 4) ℝ)
```

### lemma uPow\_dvd

[\[source\]](#)

*||ϕ1||<sup>2</sup> and ||ϕ2||<sup>2</sup>, as the distinct linear forms (g<sub>0</sub>±g<sub>3</sub>)/2 of the Gram ring, are coprime: if ||ϕ1||<sup>2N</sup> · B = ||ϕ2||<sup>2M</sup> · A then ||ϕ1||<sup>2N</sup> | A.*

```
lemma uPow_dvd {N M : ℕ} {A B : MvPolynomial (Fin 1 ⊕ Fin 3) ℝ}
(hAB : (C (1 / 2) * (X (Sum.inl 0) + X (Sum.inr 2))) ^ N * B
= (C (1 / 2) * (X (Sum.inl 0) - X (Sum.inr 2))) ^ M * A) :
(C (1 / 2) * (X (Sum.inl 0) + X (Sum.inr 2))) ^ N | A
```

### lemma exists\_polynomial\_on\_repHiggs

[\[source\]](#)

*\*\*On representatives.\*\* A gauge-invariant polynomial potential, restricted to the polynomial family of orbit representatives repHiggs X, is a polynomial in the Gram components of that family. Proved by the doublet-swap argument: clearing the ||ϕ1||<sup>2</sup> factor (aligning ϕ1) and the ||ϕ2||<sup>2</sup> factor (aligning ϕ2, via the gauge-commuting swap), then using that ||ϕ1||<sup>2</sup> and ||ϕ2||<sup>2</sup> are coprime in the Gram ring.*

```
lemma exists_polynomial_on_repHiggs {V : EffectivePotential} {n : ℕ}
(hI : IsInvariant V) (h : HasMaxMassDimLE V n) :
∃ p : MvPolynomial (Fin 1 ⊕ Fin 3) ℝ,
∀ X : Fin 4 → ℝ, V (repHiggs X) = p.eval (repHiggs X).gramVector
```

### lemma effectivePotential\_is\_polynomial\_gramVector

[\[source\]](#)

*\*\*The two Higgs doublet potential in the bilinear formalism.\*\* A gauge-invariant polynomial effective potential of maximum mass dimension n is a polynomial in the four gauge-invariant bilinears — the entries of the Gram vector.*

```
lemma effectivePotential_is_polynomial_gramVector {V : EffectivePotential} {n : ℕ}
(hI : IsInvariant V) (h : HasMaxMassDimLE V n) :
∃ p : MvPolynomial (Fin 1 ⊕ Fin 3) ℝ, (∀ ϕ : TwoHiggsDoublet, V ϕ = p.eval ϕ.gramVector)
```

## 2.106 PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.Module

[PhyslibAlpha/Particles/BeyondTheStandardModel/TwoHDM/Module.lean](#) on GitHub.

**instance Add TwoHiggsDoublet** [\[source\]](#)

```
instance : Add TwoHiggsDoublet where
```

**lemma add\_fst** [\[source\]](#)

```
@[simp]
lemma add_fst (H1 H2 : TwoHiggsDoublet) : (H1 + H2).ϕ1 = H1.ϕ1 + H2.ϕ1
```

**lemma add\_snd** [\[source\]](#)

```
@[simp]
lemma add_snd (H1 H2 : TwoHiggsDoublet) : (H1 + H2).ϕ2 = H1.ϕ2 + H2.ϕ2
```

**instance Zero TwoHiggsDoublet** [\[source\]](#)

```
instance : Zero TwoHiggsDoublet where
```

**lemma zero\_fst** [\[source\]](#)

```
@[simp]
lemma zero_fst : (0 : TwoHiggsDoublet).ϕ1 = 0
```

**lemma zero\_snd** [\[source\]](#)

```
@[simp]
lemma zero_snd : (0 : TwoHiggsDoublet).ϕ2 = 0
```

**instance SMul ℂ TwoHiggsDoublet** [\[source\]](#)

```
instance : SMul ℂ TwoHiggsDoublet where
```

**lemma smul\_fst** [\[source\]](#)

```
@[simp]
lemma smul_fst (c : ℂ) (H : TwoHiggsDoublet) : (c • H).ϕ1 = c • H.ϕ1
```

**lemma smul\_snd** [\[source\]](#)

```
@[simp]
lemma smul_snd (c : ℂ) (H : TwoHiggsDoublet) : (c • H).ϕ2 = c • H.ϕ2
```

**instance Neg TwoHiggsDoublet** [\[source\]](#)

```
instance : Neg TwoHiggsDoublet where
```

**lemma neg\_fst**[\[source\]](#)

```
@[simp]
lemma neg_fst (H : TwoHiggsDoublet) : (-H).Φ1 = -H.Φ1
```

**lemma neg\_snd**[\[source\]](#)

```
@[simp]
lemma neg_snd (H : TwoHiggsDoublet) : (-H).Φ2 = -H.Φ2
```

**instance AddCommGroup TwoHiggsDoublet**[\[source\]](#)

```
instance : AddCommGroup TwoHiggsDoublet where
```

**instance Module ℂ TwoHiggsDoublet**[\[source\]](#)

```
instance : Module ℂ TwoHiggsDoublet where
```

**2.107 PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.OrbitRepresentative**

[PhyslibAlpha/Particles/BeyondTheStandardModel/TwoHDM/OrbitRepresentative.lean](#) on GitHub.

**def repHiggs**[\[source\]](#)

*A four-real-parameter polynomial family of representatives for the gauge orbits: the first doublet is  $(X_0, \theta)$  and the second is  $(X_1 + i X_2, X_3)$ .*

```
def repHiggs (X : Fin 4 → ℝ) : TwoHiggsDoublet where
```

**lemma repHiggs\_Φ1**[\[source\]](#)

```
@[simp]
lemma repHiggs_Φ1 (X : Fin 4 → ℝ) : (repHiggs X).Φ1 = !₂[(X 0 : ℂ), 0]
```

**lemma repHiggs\_Φ2**[\[source\]](#)

```
@[simp]
lemma repHiggs_Φ2 (X : Fin 4 → ℝ) :
  (repHiggs X).Φ2 = !₂[(X 1 : ℂ) + Complex.I * (X 2 : ℂ), (X 3 : ℂ)]
```

**lemma normSq\_repHiggs\_Φ1**[\[source\]](#)

```
lemma normSq_repHiggs_Φ1 (X : Fin 4 → ℝ) : ||(repHiggs X).Φ1|| ^ 2 = X 0 ^ 2
```

**lemma normSq\_repHiggs\_Φ2**[\[source\]](#)

```
lemma normSq_repHiggs_Φ2 (X : Fin 4 → ℝ) :
  ||(repHiggs X).Φ2|| ^ 2 = X 1 ^ 2 + X 2 ^ 2 + X 3 ^ 2
```

**lemma inner\_repHiggs**[\[source\]](#)

```
lemma inner_repHiggs (X : Fin 4 → ℝ) :
  ⟨⟨(repHiggs X).Φ1, (repHiggs X).Φ2⟩⟩_ℂ = (X 0 : ℂ) * ((X 1 : ℂ) + Complex.I * (X 2 : ℂ))
```

**lemma gramVector\_repHiggs\_inl**[\[source\]](#)

```
@[simp]
lemma gramVector_repHiggs_inl (X : Fin 4 → ℝ) :
  (repHiggs X).gramVector (Sum.inl 0) = X 0 ^ 2 + (X 1 ^ 2 + X 2 ^ 2 + X 3 ^ 2)
```

**lemma gramVector\_repHiggs\_inr0**[\[source\]](#)

```
@[simp]
lemma gramVector_repHiggs_inr0 (X : Fin 4 → ℝ) :
  (repHiggs X).gramVector (Sum.inr 0) = 2 * X 0 * X 1
```

**lemma gramVector\_repHiggs\_inr1**[\[source\]](#)

```
@[simp]
lemma gramVector_repHiggs_inr1 (X : Fin 4 → ℝ) :
  (repHiggs X).gramVector (Sum.inr 1) = 2 * X 0 * X 2
```

**lemma gramVector\_repHiggs\_inr2**[\[source\]](#)

```
@[simp]
lemma gramVector_repHiggs_inr2 (X : Fin 4 → ℝ) :
  (repHiggs X).gramVector (Sum.inr 2) = X 0 ^ 2 - (X 1 ^ 2 + X 2 ^ 2 + X 3 ^ 2)
```

**lemma exists\_smul\_eq\_repHiggs**[\[source\]](#)

*Every field configuration is in the gauge orbit of a representative `repHiggs X`. This is the key fact that lets a gauge invariant potential be evaluated on the polynomial family.*

```
lemma exists_smul_eq_repHiggs (φ : TwoHiggsDoublet) :
  ∃ (X : Fin 4 → ℝ) (g : GaugeGroupI), g • φ = repHiggs X
```

**2.108 PhyslibAlpha.Particles.BeyondTheStandardModel.TwoHDM.SwapDoublet**

[PhyslibAlpha/Particles/BeyondTheStandardModel/TwoHDM/SwapDoublet.lean](#) on GitHub.

**def swapDoublet**[\[source\]](#)

*Swapping the two doublets, as an  $\mathbb{R}$ -linear map. It commutes with the gauge action, so it sends gauge-invariant polynomial potentials to gauge-invariant polynomial potentials, but turns the alignment of  $\Phi 1$  into the alignment of  $\Phi 2$ .*

```
def swapDoublet : TwoHiggsDoublet →ℓ[ℝ] TwoHiggsDoublet where
```

**lemma swapDoublet\_smul**[\[source\]](#)

```
lemma swapDoublet_smul (g : StandardModel.GaugeGroupI) (φ : TwoHiggsDoublet) :
  swapDoublet (g • φ) = g • swapDoublet φ
```

**lemma gramVector\_swapDoublet\_inl**[\[source\]](#)

*Swapping the doublets sends the gram vector through the sign flip of the imaginary and difference components.*

```
lemma gramVector_swapDoublet_inl (φ : TwoHiggsDoublet) :
  (swapDoublet φ).gramVector (Sum.inl 0) = φ.gramVector (Sum.inl 0)
```

**lemma gramVector\_swapDoublet\_inr0**[\[source\]](#)

```
lemma gramVector_swapDoublet_inr0 (φ : TwoHiggsDoublet) :
  (swapDoublet φ).gramVector (Sum.inr 0) = φ.gramVector (Sum.inr 0)
```

**lemma gramVector\_swapDoublet\_inr1**[\[source\]](#)

```
lemma gramVector_swapDoublet_inr1 (φ : TwoHiggsDoublet) :
  (swapDoublet φ).gramVector (Sum.inr 1) = -φ.gramVector (Sum.inr 1)
```

**lemma gramVector\_swapDoublet\_inr2**[\[source\]](#)

```
lemma gramVector_swapDoublet_inr2 (φ : TwoHiggsDoublet) :
  (swapDoublet φ).gramVector (Sum.inr 2) = -φ.gramVector (Sum.inr 2)
```

**lemma HasMaxMassDimLE.comp\_swapDoublet**[\[source\]](#)

```
lemma HasMaxMassDimLE.comp_swapDoublet {V : EffectivePotential} {n : ℕ}
  (h : HasMaxMassDimLE V n) : HasMaxMassDimLE (fun φ => V (swapDoublet φ)) n
```

**lemma IsInvariant.comp\_swapDoublet**[\[source\]](#)

```
lemma IsInvariant.comp_swapDoublet {V : EffectivePotential} (hI : IsInvariant V) :
  IsInvariant (fun φ => V (swapDoublet φ))
```

**2.109 QuantumInfo.ClassicalInfo.Prob**

[QuantumInfo/ClassicalInfo/Prob.lean](#) on GitHub.

**theorem mix\_one**[\[source\]](#)

```
@[simp]
theorem mix_one [inst : Mixable U T] (x₁ x₂ : T) : (1 : Prob) [ x₁ ↔ x₂ : inst] = x₁
```

**2.110 QuantumInfo.Entropy.DPI**

[QuantumInfo/Entropy/DPI.lean](#) on GitHub.

**theorem sandwichedRelEntropy\_tendsto\_qRelativeEnt**[\[source\]](#)

As  $\alpha \rightarrow 1^+$ , the sandwiched Rényi relative entropy  $D_{\alpha}(\rho||\sigma)$  tends to the relative entropy  $D(\rho||\sigma) = D_1(\rho||\sigma)$ , by continuity of  $\alpha \mapsto D_{\alpha}$  on  $(0, \infty)$ .

```
theorem sandwichedRelEntropy_tendsto_qRelativeEnt (p σ : MState d) :
  Filter.Tendsto (fun α : ℝ => D_α (p||σ)) (N[>] 1) (N D (p||σ))
```

**lemma sandwichedTraceFunctional\_sub\_one\_div\_eventually\_le**[\[source\]](#)

As  $\alpha \rightarrow 1^+$ , the difference quotient  $(Q_{\alpha}(\rho||\sigma) - 1) / (\alpha - 1)$  is eventually bounded above by a function tending to  $D(\rho||\sigma).toReal$ . This is the key estimate for transferring joint convexity of the trace functional  $Q_{\alpha}$  to the relative entropy  $D$  in `qRelativeEnt_joint_convexity`.

```
private lemma sandwichedTraceFunctional_sub_one_div_eventually_le
  (p σ : MState d) (hker : σ.M.ker ≤ p.M.ker) :
  ∃ u : ℝ → ℝ, Filter.Tendsto u (N[>] 1) (N (D (p||σ)).toReal) ∧
  ∀f α in N[>] 1, (Q_α (p||σ) - 1) / (α - 1) ≤ u α
```

**lemma mix\_M\_eq\_weighted\_sum**[\[source\]](#)

A binary Mixable mixture of states, written as a weighted sum of matrices over `Fin 2` — the form consumed by `sandwichedTraceFunctional_jointly_convex` and `HermitianMat.ker_weighted_sum_le`.

```
private lemma mix_M_eq_weighted_sum (p : Prob) (τ₁ τ₂ : MState d) :
  (p [τ₁ ↔ τ₂]).M = ∑ i, ![(p : ℝ), 1 - (p : ℝ)] i • (![τ₁, τ₂] i).M
```

**lemma ker\_mix\_le**[\[source\]](#)

A binary mixture preserves the support condition (kernel inclusion) of its components.

```
private lemma ker_mix_le (p : Prob) {ρ₁ ρ₂ σ₁ σ₂ : MState d}
  (hker₁ : σ₁.M.ker ≤ ρ₁.M.ker) (hker₂ : σ₂.M.ker ≤ ρ₂.M.ker) :
  (p [σ₁ ↔ σ₂]).M.ker ≤ (p [ρ₁ ↔ ρ₂]).M.ker
```

**lemma sandwichedTraceFunctional\_mix\_le**[\[source\]](#)

Binary case of the joint convexity of the trace functional  $Q_{\alpha}$  for  $\alpha > 1$  (`sandwichedTraceFunctional_jointly_convex` stated for a Mixable mixture).

```
private lemma sandwichedTraceFunctional_mix_le (hα : 1 < α) (p : Prob)
  {ρ₁ ρ₂ σ₁ σ₂ : MState d}
  (hker₁ : σ₁.M.ker ≤ ρ₁.M.ker) (hker₂ : σ₂.M.ker ≤ ρ₂.M.ker) :
  Q_α (p [ρ₁ ↔ ρ₂]||p [σ₁ ↔ σ₂]) ≤
  (p : ℝ) * Q_α (ρ₁||σ₁) + (1 - (p : ℝ)) * Q_α (ρ₂||σ₂)
```

**2.111 QuantumInfo.Entropy.Relative**

[QuantumInfo/Entropy/Relative.lean](#) on GitHub.

**theorem qRelativeEnt\_ne\_top\_iff**[\[source\]](#)



*The quantum relative entropy is finite exactly when the support condition  $\sigma.M.ker \leq \rho.M.ker$  holds.*

```
theorem qRelativeEnt_ne_top_iff { $\rho \ \sigma$  : MState d} :  $D(\rho||\sigma) \neq \tau \leftrightarrow \sigma.M.ker \leq \rho.M.ker$ 
```

**theorem qRelativeEnt\_eq\_top\_iff**

[\[source\]](#)

*The quantum relative entropy is  $\tau$  exactly when the support condition fails.*

```
theorem qRelativeEnt_eq_top_iff { $\rho \ \sigma$  : MState d} :  $D(\rho||\sigma) = \tau \leftrightarrow \neg \sigma.M.ker \leq \rho.M.ker$ 
```

## 2.112 scripts.autolabel

[scripts/autolabel.lean](#) on GitHub.

**def MAX\_LABELS**

[\[source\]](#)

*Maximal number of labels which can be added. If more are applicable, nothing will be added.*

```
def MAX_LABELS
```

**inductive Label**

[\[source\]](#)

*Physlib's Github topic labels*

```
inductive Label where
  | «t-classical-field-theory»
  | «t-classical-mechanics»
  | «t-condensed-matter»
  | «t-cosmology»
  | «t-electromagnetism»
  | «t-fluid-dynamics»
  | «t-mathematics»
  | «t-meta»
  | «t-optics»
  | «t-particles»
  | «t-qft»
  | «t-quantum-mechanics»
  | «t-relativity»
  | «t-space-and-time»
  | «t-statistical-mechanics»
  | «t-string-theory»
  | «t-thermodynamics»
  | «t-units»
  | «t-capacity-qi»
  | «t-channels-qi»
  | «t-classical-info-qi»
  | «t-entropy-qi»
  | «t-for-mathlib-qi»
  | «t-measurements-qi»
  | «t-operators-qi»
  | «t-resource-theory-qi»
  | «t-states-qi»
  | «CI»
deriving BEq, Hashable, Repr
```

**def physlibLabels**[\[source\]](#)

```
def physlibLabels : Array Label
```

**def toString**[\[source\]](#)

```
def Label.toString : Label → String
| «t-classical-field-theory»      => "t-classical-field-theory"
| «t-classical-mechanics»        => "t-classical-mechanics"
| «t-condensed-matter»          => "t-condensed-matter"
| «t-cosmology»                  => "t-cosmology"
| «t-electromagnetism»           => "t-electromagnetism"
| «t-fluid-dynamics»             => "t-fluid-dynamics"
| «t-mathematics»                => "t-mathematics"
| «t-meta»                       => "t-meta"
| «t-optics»                     => "t-optics"
| «t-particles»                  => "t-particles"
| «t-qft»                        => "t-qft"
| «t-quantum-mechanics»          => "t-quantum-mechanics"
| «t-relativity»                 => "t-relativity"
| «t-space-and-time»             => "t-space-and-time"
| «t-statistical-mechanics»      => "t-statistical-mechanics"
| «t-string-theory»              => "t-string-theory"
| «t-thermodynamics»            => "t-thermodynamics"
| «t-units»                      => "t-units"
| «t-capacity-qi»                => "t-capacity-qi"
| «t-channels-qi»                => "t-channels-qi"
| «t-classical-info-qi»          => "t-classical-info-qi"
| «t-entropy-qi»                 => "t-entropy-qi"
| «t-for-mathlib-qi»             => "t-for-mathlib-qi"
| «t-measurements-qi»           => "t-measurements-qi"
| «t-operators-qi»              => "t-operators-qi"
| «t-resource-theory-qi»         => "t-resource-theory-qi"
| «t-states-qi»                  => "t-states-qi"
| «CI»                           => "CI"
```

**instance ToString Label**[\[source\]](#)

```
instance : ToString Label where
```

**structure LabelData**[\[source\]](#)

*A LabelData consists of the*

- \* The dirs field is the array of all "root paths" such that a modification in a file contained in one of these paths should be labelled with label.*
- \* The exclusions field is the array of all "root paths" that are excluded, among the ones that start with the ones in dirs. Any modifications to a file in an excluded path is ignored for the purposes of labelling.*
- \* The dependencies field is the array of all labels, which are lower in the import hierarchy and which should be excluded if the label is present.*

```
structure LabelData (label : Label) where
  dirs : Array FilePath := if label.toString.startsWith "t-" && label.toString.endsWith "-qi"
  then
    #["QuantumInfo" / ("".intercalate (label.toString.splitOn "-" |>.drop 1 |>.dropLast |>.
    map .capitalize))]
  else if label.toString.startsWith "t-" then
```

```

    #["Physlib" / ("".intercalate (label.toString.splitOn "-" |>.drop 1 |>.map .capitalize))]
    else #[]
exclusions : Array FilePath := #[]
dependencies : Array Label := #[]
deriving BEq, Hashable

```

**def physlibLabelData**[\[source\]](#)

*This function maps each label to the corresponding paths For now, no exclusions or dependencies are specified -*

```

def physlibLabelData: (l: Label) → LabelData l
| .«t-classical-field-theory» => {}
| .«t-classical-mechanics» => {}
| .«t-condensed-matter» => {}
| .«t-cosmology» => {}
| .«t-electromagnetism» => {}
| .«t-fluid-dynamics» => {}
| .«t-mathematics» => {}
| .«t-meta» => {}
| .«t-optics» => {}
| .«t-particles» => {}
| .«t-qft» => { dirs := #["Physlib" / "QFT"] }
| .«t-quantum-mechanics» => {}
| .«t-relativity» => {}
| .«t-space-and-time» => {}
| .«t-statistical-mechanics» => {}
| .«t-string-theory» => {}
| .«t-thermodynamics» => {}
| .«t-units» => {}
| .«t-capacity-qi» => {}
| .«t-channels-qi» => {}
| .«t-classical-info-qi» => {}
| .«t-entropy-qi» => {}
| .«t-for-mathlib-qi» => {}
| .«t-measurements-qi» => {}
| .«t-operators-qi» => {}
| .«t-resource-theory-qi» => {}
| .«t-states-qi» => {}
| .«CI» => { dirs := #["scripts"] }

```

**def physlibUnlabelled**[\[source\]](#)

*Exceptions inside Physlib/ which are not covered by any label. (For the First versions, no exceptions)*

```
def physlibUnlabelled : Array FilePath
```

**def quantuminfoUnlabelled**[\[source\]](#)

*Exceptions inside QuantumInfo/ which are not covered by any label. (For the First versions, no exceptions)*

```
def quantuminfoUnlabelled : Array FilePath
```

**def isPrefixOf**[\[source\]](#)

*Checks if the folder `path` lies inside the folder `dir`. Copied verbatim from `mathlib/scripts/autolabel.lean`*

```
def _root_.System.FilePath.isPrefixOf (dir path : FilePath) : Bool
```

**def getMatchingLabels**[\[source\]](#)

*Return all labels in `physlibLabels` which match at least one of the files. \* files: array of relative paths starting from the `physlib` root directory.*

```
def getMatchingLabels (files : Array FilePath) : Array Label
```

**def collectLabelsAndDependentLabels**[\[source\]](#)

*Helper function: union of all labels and all their dependent labels*

```
partial def collectLabelsAndDependentLabels (labels: Array Label) : Array Label
```

**def dropDependentLabels**[\[source\]](#)

*Reduce a list of labels to not include any which are dependencies of other labels in the list*

```
def dropDependentLabels (labels: Array Label) : Array Label
```

**def findUncoveredPaths**[\[source\]](#)

*Testing function to ensure the labels defined in `physlibLabels` cover all subfolders of `Physlib/` and `QuantumInfo/`.*

```
partial def findUncoveredPaths (path : FilePath) (exceptions : Array FilePath := #[]) :  
  IO <| Array FilePath
```

**def githubAnnotation**[\[source\]](#)

*Create a message which GitHub CI parses as annotation and displays at the specified file.*

*Note: file is duplicated below so that it is also visible in the plain text output.*

*\* type: "error" or "warning" \* file: file where the annotation should be displayed  
\* title: title of the annotation \* message: annotation message*

```
def githubAnnotation (type file title message : String) : String
```

**def main**[\[source\]](#)

*args is expected to have length 0 or 1, where the first argument is the PR number.*

*If a PR number is provided, the script requires GitHub CLI `gh` to be installed in order to add the label to the PR.*

*## Exit codes:*

- 0: success - 1: invalid arguments provided - 2: invalid labels defined - 3: ~labels do not cover all of *Physlib*/~ (unused; only emitting warning)

```
unsafe def main (args : List String): IO UInt32
```